Angular Learning Series

Topics 20 - 26 | Sections 0 - 12

Angular Core: RxJS, Reactivity & Components

Contents

Topic 20 -- RxJS Subscription Management in Angular

0. Difficulty & Priority Rating

* Difficulty: Intermediate -- creating a subscription is one line; understanding why unmanaged ones cause memory leaks and knowing the three modern cleanup patterns takes real practice to fully internalize

* Angular Relevance: Critical -- every HTTP call, every form valueChanges, every real-time stream in Angular produces an Observable that needs subscribing and cleaning up; one forgotten subscription silently keeps a destroyed component alive in memory forever

1. Explanation

The Newspaper Subscription Analogy

Imagine you subscribe to a daily newspaper delivery. Every morning it arrives at your door. Then you move house -- but forget to cancel the subscription. The newspaper keeps arriving at your old address. The delivery van keeps going there. The printing press keeps printing your copy. Resources consumed indefinitely for a house nobody lives in.

An unmanaged Angular subscription is that forgotten newspaper delivery.

You navigate away from a component -- Angular destroys it. But if its Observable subscriptions weren't cancelled, they keep running. HTTP callbacks try to update a component that no longer exists. Event streams keep firing into the void. Memory that should be freed stays permanently occupied.

```
Subscribe = start the newspaper delivery

Unsubscribe = cancel when you move house

Component destroyed without unsubscribe

= delivery to an empty house forever
```

The Tap Left Running Analogy

Picture a tap left running after washing up. Water keeps flowing -- down the drain, wasted. In a large app with dozens of components subscribing and never unsubscribing, it's dozens of taps all running simultaneously.

Memory leaks are taps that were never turned off.

Each unmanaged subscription holds a reference to its component, preventing the garbage collector from freeing memory. Over time, navigating back and forth creates more ghost components -- all still receiving data, all consuming memory, all potentially causing unexpected errors.

In Plain English

* Subscription = the active connection between an Observable and its observer -- like newspaper delivery being active

* Unsubscribe = cancelling the delivery -- Observable stops sending, memory is freed

* Memory leak = component is "destroyed" but subscriptions keep it alive in memory -- tap left running

* takeUntilDestroyed = Angular 16+ operator that automatically cancels subscriptions when the component is destroyed -- the smart tap that turns itself off

* async pipe = lets Angular manage the entire subscription lifecycle in the template -- Angular subscribes on create, unsubscribes on destroy

Now the Technical Terms

* Subscription = the object returned by .subscribe() -- calling .unsubscribe() on it cancels the Observable

* ngOnDestroy = Angular lifecycle hook where manual cleanup belongs

* takeUntilDestroyed() = RxJS interop operator from Angular 16 that completes the Observable when DestroyRef fires

* DestroyRef = Angular service that notifies when the current injection context is destroyed

* async pipe = template pipe that subscribes and automatically unsubscribes on component destruction

---

2. Connection to Prior Concepts

Subscription management is where the first 19 topics converge in practice:

* Topic 3 (Closures) -- every .subscribe() callback is a closure over this. Without cleanup, the closure keeps the component's entire scope alive in memory -- the tent that never came down. This is the exact memory leak mechanism from the closure topic

* Topic 5 (Event Loop) -- subscriptions receive data from the queue asynchronously. An uncleared subscription keeps putting callbacks on the queue even after the component is gone -- the waiter keeps serving a table that left the restaurant

* Topic 6 (Callbacks) -- .subscribe({ next, error, complete }) are the three callbacks you're managing. Unsubscribing stops all three from being called

* Topic 18 (Error Handling) -- catchError in the pipe protects the stream; the error handler in subscribe catches anything that reaches it. Both layers needed for resilient subscriptions

* Topic 19 (. and) -- unmanaged subscriptions cause Cannot set property of null errors -- the component is destroyed but the callback fires. . doesn't fix this -- unsubscribing does

Bridging note for your records:

"Every .subscribe() you write creates a tap. The three modern ways to turn it off: takeUntilDestroyed (automatic, recommended), async pipe (template-level, cleanest), manual unsubscribe() in ngOnDestroy (always valid). The goal is simple: if the component is gone, the subscription must be gone too."

---

3. Code Example

Example 1 -- The problem -- what a memory leak looks like

Showing exactly what happens without cleanup -- so you recognise it when you see it.

```
// THE LEAK subscription created, never cancelled


@Component({ selector: 'app-user-list', template: '...' })

export class UserListComponent implements OnInit {


users: User[] = [];
```

```
constructor(private readonly userService: UserService) {}

ngOnInit(): void {

// One-shot HTTP technically completes, low risk

this.userService.getUsers().subscribe(users =&gt; {

this.users = users;

});

// Polling stream NEVER completes definite leak

interval(5000).pipe(

switchMap(() =&gt; this.userService.getUsers())

).subscribe(users =&gt; {

this.users = users; // fires every 5 seconds forever

});

}

// No ngOnDestroy no cleanup

// User navigates away component "destroyed" by Angular

// BUT: interval keeps ticking callback keeps firing

// ERROR: Cannot set properties of null

// Memory: component stays alive GC cannot collect it

}
```

Example 2 -- Manual unsubscribe -- the baseline pattern
Always works -- valid in all Angular versions -- what you fall back to.

```
// MANUAL UNSUBSCRIBE traditional, always valid

import { Subscription } from 'rxjs';

@Component({ selector: 'app-user-list', template: '...' })

export class UserListComponent implements OnInit, OnDestroy {

users: User[] = [];
```

```
// One Subscription object collects all subscriptions

private readonly sub = new Subscription();


constructor(private readonly userService: UserService) {}


ngOnInit(): void {


// .add() collects multiple subscriptions into one

this.sub.add(

this.userService.getUsers().subscribe(users =&gt; {

this.users = users;

})

);


this.sub.add(

this.userService.getNotifications().subscribe(notif =&gt; {

this.notifications = notif;

})

);


this.sub.add(

interval(5000).pipe(

switchMap(() =&gt; this.userService.getUsers())

).subscribe(users =&gt; {

this.users = users;

})

);

}


ngOnDestroy(): void {

this.sub.unsubscribe(); // one call cancels ALL collected subscriptions

}

}
```

Example 3 -- takeUntil -- the pre-Angular-16 pattern
Still widely used in existing codebases -- you will see this everywhere.

```
// takeUntil pre-Angular 16 pattern

// Common in existing Angular 13/14/15 codebases


import { Subject, takeUntil } from 'rxjs';


@Component({ selector: 'app-user-list', template: '...' })

export class UserListComponent implements OnInit, OnDestroy {


users: User[] = [];


// A Subject that fires once as a "shutdown signal"

private readonly destroy$ = new Subject&lt;void&gt;();


constructor(private readonly userService: UserService) {}


ngOnInit(): void {


this.userService.getUsers().pipe(

takeUntil(this.destroy$) // "keep going UNTIL destroy$ fires"

).subscribe(users =&gt; { this.users = users; });


interval(5000).pipe(

switchMap(() =&gt; this.userService.getUsers()),

takeUntil(this.destroy$) // same signal all cancelled together

).subscribe(users =&gt; { this.users = users; });

}


ngOnDestroy(): void {

this.destroy$.next(); // fire the shutdown signal

this.destroy$.complete(); // clean up the Subject itself

// All takeUntil(this.destroy$) subscriptions cancel automatically

}

}
```

Angular Example -- takeUntilDestroyed -- the modern recommended pattern
Angular 16+ -- least boilerplate, automatic, impossible to forget.

```
// takeUntilDestroyed Angular 16+ recommended pattern

import { takeUntilDestroyed } from '@angular/core/rxjs-interop';

import { DestroyRef, inject } from '@angular/core';

@Component({

selector: 'app-order-list',

standalone: true,

imports: [CommonModule],

template: `

&lt;div *ngIf="isLoading"&gt;Loading orders...&lt;/div&gt;

@for (order of orders; track order.id) {

&lt;div&gt;{{ order.id }} {{ order.status }}&lt;/div&gt;

}

`

})

export class OrderListComponent implements OnInit {

orders: Order[] = [];

isLoading = false;

// inject DestroyRef works anywhere in the component

private readonly destroyRef = inject(DestroyRef);

constructor(

private readonly orderService: OrderService,

private readonly userService: UserService

) {}

ngOnInit(): void {

this.isLoading = true;
```

```
// All subscriptions use the same destroyRef all auto-cancelled

this.orderService.getOrders().pipe(

takeUntilDestroyed(this.destroyRef)

).subscribe({

next: orders =&gt; { this.orders = orders; this.isLoading = false; },

error: () =&gt; { this.isLoading = false; },

complete: () =&gt; { this.isLoading = false; }

});


this.userService.getCurrentUser().pipe(

takeUntilDestroyed(this.destroyRef)

).subscribe(user =&gt; { this.currentUser = user; });


// Polling timer + switchMap + takeUntilDestroyed

timer(0, 30_000).pipe(

switchMap(() =&gt; this.orderService.getOrders()),

takeUntilDestroyed(this.destroyRef) // interval AND HTTP both cancelled

).subscribe(orders =&gt; { this.orders = orders; });

}


// No ngOnDestroy needed for subscription cleanup

}



// ---- EVEN CLEANER without explicit destroyRef ----

// When called inside an injection context (field initialiser)

// takeUntilDestroyed() infers destroyRef automatically


@Component({ selector: 'app-dashboard', template: '...' })

export class DashboardComponent {


// Field initialiser IS an injection context no argument needed

private readonly data$ = this.dataService.getData().pipe(
```

```
takeUntilDestroyed() // auto-infers DestroyRef

);


constructor(private readonly dataService: DataService) {}

}
```

4. Before & After Refactor

The Problem in Plain English:

Imagine hiring contractors for a renovation but never telling them when to stop. Each contractor keeps working indefinitely -- long after you've moved out. The new owners are baffled. Resources drain away. Nobody gave the contractors a finish date.

Every unmanaged subscription is a contractor who was never told the job is done. takeUntilDestroyed is the project manager who automatically dismisses all contractors the moment the project closes.

```
// BEFORE subscriptions scattered, manually tracked, easy to miss one


@Component({ selector: 'app-profile', template: '...' })

export class ProfileComponent implements OnInit, OnDestroy {


user: User | null = null;

notifications: Notification[] = [];

stats: UserStats | null = null;

isLoading = false;


// Three separate variables what if a fourth is added and forgotten

private userSub: Subscription;

private notifSub: Subscription;

private statsSub: Subscription;


constructor(

private readonly userService: UserService,

private readonly notifService: NotificationService,

private readonly statsService: StatsService

) {}


ngOnInit(): void {
```

```
this.isLoading = true;

this.userSub = this.userService.getCurrentUser()

.subscribe(user =&gt; { this.user = user; });

this.notifSub = this.notifService.getNotifications()

.subscribe(notif =&gt; { this.notifications = notif; });

this.statsSub = this.statsService.getUserStats()

.subscribe(stats =&gt; { this.stats = stats; this.isLoading = false; });

// Fourth subscription added later developer forgot to track it

interval(60_000).pipe(

switchMap(() =&gt; this.statsService.getUserStats())

).subscribe(stats =&gt; {

this.stats = stats; // NO cleanup memory leak hiding in plain sight

});

}

ngOnDestroy(): void {

// Three unsubscribes interval still running

this.userSub .unsubscribe();

this.notifSub .unsubscribe();

this.statsSub .unsubscribe();

}

}



// AFTER takeUntilDestroyed automatic, consistent, impossible to miss

@Component({ selector: 'app-profile', template: '...' })

export class ProfileComponent implements OnInit {

user: User | null = null;
```

```
notifications: Notification[] = [];

stats: UserStats | null = null;

isLoading = false;


private readonly destroyRef = inject(DestroyRef);


constructor(

private readonly userService: UserService,

private readonly notifService: NotificationService,

private readonly statsService: StatsService

) {}


ngOnInit(): void {

this.isLoading = true;


// Identical pattern for every subscription impossible to forget

this.userService.getCurrentUser().pipe(

takeUntilDestroyed(this.destroyRef)

).subscribe(user =&gt; { this.user = user; });


this.notifService.getNotifications().pipe(

takeUntilDestroyed(this.destroyRef)

).subscribe(notif =&gt; { this.notifications = notif; });


this.statsService.getUserStats().pipe(

takeUntilDestroyed(this.destroyRef)

).subscribe(stats =&gt; { this.stats = stats; this.isLoading = false; });


// Fourth subscription same pattern automatically cleaned up

interval(60_000).pipe(

switchMap(() =&gt; this.statsService.getUserStats()),

takeUntilDestroyed(this.destroyRef)

).subscribe(stats =&gt; { this.stats = stats; });
```

```
}


// No ngOnDestroy needed

}
```

Why it matters: The before version has four subscriptions and only three unsubscribes -- one invisible memory leak. In a real app navigated dozens of times, these accumulate silently until performance degrades and cryptic errors appear. The after version makes forgetting impossible -- every subscription uses the same one-line pattern and Angular handles all cleanup automatically.

5. Common Mistakes & Misconceptions

"HTTP calls don't need unsubscribing -- they complete after one response"

This is partially true but dangerously incomplete. A single HttpClient GET does complete after one response -- so it technically doesn't leak. But this reasoning leads developers to never unsubscribe anything. Then a polling Observable, a valueChanges stream, or a BehaviorSubject causes a massive leak -- and nobody understands why.

Think of it like this -- most candlesticks do eventually burn out. But if you develop the habit of never blowing out candles, the ones that don't burn out will quietly burn your house down.

```
// TECHNICALLY fine HttpClient completes after one response

this.http.get('/api/users').subscribe(users =&gt; {

this.users = users;

});


// These WILL leak without unsubscribe:

this.searchControl.valueChanges.subscribe(...); // never completes

interval(5000).subscribe(...); // never completes

this.store.select(getUsers).subscribe(...); // never completes


// Best practice use takeUntilDestroyed for ALL subscriptions

// Don't reason about which ones complete just always clean up

// Consistency prevents the one forgotten infinite stream

this.http.get('/api/users').pipe(

takeUntilDestroyed(this.destroyRef)

).subscribe(users =&gt; { this.users = users; });
```

"Subscribing multiple times to the same Observable is fine"

Every .subscribe() call creates a new independent subscription -- even to the same Observable. Subscribing in ngOnChanges without cancelling the previous subscription means each @Input()

change adds another active subscription. Five input changes = five simultaneous HTTP calls = five callbacks all updating state.

Think of accidentally signing up for the same newspaper five times -- five copies arrive every morning. Each navigation cycle that re-subscribes without cleanup adds another delivery.

```
// New subscription created on every @Input change

@Component({ ... })

export class UserComponent implements OnChanges {


@Input() userId: number;


ngOnChanges(): void {

// Every userId change = NEW subscription, previous NOT cancelled

// After 5 changes: 5 active subscriptions

this.userService.getUser(this.userId).subscribe(user =&gt; {

this.user = user;

});

}

}


// Use switchMap cancels previous automatically

@Component({ ... })

export class UserComponent implements OnInit {


@Input() userId: number;


private readonly userIdChange$ = new Subject&lt;number&gt;();

private readonly destroyRef = inject(DestroyRef);


ngOnInit(): void {

this.userIdChange$.pipe(

switchMap(id =&gt; this.userService.getUser(id)), // cancels previous

takeUntilDestroyed(this.destroyRef)

).subscribe(user =&gt; { this.user = user; });

}
```

```
ngOnChanges(changes: SimpleChanges): void {

if (changes['userId']) {

this.userIdChange$.next(this.userId);

}

}

}
```

"Mixing async pipe and manual subscribe gives more control"

Mixing them on the same Observable creates two subscriptions -- two HTTP calls, two state updates, two side effects. The async pipe creates one subscription silently in the template. Your manual .subscribe() creates another. They both fire independently.

Think of it like accidentally sending the same form twice -- double the submissions, double the processing, double the side effects.

```
// Double subscription async pipe AND manual subscribe

@Component({

template: `

&lt;!-- async pipe: subscription #1 --&gt;

&lt;div *ngFor="let user of users$ | async"&gt;{{ user.name }}&lt;/div&gt;

`

})

export class UserComponent implements OnInit {

users$ = this.userService.getUsers();


ngOnInit(): void {

// Manual subscribe: subscription #2 HTTP called TWICE

this.users$.subscribe(users =&gt; {

console.log(users.length);

});

}

}


// Pick one async pipe OR manual subscribe never both on same Observable


// Option A: async pipe only cleanest for display
```

```
@Component({
template: `&lt;div *ngFor="let user of users$ | async"&gt;{{ user.name }}&lt;/div&gt;`
})
export class UserComponent {
readonly users$ = this.userService.getUsers(); // no subscribe needed
}

// Option B: manual subscribe only needed when side effects required
@Component({
template: `&lt;div *ngFor="let user of users"&gt;{{ user.name }}&lt;/div&gt;`
})
export class UserComponent implements OnInit {
users: User[] = [];
private readonly destroyRef = inject(DestroyRef);

ngOnInit(): void {
this.userService.getUsers().pipe(
takeUntilDestroyed(this.destroyRef)
).subscribe(users =&gt; {
this.users = users;
this.analytics.track('users_loaded', users.length); // side effect
});
}
}
```

6. Do's and Don'ts

| [Y] Do | [N] Don't |

|---|---|

| Use takeUntilDestroyed(this.destroyRef) for all subscriptions in Angular 16+ | Leave subscriptions without any cleanup -- even one-shot HTTP calls |

| Use async pipe for template-bound Observables -- Angular manages the full lifecycle | Mix async pipe AND manual .subscribe() on the same Observable |

| Collect multiple subscriptions with new Subscription() + .add() when doing manual cleanup | Create individual Subscription variables per stream -- easy to miss one in ngOnDestroy |

| Use switchMap for @Input()-triggered subscriptions -- cancels the previous call | Re-subscribe in ngOnChanges without cancelling the previous subscription |

| Use takeUntil(this.destroy$) pattern when maintaining Angular < 16 codebases | Use takeUntilDestroyed in Angular < 16 -- it doesn't exist there |

| Handle complete in .subscribe() when using EMPTY as an error fallback | Only handle next -- EMPTY triggers complete not next, component state never updates |

---

7. Debugging Tips

Bug: Cannot set properties of null after navigation

```
ERROR: Cannot set properties of null (reading 'users')
```

* Plain English cause: Subscription callback fired after the component was destroyed -- the tap was left running -- this is null

* Fix: Add takeUntilDestroyed(this.destroyRef) to every subscription

* Angular note: The #1 symptom of a subscription memory leak in Angular -- appears after navigating away

Bug: Same HTTP endpoint called 2, 3, or more times

```
Network tab shows duplicate requests to same URL
```

* Plain English cause: Multiple active subscriptions to the same Observable -- double-subscribe, async pipe + manual subscribe, or re-subscribing in ngOnChanges without cleanup

* Fix: Audit all .subscribe() calls -- ensure each Observable has exactly one subscriber

Bug: Data still updating in the console after navigating away

```
Console logs firing from a component on a completely different route
```

* Plain English cause: interval, WebSocket, or BehaviorSubject subscription never cancelled -- ghost component still alive in memory

* Fix: Verify takeUntilDestroyed is applied to every long-lived subscription

Bug: takeUntilDestroyed() throws Error: injection context error

```
Error: takeUntilDestroyed() can only be used within an injection context
```

* Plain English cause: takeUntilDestroyed() with no argument must be in an injection context -- ngOnInit is not one

* Fix: Pass this.destroyRef explicitly, injected via inject(DestroyRef) in the class body

```
// ngOnInit is NOT an injection context

ngOnInit(): void {

this.data$.pipe(takeUntilDestroyed()).subscribe(); // Error

}


// Always inject DestroyRef and pass explicitly
```

```
private readonly destroyRef = inject(DestroyRef); // injection context


ngOnInit(): void {

this.data$.pipe(

takeUntilDestroyed(this.destroyRef) // always works

).subscribe();

}
```

8. Real-World Applications

Application 1 -- Real-time Dashboard with Multiple Streams
Plain English first:

A dashboard typically combines multiple live data sources -- stats, notifications, activity feed. Without subscription management, each navigation cycle to and from the dashboard adds more active streams. After ten navigations, ten sets of streams are all running simultaneously. Memory fills up, performance degrades, and errors appear in the console. With takeUntilDestroyed, every stream is cancelled when you leave and fresh streams start clean when you return.

```
@Component({

selector: 'app-dashboard',

standalone: true,

imports: [CommonModule],

template: `

&lt;div&gt;Active Users: {{ stats .activeUsers 0 }}&lt;/div&gt;

&lt;div&gt;Revenue: {{ stats .revenue .toFixed(2) '0.00' }}&lt;/div&gt;

@for (notif of notifications; track notif.id) {

&lt;div&gt;{{ notif.message }}&lt;/div&gt;

}

`

})

export class DashboardComponent implements OnInit {


stats: DashboardStats | null = null;

notifications: Notification[] = [];


private readonly destroyRef = inject(DestroyRef);
```

```
constructor(

private readonly statsService: StatsService,

private readonly notifService: NotificationService

) {}


ngOnInit(): void {

// Polling refreshes every 30 seconds

timer(0, 30_000).pipe(

switchMap(() =&gt; this.statsService.getDashboardStats()),

takeUntilDestroyed(this.destroyRef) // interval + HTTP both cancelled

).subscribe(stats =&gt; { this.stats = stats; });


// Real-time stream fires on every new notification

this.notifService.notifications$.pipe(

takeUntilDestroyed(this.destroyRef)

).subscribe(notif =&gt; {

this.notifications = [notif, ...this.notifications].slice(0, 10);

});

}

// Navigate away both streams cancelled

// Navigate back both streams start fresh

}
```

Application 2 -- Search with Cancellable HTTP Requests

Plain English first:

A search box that queries an API on each keystroke is where subscription management, switchMap, and takeUntilDestroyed work together. Without switchMap, every keystroke fires a new HTTP call and the previous one is never cancelled -- results arrive out of order and the wrong data gets displayed. Without takeUntilDestroyed, navigating away leaves the valueChanges stream and any in-flight HTTP calls still active. Together they give you a search that always shows the right result and always cleans up after itself.

```
@Component({

selector: 'app-product-search',

standalone: true,
```

```
imports: [CommonModule, ReactiveFormsModule],

template: `

&lt;input [formControl]="searchControl" placeholder="Search..."&gt;

@if (isSearching) { &lt;div&gt;Searching...&lt;/div&gt; }

@for (product of results; track product.id) {

&lt;div&gt;{{ product.name }} {{ product.price }}&lt;/div&gt;

}

@if (!isSearching &amp;&amp; results.length === 0 &amp;&amp; searchControl.value) {

&lt;p&gt;No results for "{{ searchControl.value }}"&lt;/p&gt;

}

`

})

export class ProductSearchComponent implements OnInit {


readonly searchControl = new FormControl('');

results: Product[] = [];

isSearching = false;


private readonly destroyRef = inject(DestroyRef);


constructor(private readonly productService: ProductService) {}


ngOnInit(): void {

this.searchControl.valueChanges.pipe(

debounceTime(300),

distinctUntilChanged(),

filter(q =&gt; (q .length 0) &gt;= 2),

tap(() =&gt; { this.isSearching = true; this.results = []; }),

switchMap(query =&gt;

this.productService.search(query '').pipe(

catchError(() =&gt; of([])) // inner catchError Topic 18

)
```

```
  ),

  takeUntilDestroyed(this.destroyRef) // cleans up valueChanges stream

  ).subscribe(results =&gt; {

  this.results = results;

  this.isSearching = false;

  });

  }

  }
```

9. Practice Exercises

Exercise 1 -- Beginner

Build an ActivityFeedComponent that polls ActivityService.getFeed() every 10 seconds and displays the last 5 activities. First implement it WITHOUT any cleanup -- observe the leak by navigating away and back (add a console.log in the callback to confirm it fires on a destroyed component). Then add takeUntilDestroyed -- verify the logs stop on navigation. Document exactly what you observe in both cases and explain why the behaviour differs.

Exercise 2 -- Intermediate

Build a UserSearchComponent with a text input that searches a UserService.search(query) Observable with 300ms debounce. Display: search results, a loading indicator, error state, and "no results" message. Use takeUntilDestroyed throughout. Add a second subscription for UserService.getRecentSearches() -- a live stream that updates automatically. Verify by logging that navigating away and back creates exactly one set of active subscriptions each time -- never accumulating.

Exercise 3 -- Advanced

Build a DataSyncComponent managing three simultaneous streams: user profile (polls every 60 seconds), unread notification count (simulate WebSocket with interval(5000)), and a search stream triggered by an @Input() searchQuery that should cancel and restart on every input change. All three must use takeUntilDestroyed. Add a connectionStatus property showing 'connected' when all streams are active and 'reconnecting' when any stream errors and retries. Prove zero memory leaks by tracking active subscription count in a shared DebugService that components increment on subscribe and decrement on complete/error.

10. Key Takeaways

Core Concepts in Plain English

* Subscription = the active delivery -- must be cancelled when the component is gone
* Memory leak = a destroyed component kept alive by uncancelled subscriptions -- ghost component
* takeUntilDestroyed(destroyRef) = Angular 16+ -- automatic cleanup -- the recommended default
* async pipe = template-level management -- Angular subscribes on create, unsubscribes on destroy

* Manual unsubscribe = Subscription.add() + ngOnDestroy -- always valid, always works
* switchMap = cancels previous inner Observable when a new value arrives -- essential for search and @Input() streams

The Three Cleanup Patterns

```
// Pattern 1 takeUntilDestroyed (Angular 16+ recommended)

private readonly destroyRef = inject(DestroyRef);

stream$.pipe(takeUntilDestroyed(this.destroyRef)).subscribe(...)


// Pattern 2 async pipe (cleanest for templates)

readonly data$ = this.service.getData();

// In template: {{ data$ | async }}


// Pattern 3 manual (always valid, pre-Angular 16)

private readonly sub = new Subscription();

ngOnInit(): void { this.sub.add(stream$.subscribe(...)); }

ngOnDestroy(): void { this.sub.unsubscribe(); }
```

Quick Reference Table

Scenario	Best Pattern	Why
Template data display	async pipe	Angular manages full lifecycle
Any subscription with side effects	takeUntilDestroyed	Full control + auto cleanup
Angular < 16 codebase	takeUntil(destroy$) or manual	takeUntilDestroyed unavailable
@Input() triggers new HTTP call	switchMap + takeUntilDestroyed	Cancels previous on new input
Polling / intervals	timer(0, ms) + switchMap + takeUntilDestroyed	Interval + HTTP both cancelled

Angular-Specific Rules

* takeUntilDestroyed() with no argument = injection context only (field initialiser, constructor)
* takeUntilDestroyed(this.destroyRef) with explicit ref = works anywhere including ngOnInit
* Never mix async pipe AND manual .subscribe() on the same Observable -- double subscription bug
* Use switchMap for @Input()-triggered streams -- cancels previous automatically
* One-shot HTTP calls technically don't leak -- but use takeUntilDestroyed anyway for consistency
* Always handle complete callback in subscribe when using EMPTY as an error fallback

One-Line Memory Aid

> Every .subscribe() is a tap you turned on. takeUntilDestroyed is the smart valve that turns it off when you leave the room. async pipe does it automatically. Never leave a tap running in an empty house.

---

11. Thought-Provoking Question + Answer

> Angular Signals are positioned as a replacement for much of RxJS in Angular apps. If Signals manage state reactively without subscriptions -- does subscription management become irrelevant Or will RxJS subscriptions always be needed

Plain English explanation first:

Think of it like automatic doors being invented. Before, someone had to physically open and close every door. Automatic doors handle themselves. But automatic doors didn't replace all doors -- some places still need manual doors, some can't fit automatic ones, some manual doors are simply better for the job.

Signals are automatic doors for state -- reactive, self-managing, no subscription cleanup needed. RxJS Observables are powerful manual doors -- more flexible, handling complex async flows, HTTP operations, and time-based logic that signals can't do yet.

```
// SIGNALS no subscription, no cleanup needed

@Component({ template: `&lt;p&gt;Count: {{ count() }}&lt;/p&gt;` })

export class CounterComponent {

count = signal(0);

doubled = computed(() =&gt; this.count() * 2);


increment(): void { this.count.update(c =&gt; c + 1); }

// Zero subscription management Angular tracks it automatically

}


// RxJS OBSERVABLES subscriptions still required

@Component({ ... })

export class SearchComponent implements OnInit {

private readonly destroyRef = inject(DestroyRef);


ngOnInit(): void {

// Signals can't do this RxJS operators are essential here

this.searchControl.valueChanges.pipe(

debounceTime(300),

switchMap(q =&gt; this.http.get(`/search q=${q}`)),

takeUntilDestroyed(this.destroyRef) // still needed

).subscribe(results =&gt; this.results.set(results)); // set a Signal from RxJS

}

}
```

The current reality -- signals and RxJS coexist:

```
Signals replace RxJS for:

Component local state (count, isLoading, selectedItem)

Derived computed values (filteredList, totalPrice)

Simple shared state between components

Replacing BehaviorSubject for state in services


RxJS is still needed for:

HTTP calls HttpClient returns Observable, not changing

WebSockets and SSE real-time streams

Complex operators debounce, switchMap, retry, forkJoin

Form valueChanges Angular forms are still Observable-based

Third-party libraries most return Observables


The bridge toSignal():

readonly users = toSignal(

this.userService.getUsers(), // Observable source

{ initialValue: [] } // initial value while loading

);

// Observable subscribed internally by Angular

// Subscription managed automatically no takeUntilDestroyed needed

// Template: {{ users() }} accesses like a signal
```

Practical rule to take away:

> "Signals dramatically reduce how much RxJS you write for state -- but subscription management is not dead. HTTP, WebSockets, forms, and complex async chains remain Observable-based and still need takeUntilDestroyed. The emerging best practice: use Signals for state, RxJS for async operations, and toSignal() to bridge them -- which manages the subscription automatically. Master both -- they are complementary, not competing."

---

12. Related Topics to Learn Next

Angular-Specific Concepts

- takeUntilDestroyed & DestroyRef in depth (Topic 21 -- the mechanism powering this topic -- DestroyRef hooks, toSignal, injection context rules)*
- async pipe (Topic 22 -- the template alternative that eliminates subscription management entirely)*

Angular Signals (Topic 23 -- signal(), computed(), toSignal() -- directly reduces how much of Topic 20 you need)*

OnPush Change Detection (Topic 24 -- subscriptions + signals + immutability + OnPush = the performance quartet)*

RxJS Operators in depth (Topic 46 -- switchMap, mergeMap, forkJoin -- the operators that make subscriptions complex)*

---

13. Updated Learning Tracker

```
Topics 1 19 ES6/JS + TypeScript Foundation COMPLETE

F1 F5 Angular Foundations Overview COMPLETE

Topic 20 RxJS Subscription Management COMPLETE

Topic 21 takeUntilDestroyed &amp; DestroyRef NEXT


TIER 1 REMAINING:

21. takeUntilDestroyed &amp; DestroyRef

22. async pipe

23. Angular Signals (signal, computed, effect)

24. OnPush Change Detection

25. Component Lifecycle Hooks

26. @Input() &amp; @Output() in depth

27. Services &amp; Dependency Injection


TIER 2 UPCOMING:

28 36. Template Syntax HTTP Interceptors


TIER 3 WHEN NEEDED:

37 46. Directives RxJS Operators
```

Key Concept Added to Quick Reference
Topic 20 -- RxJS Subscription Management

> Every .subscribe() is a tap you turned on. takeUntilDestroyed turns it off when you leave. async pipe does it automatically. Never leave a tap running in an empty house.

* Subscription = active delivery -- must be cancelled when component is destroyed
* Memory leak = destroyed component kept alive by uncancelled subscriptions
* takeUntilDestroyed(destroyRef) = Angular 16+ automatic cleanup -- recommended default
* async pipe = template-level -- Angular manages full lifecycle automatically
* Manual unsubscribe = Subscription.add() + ngOnDestroy -- always valid

* switchMap = cancels previous inner stream -- essential for search and @Input() changes
* Never mix async pipe + manual subscribe on same Observable -- double subscription
* Signals + toSignal() = the future -- reduces but does not eliminate subscription management

Topic 21 -- takeUntilDestroyed & DestroyRef in Angular

0. Difficulty & Priority Rating

Difficulty: Beginner-Intermediate -- the usage is simple (one line added to every pipe); understanding why* it works and the injection context rules takes one careful read

* Angular Relevance: Critical -- this is the modern Angular standard for subscription cleanup; every component you write from Angular 16+ should use this pattern; understanding DestroyRef directly unlocks toSignal(), service-level cleanup, and Angular's new functional patterns

1. Explanation

The Hotel Checkout Analogy

Imagine a hotel with a smart room system. When you check in, the system activates everything in your room -- lights, TV, heating, newspapers delivered each morning. When you check out, the system automatically deactivates everything -- lights off, TV off, heating off, newspaper delivery cancelled.

You don't have to go around the room manually turning everything off. The hotel's checkout system knows exactly which room you had and cancels everything tied to it -- automatically, completely, the moment you leave.

DestroyRef is the hotel's checkout system.

It's Angular's notification that a specific context -- a component, a service, a directive -- is being destroyed. takeUntilDestroyed listens to that checkout notification and automatically cancels any subscription tied to it the moment it fires.

```
Check in (component created) subscriptions start

Room activities (subscriptions) streams running, data flowing

Check out (component destroyed) DestroyRef fires

Hotel system (takeUntilDestroyed) everything cancelled automatically
```

The Smart Plug Analogy

Think of DestroyRef as a smart power strip. All your devices (subscriptions) are plugged into it. When you leave the room and flip the master switch (component destroyed), every device plugged into that strip turns off simultaneously. You didn't have to unplug each device individually. The strip handled it all from one central switch.

```
Smart power strip DestroyRef

Devices plugged in subscriptions using takeUntilDestroyed(destroyRef)

Master switch off component destroyed DestroyRef fires

All devices off all subscriptions cancelled automatically
```

In Plain English

* DestroyRef = Angular's injectable notification system -- tells you when a specific context is being destroyed

* takeUntilDestroyed(destroyRef) = an RxJS operator that plugs a subscription into the smart power strip -- when the strip fires, the subscription cancels

* Injection context = a place where Angular's DI system is active -- constructor, field initialisers, inject() calls -- where DestroyRef can be automatically detected

* inject(DestroyRef) = manually getting the smart power strip so you can use it anywhere in your class

* toSignal() = Angular's bridge between Observable and Signal -- uses DestroyRef internally so you never need to manage the subscription yourself

Now the Technical Terms

* DestroyRef = an Angular service representing the lifecycle of the current injection context -- provides onDestroy(callback) to register cleanup logic

* takeUntilDestroyed(destroyRef) = RxJS operator from @angular/core/rxjs-interop -- completes the Observable when the DestroyRef notifies destruction

* inject() = Angular's functional DI API -- works in injection contexts to get any injectable without constructor injection

* Injection context = any location where Angular's DI system is active: class field initialisers, constructors, functions called during construction

* toSignal(observable, options) = converts an Observable to a Signal -- internally uses DestroyRef to manage cleanup

* onDestroy(callback) = DestroyRef's method -- registers any callback to run when the context is destroyed -- not just for subscriptions

---

2. Connection to Prior Concepts

DestroyRef and takeUntilDestroyed are the Angular-specific implementation of patterns you've already learned:

Topic 20 (RxJS Subscription Management) -- Topic 20 introduced the problem (memory leaks) and the three patterns -- takeUntilDestroyed was the recommended solution. This topic explains how it actually works* under the hood and unlocks its full capabilities

* Topic 3 (Closures) -- DestroyRef.onDestroy(callback) registers a closure -- that callback closes over whatever it needs to clean up. The same backpack concept -- the cleanup callback carries references to what needs cancelling

Topic 17 (Decorators) -- @Injectable, @Component, @Directive -- Angular creates a DestroyRef for each decorated class instance. The decorator is what tells Angular "create a destruction context for this"*

* Topic 15 (Access Modifiers) -- private readonly destroyRef = inject(DestroyRef) -- the pattern you'll write in every component. private because nothing outside needs it, readonly because it's set once and never changes

* Topic 16 (Generics) -- toSignal<T>(observable$) -- the generic <T> is inferred from the Observable type, giving you a fully typed Signal without manual type annotation

Bridging note for your records:

"DestroyRef is Angular's smart power strip -- one central switch that cancels everything connected to it. takeUntilDestroyed plugs subscriptions into that strip. inject(DestroyRef) gets the strip. onDestroy(callback) lets you connect ANY cleanup to the strip -- not just subscriptions."

---

3. Code Example

Example 1 -- What DestroyRef actually is -- under the hood
Before using it, see what it actually does -- it's just a notification system.

```
// DestroyRef is an injectable you can get it like any other service

import { DestroyRef, inject, Component, OnInit } from '@angular/core';


@Component({ selector: 'app-demo', template: '...' })

export class DemoComponent implements OnInit {


// Get the DestroyRef for THIS component instance

private readonly destroyRef = inject(DestroyRef);


ngOnInit(): void {


// DestroyRef has ONE method: onDestroy(callback)

// Register ANY cleanup logic runs when this component is destroyed

this.destroyRef.onDestroy(() =&gt; {

console.log('DemoComponent is being destroyed cleanup running');

// Put ANY cleanup here not just subscriptions:

clearInterval(this.pollingTimer); // clear a timer

this.socket .close(); // close a WebSocket

this.thirdPartyLib .dispose(); // clean up a third-party lib

localStorage.removeItem('temp'); // clean up temporary storage

});


// Every destroyRef.onDestroy() call adds ANOTHER callback

// All of them run when the component is destroyed

this.destroyRef.onDestroy(() =&gt; {

console.log('Second cleanup callback also runs on destroy');

});

}

}


// When the component is destroyed:

// Console: "DemoComponent is being destroyed cleanup running"
```

```
// Console: "Second cleanup callback also runs on destroy"

// All cleanup happened automatically
```

Example 2 -- takeUntilDestroyed -- three ways to use it

The injection context rule is the most important thing to understand here.

```
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';

import { DestroyRef, inject } from '@angular/core';


// ---- WAY 1 Explicit destroyRef works ANYWHERE in the class ----

// Most flexible recommended default


@Component({ selector: 'app-way1', template: '...' })

export class Way1Component implements OnInit {


// inject() in field initialiser = injection context

private readonly destroyRef = inject(DestroyRef);


ngOnInit(): void {

// destroyRef passed explicitly works in ngOnInit, methods, anywhere

this.dataService.getData().pipe(

takeUntilDestroyed(this.destroyRef)

).subscribe(data =&gt; { this.data = data; });

}

}



// ---- WAY 2 No argument field initialiser injection context ----

// Clean, concise but subscription must be set up in the field


@Component({ selector: 'app-way2', template: '...' })

export class Way2Component {


// Field initialiser = injection context

// takeUntilDestroyed() with no argument auto-detects destroyRef

private readonly data$ = this.dataService.getData().pipe(
```

```
takeUntilDestroyed() // injection context auto-detects

);

// Alternative subscribe inline in the field

private readonly dataSub = this.dataService.getData().pipe(

takeUntilDestroyed()

).subscribe(data =&gt; { this.data = data; }); // subscription in field

constructor(private readonly dataService: DataService) {}

}

// ---- WAY 3 No argument constructor injection context ----

@Component({ selector: 'app-way3', template: '...' })

export class Way3Component {

constructor(private readonly dataService: DataService) {

// Constructor = injection context

// takeUntilDestroyed() auto-detects here too

this.dataService.getData().pipe(

takeUntilDestroyed() // constructor is injection context

).subscribe(data =&gt; { this.data = data; });

}

}

// ---- WRONG ngOnInit is NOT an injection context ----

@Component({ selector: 'app-wrong', template: '...' })

export class WrongComponent implements OnInit {

ngOnInit(): void {

this.dataService.getData().pipe(

takeUntilDestroyed() // Error: not in injection context
```

```
).subscribe(data =&gt; { this.data = data; });

}

}

// Fix: either use Way 1 (explicit destroyRef)

// or move subscription to constructor/field initialiser
```

Example 3 -- DestroyRef for non-subscription cleanup

DestroyRef does more than just subscriptions -- any cleanup logic can hook into it.

```
@Injectable({ providedIn: 'root' })

export class WebSocketService {


// DestroyRef works in services too cleans up when service is destroyed

// For root services this fires when the app closes

// For component-scoped services this fires when the component is destroyed

private readonly destroyRef = inject(DestroyRef);

private socket: WebSocket | null = null;


connect(url: string): Observable&lt;MessageEvent&gt; {

this.socket = new WebSocket(url);


// Register cleanup on service destroy not just subscription

this.destroyRef.onDestroy(() =&gt; {

this.socket .close();

this.socket = null;

console.log('WebSocket closed on service destroy');

});


return new Observable(observer =&gt; {

this.socket!.onmessage = event =&gt; observer.next(event);

this.socket!.onerror = err =&gt; observer.error(err);

this.socket!.onclose = () =&gt; observer.complete();

});

}
```

```
}



// ---- Component with third-party library cleanup ----

@Component({ selector: 'app-chart', template: '&lt;canvas
#chartCanvas&gt;&lt;/canvas&gt;' })

export class ChartComponent implements AfterViewInit {


@ViewChild('chartCanvas') canvas!: ElementRef&lt;HTMLCanvasElement&gt;;


private readonly destroyRef = inject(DestroyRef);

private chart: ChartJS | null = null;


ngAfterViewInit(): void {

// Third-party library not an Observable needs manual cleanup

this.chart = new ChartJS(this.canvas.nativeElement, {

type: 'bar',

data: { labels: [], datasets: [] }

});


// Hook into DestroyRef clean up the chart when component is destroyed

this.destroyRef.onDestroy(() =&gt; {

this.chart .destroy(); // third-party cleanup

this.chart = null;

});

}

}
```

Angular Example -- toSignal -- DestroyRef working invisibly

The cleanest pattern -- Observable source, Signal result, zero subscription management.

```
import { toSignal, takeUntilDestroyed } from '@angular/core/rxjs-interop';

import { inject, DestroyRef, signal, computed } from '@angular/core';


@Component({

selector: 'app-product-list',
```

```
standalone: true,
imports: [CommonModule],
template: `
&lt;!-- Signals access with () no async pipe needed --&gt;
@if (isLoading()) {
&lt;div&gt;Loading products...&lt;/div&gt;
}

@for (product of filteredProducts(); track product.id) {
&lt;div&gt;{{ product.name }} {{ product.price.toFixed(2) }}&lt;/div&gt;
}

&lt;p&gt;Showing {{ filteredProducts().length }} of {{ products().length }}&lt;/p&gt;
`
})
export class ProductListComponent {

// toSignal converts Observable to Signal
// DestroyRef managed INTERNALLY by Angular zero boilerplate
readonly products = toSignal(
this.productService.getProducts(),
{ initialValue: [] as Product[] } // value before first emission
);

// toSignal with loading state
readonly productsWithStatus = toSignal(
this.productService.getProducts().pipe(
map(products =&gt; ({ products, isLoading: false })),
startWith({ products: [] as Product[], isLoading: true })
),
{ initialValue: { products: [] as Product[], isLoading: true } }
);
```

```
// computed Signal derived from other signals always up to date
readonly isLoading = computed(() =&gt; this.productsWithStatus().isLoading);
readonly allProducts = computed(() =&gt; this.productsWithStatus().products);

// Search filter writable signal updated by user input
readonly searchTerm = signal('');

// Derived automatically updates when products OR searchTerm changes
readonly filteredProducts = computed(() =&gt; {
const term = this.searchTerm().toLowerCase();
return this.allProducts().filter(p =&gt;
p.name.toLowerCase().includes(term)
);
});

constructor(private readonly productService: ProductService) {}

// No ngOnDestroy, no takeUntilDestroyed, no subscribe management
// toSignal handles the subscription internally using DestroyRef

onSearch(term: string): void {
this.searchTerm.set(term); // Signal update computed re-runs automatically
}
}


// ---- Mixing takeUntilDestroyed AND toSignal for side effects ----
@Component({ selector: 'app-orders', template: '...' })
export class OrdersComponent implements OnInit {

// toSignal for pure data display no subscription management
readonly orders = toSignal(
this.orderService.getOrders(),
```

```
{ initialValue: [] as Order[] }

);

// toSignal for derived counts

readonly orderCount = computed(() =&gt; this.orders().length);

readonly pendingCount = computed(() =&gt;

this.orders().filter(o =&gt; o.status === 'pending').length

);

private readonly destroyRef = inject(DestroyRef);

constructor(

private readonly orderService: OrderService,

private readonly analyticsService: AnalyticsService

) {}

ngOnInit(): void {

// takeUntilDestroyed for the stream that has SIDE EFFECTS

// (analytics tracking) can't use toSignal for this

this.orderService.getOrders().pipe(

takeUntilDestroyed(this.destroyRef)

).subscribe(orders =&gt; {

this.analyticsService.track('orders_loaded', { count: orders.length });

});

}

}
```

4. Before & After Refactor

The Problem in Plain English:

Before DestroyRef and takeUntilDestroyed, every component needed a destroy$ Subject, an ngOnDestroy hook, and two lines of cleanup code -- even when the component was otherwise simple. It was unavoidable boilerplate. Forgetting any part of it caused a memory leak. DestroyRef collapses all of that into one injected object and one operator -- the smart power strip replaces individually unplugging every device.

```
// BEFORE Angular 15 and earlier mandatory boilerplate

@Component({ selector: 'app-user-dashboard', template: '...' })
export class UserDashboardComponent implements OnInit, OnDestroy {

userData: User | null = null;
notifications: Notification[] = [];
recentOrders: Order[] = [];
isLoading = false;

// Boilerplate #1 destroy Subject
private readonly destroy$ = new Subject&lt;void&gt;();

constructor(
private readonly userService: UserService,
private readonly notifService: NotificationService,
private readonly orderService: OrderService
) {}

ngOnInit(): void {
this.isLoading = true;

// takeUntil on every single pipe repetitive but required
this.userService.getCurrentUser().pipe(
takeUntil(this.destroy$)
).subscribe(user =&gt; { this.userData = user; });

this.notifService.getNotifications().pipe(
takeUntil(this.destroy$)
).subscribe(notif =&gt; { this.notifications = notif; });

this.orderService.getRecentOrders().pipe(
takeUntil(this.destroy$)
).subscribe(orders =&gt; {
```

```
this.recentOrders = orders;

this.isLoading = false;

});


// Polling same pattern

timer(0, 60_000).pipe(

switchMap(() =&gt; this.orderService.getRecentOrders()),

takeUntil(this.destroy$)

).subscribe(orders =&gt; { this.recentOrders = orders; });

}


// Boilerplate #2 ngOnDestroy with two lines

ngOnDestroy(): void {

this.destroy$.next(); // fire the signal

this.destroy$.complete(); // clean up Subject

}

// Total boilerplate: Subject declaration + ngOnDestroy + 2 lines = unavoidable

}


// AFTER Angular 16+ with DestroyRef + takeUntilDestroyed + toSignal


@Component({ selector: 'app-user-dashboard', template: '...' })

export class UserDashboardComponent implements OnInit {


// toSignal zero subscription management for pure data

readonly userData = toSignal(

this.userService.getCurrentUser(),

{ initialValue: null }

);

readonly notifications = toSignal(

this.notifService.getNotifications(),

{ initialValue: [] as Notification[] }
```

```
);

recentOrders: Order[] = [];

isLoading = false;

// One line no Subject, no ngOnDestroy

private readonly destroyRef = inject(DestroyRef);

constructor(

private readonly userService: UserService,

private readonly notifService: NotificationService,

private readonly orderService: OrderService

) {}

ngOnInit(): void {

this.isLoading = true;

// takeUntilDestroyed for streams needing side effects or local state

this.orderService.getRecentOrders().pipe(

takeUntilDestroyed(this.destroyRef)

).subscribe(orders =&gt; {

this.recentOrders = orders;

this.isLoading = false;

});

// Polling one clean pattern

timer(0, 60_000).pipe(

switchMap(() =&gt; this.orderService.getRecentOrders()),

takeUntilDestroyed(this.destroyRef)

).subscribe(orders =&gt; { this.recentOrders = orders; });

}

// No ngOnDestroy at all
```

```
// No destroy$ Subject

// No boilerplate

}
```

Why it matters: The before version has mandatory boilerplate on every component -- Subject, ngOnDestroy, next(), complete(). In a 50-component app that's 150 lines of pure boilerplate doing nothing except preventing memory leaks. The after version eliminates all of it. DestroyRef is one injectable that does the job of the Subject + ngOnDestroy pattern -- and toSignal goes even further, eliminating manual subscribe entirely.

5. Common Mistakes & Misconceptions

"DestroyRef is just the new name for ngOnDestroy -- same thing"

They're related but meaningfully different. ngOnDestroy is a lifecycle hook that you implement -- it's called by Angular when the component is destroyed. DestroyRef is an injectable service you can pass around, inject into other functions, and register multiple callbacks on. The key difference: DestroyRef can be passed to functions and utilities outside the component -- ngOnDestroy can't.

Think of ngOnDestroy like a checkout desk you have to physically go to yourself. DestroyRef is like a checkout card you carry -- you can hand it to anyone (a utility function, a service, takeUntilDestroyed) and they can use it to check you out on your behalf.

```
// ngOnDestroy you implement it, Angular calls it

// Limited to the component class itself

export class MyComponent implements OnDestroy {

ngOnDestroy(): void {

// cleanup but this is inside the component

}

}


// DestroyRef injectable, passable, flexible

export class MyComponent {

private readonly destroyRef = inject(DestroyRef);


ngOnInit(): void {

// Pass destroyRef to a utility function OUTSIDE the component

setupPolling(this.dataService, this.destroyRef);


// Pass destroyRef to a helper service method

this.helperService.registerWithCleanup(this.destroyRef);
```

```
}

}


// Utility function outside the component receives destroyRef

function setupPolling(service: DataService, destroyRef: DestroyRef): void {

timer(0, 30_000).pipe(

switchMap(() =&gt; service.getData()),

takeUntilDestroyed(destroyRef) // uses the component's destroyRef

).subscribe(data =&gt; console.log(data));

}

// The function has no component reference just the destroyRef

// Clean separation of concerns
```

"takeUntilDestroyed() without an argument always works"

Only works in an injection context -- field initialisers and constructors. ngOnInit, ngOnChanges, event handlers, and any method called after construction are NOT injection contexts. Calling takeUntilDestroyed() without an argument in ngOnInit throws a runtime error.

Think of the smart power strip analogy -- without an argument, takeUntilDestroyed has to find the strip automatically. It can only do that when it's standing next to the fuse box (injection context). Once you've moved to a different room (ngOnInit), it can't find the fuse box anymore -- you have to carry the strip reference yourself.

```
// Common mistake using no argument in ngOnInit

@Component({ ... })

export class MyComponent implements OnInit {


ngOnInit(): void {

this.service.getData().pipe(

takeUntilDestroyed() // RuntimeError: not in injection context

).subscribe();

}

}


// Fix 1 inject destroyRef in field (injection context) and pass it

@Component({ ... })

export class MyComponent implements OnInit {
```

```
private readonly destroyRef = inject(DestroyRef); // field = injection context

ngOnInit(): void {
this.service.getData().pipe(
takeUntilDestroyed(this.destroyRef) // explicit always works
).subscribe();
}
}

// Fix 2 move subscription to constructor (injection context)
@Component({ ... })
export class MyComponent {

constructor(private readonly service: DataService) {
this.service.getData().pipe(
takeUntilDestroyed() // constructor = injection context
).subscribe();
}
}

// Fix 3 move subscription to field initialiser (injection context)
@Component({ ... })
export class MyComponent {

private readonly sub = this.service.getData().pipe(
takeUntilDestroyed() // field = injection context
).subscribe(data =&gt; { this.data = data; });

constructor(private readonly service: DataService) {}
}
```

"toSignal is just a shortcut for async pipe -- same result"

They solve the same problem differently and have meaningful differences. async pipe lives in the template -- it subscribes when the component renders and unsubscribes when destroyed. toSignal

lives in the class -- it subscribes immediately (even before the template renders), gives you a Signal you can use in computed values, and integrates with Angular's signal graph for fine-grained reactivity.

```
// async pipe template only, cannot be used in computed()

@Component({

template: `

&lt;!-- Can only use async result in the template --&gt;

&lt;div *ngFor="let user of users$ | async"&gt;{{ user.name }}&lt;/div&gt;

&lt;!-- Cannot derive from it in class code --&gt;

`

})

export class AsyncPipeComponent {

readonly users$ = this.service.getUsers(); // Observable

// Cannot do: computed(() =&gt; this.users().filter(...)) not a signal

}


// toSignal class-level, usable in computed, full signal integration

@Component({

template: `

&lt;!-- Access like a signal with () --&gt;

@for (user of filteredUsers(); track user.id) {

&lt;div&gt;{{ user.name }}&lt;/div&gt;

}

&lt;p&gt;{{ filteredUsers().length }} users shown&lt;/p&gt;

`

})

export class ToSignalComponent {

readonly users = toSignal(this.service.getUsers(), { initialValue: [] as User[] });

readonly searchTerm = signal('');


// Can derive from toSignal result in computed async pipe can't do this

readonly filteredUsers = computed(() =&gt;

this.users().filter(u =&gt;
```

```
  u.name.toLowerCase().includes(this.searchTerm().toLowerCase())

  )

  );


  constructor(private readonly service: UserService) {}

  }
```

6. Do's and Don'ts

| [Y] Do | [N] Don't |

|---|---|

| Always private readonly destroyRef = inject(DestroyRef) in every component -- make it a habit | Use takeUntilDestroyed() without argument in ngOnInit -- injection context error |

| Use toSignal() for Observables used purely for display in templates | Use async pipe when you need the value in computed() -- toSignal is the right tool |

| Use destroyRef.onDestroy() for non-subscription cleanup -- timers, third-party libs, WebSockets | Only use ngOnDestroy for cleanup -- DestroyRef.onDestroy() is more flexible and composable |

| Pass destroyRef to utility functions that set up subscriptions -- clean separation of concerns | Couple utility functions to components by importing component-specific things -- pass destroyRef instead |

| Provide initialValue with toSignal() -- prevents the Signal being undefined before first emission | Skip initialValue on toSignal() -- the Signal type becomes T \| undefined and needs . everywhere |

| Use toSignal for most Observable-to-template cases -- it's the modern Angular way | Mix toSignal AND manual subscribe AND async pipe on the same Observable -- pick one |

7. Debugging Tips

Bug: Error: NG0203: inject() must be called from an injection context

```
  Error: NG0203: inject() must be called from an injection context
```

* Plain English cause: inject(DestroyRef) or takeUntilDestroyed() called outside an injection context -- inside ngOnInit, a method, or a callback

* Fix: Move inject(DestroyRef) to a field initialiser and pass destroyRef explicitly to wherever it's needed

Bug: toSignal() value is undefined before data loads

```
  Template shows undefined or errors on initial render
```

* Plain English cause: toSignal() called without initialValue -- Signal is undefined until Observable emits

* Fix: Always provide initialValue matching the expected type

```
  // No initialValue users is Signal&lt;User[] | undefined&gt;

  readonly users = toSignal(this.service.getUsers());
```

```
// Template: users() .length needs . everywhere


// With initialValue users is Signal&lt;User[]&gt;

readonly users = toSignal(

this.service.getUsers(),

{ initialValue: [] as User[] } // always an array never undefined

);

// Template: users().length clean
```

Bug: destroyRef.onDestroy() callback not running

```
Third-party library not cleaned up after component destruction
```

* Plain English cause: onDestroy callback registered after the component was already destroyed -- race condition, or registered in the wrong place

* Fix: Register onDestroy callbacks in ngOnInit or field initialisers -- not in async callbacks

Bug: toSignal subscription firing after component destroyed

```
Console errors about destroyed component context
```

* Plain English cause: toSignal was called outside an injection context -- DestroyRef not found -- subscription not managed

* Fix: Ensure toSignal() is called in a field initialiser or constructor -- never in ngOnInit

```
// toSignal in ngOnInit injection context error

ngOnInit(): void {

this.users = toSignal(this.service.getUsers()); //

}


// toSignal in field initialiser injection context

readonly users = toSignal(

this.service.getUsers(),

{ initialValue: [] as User[] }

); // injection context, DestroyRef auto-detected
```

---

8. Real-World Applications

Application 1 -- Composable Subscription Utility
Plain English first:

As your Angular app grows, the same subscription patterns repeat across components -- polling, WebSocket connections, form value streams. Instead of copy-pasting

takeUntilDestroyed(this.destroyRef) everywhere, you can extract these patterns into utility functions that accept a DestroyRef. This is DestroyRef's killer feature -- you can pass it to anything. The utility function sets up and manages the subscription, the component just provides its destroyRef as the cleanup hook.

```
// ---- Reusable subscription utilities accept DestroyRef ----

// Generic polling utility works for any Observable

function createPollingStream&lt;T&gt;(

source$: Observable&lt;T&gt;,

intervalMs: number,

destroyRef: DestroyRef

): Observable&lt;T&gt; {

return timer(0, intervalMs).pipe(

switchMap(() =&gt; source$),

takeUntilDestroyed(destroyRef) // uses component's destroyRef

);

}

// Generic form watcher tracks form changes with cleanup

function watchFormControl&lt;T&gt;(

control: AbstractControl,

destroyRef: DestroyRef,

callback: (value: T) =&gt; void

): void {

control.valueChanges.pipe(

debounceTime(300),

distinctUntilChanged(),

takeUntilDestroyed(destroyRef)

).subscribe(callback);

}

// ---- Components use utilities cleanly ----

@Component({ selector: 'app-dashboard', template: '...' })
```

```
export class DashboardComponent implements OnInit {

stats: DashboardStats | null = null;

orders: Order[] = [];

// DestroyRef injected once passed to any utility that needs it

private readonly destroyRef = inject(DestroyRef);

constructor(

private readonly statsService: StatsService,

private readonly orderService: OrderService

) {}

ngOnInit(): void {

// Utilities handle all subscription management

createPollingStream(

this.statsService.getStats(),

30_000,

this.destroyRef // hand over the checkout card

).subscribe(stats =&gt; { this.stats = stats; });

createPollingStream(

this.orderService.getRecentOrders(),

60_000,

this.destroyRef

).subscribe(orders =&gt; { this.orders = orders; });

}

// Component is clean all subscription logic in utilities

}
```

Application 2 -- Component-Scoped Service Cleanup

Plain English first:

Sometimes a service is provided at the component level -- it's created when the component is created and should be destroyed with it. Without DestroyRef, the service has no way to know when the component is gone. With DestroyRef injected into the service, the service can register its own

cleanup logic and self-destruct when its owner component is destroyed -- like an employee who automatically clears their desk when their manager's department closes.

```
// ---- Component-scoped service with self-cleanup via DestroyRef ----

@Injectable() // No providedIn provided at component level

export class ComponentCacheService {

private readonly cache = new Map&lt;string, unknown&gt;();

private readonly destroyRef = inject(DestroyRef);

constructor() {

// Service registers its OWN cleanup no component involvement needed

this.destroyRef.onDestroy(() =&gt; {

this.cache.clear();

console.log('ComponentCacheService cleaned up');

});

}

set&lt;T&gt;(key: string, value: T): void {

this.cache.set(key, value);

}

get&lt;T&gt;(key: string): T | undefined {

return this.cache.get(key) as T | undefined;

}

}

// Component provides the service service manages its own lifecycle

@Component({

selector: 'app-product-detail',

standalone: true,

providers: [ComponentCacheService], // scoped to THIS component

template: '...'
```

```
})

export class ProductDetailComponent implements OnInit {


constructor(

private readonly cache: ComponentCacheService,

private readonly productService: ProductService

) {}


ngOnInit(): void {

const cached = this.cache.get&lt;Product&gt;('current-product');

if (!cached) {

this.productService.getProduct(this.productId).subscribe(product =&gt; {

this.cache.set('current-product', product);

this.product = product;

});

} else {

this.product = cached;

}

}

// When ProductDetailComponent is destroyed:

// ComponentCacheService.destroyRef fires

// cache.clear() runs automatically

// Service is garbage collected with the component

}
```

9. Practice Exercises

Exercise 1 -- Beginner

Create a CleanupDemoComponent that demonstrates DestroyRef.onDestroy() directly -- without using takeUntilDestroyed. Register three separate onDestroy callbacks: one that logs a message, one that clears a setInterval, and one that removes an item from localStorage. Verify all three run when the component is destroyed by navigating away. Then refactor the setInterval to use takeUntilDestroyed instead of the onDestroy callback -- explain in comments when you'd use each approach.

Exercise 2 -- Intermediate

Build a LiveStockTickerComponent that displays live stock prices. Use toSignal() to convert a StockService.getPrices() Observable into a Signal. Add a computed() that derives the highest and lowest priced stock from the signal. Add a second computed() that formats all prices as currency strings. Show all three in the template. Verify that toSignal manages the subscription automatically -- confirm by adding a console.log in the service Observable that stops firing when you navigate away.

Exercise 3 -- Advanced

Build a reusable createPaginatedStream<T> utility function (not a component -- just a function) that:

* Accepts a loadPage: (page: number) => Observable<PagedResult<T>> function
* Accepts a DestroyRef for cleanup
* Returns an object with: items: Signal<T[]>, currentPage: Signal<number>, totalPages: Signal<number>, isLoading: Signal<boolean>, nextPage(): void, prevPage(): void
* Manages all subscriptions internally using the provided DestroyRef
* Uses toSignal or takeUntilDestroyed as appropriate for each stream

Use this utility in two different components -- UserListComponent and ProductListComponent -- each passing their own destroyRef. Verify that both components' streams are independently managed and cleaned up correctly.

---

10. Key Takeaways

Core Concepts in Plain English

* DestroyRef = Angular's smart power strip -- fires a signal when a component/service/directive is destroyed
* inject(DestroyRef) = get the strip -- works in field initialisers and constructors (injection context)
* takeUntilDestroyed(destroyRef) = plug a subscription into the strip -- auto-cancels on destroy
* destroyRef.onDestroy(callback) = connect ANY cleanup to the strip -- not just subscriptions
* toSignal(obs$, options) = converts Observable to Signal -- uses DestroyRef internally, zero management needed
* Injection context = where Angular's DI is active -- field initialisers, constructors -- NOT ngOnInit
* takeUntilDestroyed() no argument = injection context only -- auto-detects DestroyRef
* takeUntilDestroyed(destroyRef) explicit = works anywhere -- always safe

DestroyRef vs ngOnDestroy -- When to Use Each

```
ngOnDestroy:

Always valid the baseline

Required if you implement OnDestroy interface

Couples cleanup to the component class

Can't be passed to utility functions


DestroyRef:

Injectable can be passed anywhere

Multiple onDestroy callbacks each independent
```

```
Powers takeUntilDestroyed and toSignal

Works in services, directives, and components

No interface to implement less boilerplate
```

Quick Reference -- Which Pattern to Use

```
// Pure display in template no side effects needed

readonly data = toSignal(obs$, { initialValue: defaultValue });


// Data display + derived computations

readonly data = toSignal(obs$, { initialValue: [] });

readonly filtered = computed(() =&gt; this.data().filter(...));


// Side effects needed (analytics, logging, external updates)

private readonly destroyRef = inject(DestroyRef);

obs$.pipe(takeUntilDestroyed(this.destroyRef)).subscribe(data =&gt; {

this.sideEffect(data);

});


// Non-subscription cleanup (timers, third-party libs, WebSockets)

this.destroyRef.onDestroy(() =&gt; { this.cleanup(); });


// Utility function that manages its own subscriptions

function myUtil(obs$: Observable&lt;T&gt;, destroyRef: DestroyRef): void {

obs$.pipe(takeUntilDestroyed(destroyRef)).subscribe(...);

}
```

Angular-Specific Rules

* Always declare private readonly destroyRef = inject(DestroyRef) as a class field -- not in ngOnInit
* takeUntilDestroyed() with no arg = injection context only (field, constructor)
* takeUntilDestroyed(this.destroyRef) = works everywhere -- the safe default
* Always provide initialValue with toSignal() -- avoids T | undefined type
* toSignal() must be in injection context -- field initialiser or constructor -- never ngOnInit
* Pass destroyRef to utility functions -- better than coupling utilities to component classes

One-Line Memory Aid

> DestroyRef is the hotel checkout system -- one switch turns everything off. inject(DestroyRef) gets the switch. takeUntilDestroyed plugs subscriptions into it. toSignal plugs itself in automatically. Always declare it as a field -- injection context.

11. Thought-Provoking Question + Answer

> DestroyRef can be injected into services, not just components. A providedIn: 'root' service lives for the entire app lifetime -- so DestroyRef in a root service fires only when the app closes. But a component-scoped service has DestroyRef firing when the component is destroyed. How does this difference in DestroyRef lifetime change how you architect services in Angular, and when should a service be component-scoped rather than root-scoped

Plain English explanation first:

Imagine two types of employees at a company:

* Permanent staff (root services) -- hired by the company, work there forever, their desk is always available to everyone
* Contractors (component-scoped services) -- hired for a specific project, work only while that project exists, their desk is cleared when the project closes

Most Angular services are permanent staff -- providedIn: 'root' means one instance, always available, lives as long as the app. Their DestroyRef fires when the browser closes -- practically never during normal use.

Component-scoped services are contractors -- created when their component is created, destroyed with it. Their DestroyRef fires every time the component is navigated away from.

```
// ---- ROOT SERVICE DestroyRef fires when app closes ----

@Injectable({ providedIn: 'root' })

export class AuthService {

private readonly destroyRef = inject(DestroyRef);


constructor() {

// This onDestroy runs when the APP closes not on navigation

this.destroyRef.onDestroy(() =&gt; {

console.log('App closing auth cleanup');

// Useful for: flushing analytics, saving draft state, closing DB connections

});

}

}


// ---- COMPONENT-SCOPED SERVICE DestroyRef fires on component destroy ----

@Injectable() // No providedIn

export class FormDraftService {

private readonly destroyRef = inject(DestroyRef);
```

```
  private draft: FormData | null = null;

  constructor() {
  // This fires when the COMPONENT using it is destroyed
  this.destroyRef.onDestroy(() =&gt; {
  // Save draft to localStorage when user leaves the form
  if (this.draft) {
  localStorage.setItem('form-draft', JSON.stringify(this.draft));
  }
  });
  }

  saveDraft(data: FormData): void { this.draft = data; }
  getDraft(): FormData | null { return this.draft; }
  }

  // Component provides the scoped service
  @Component({
  providers: [FormDraftService], // new instance per component
  template: '...'
  })
  export class CheckoutFormComponent {
  constructor(private readonly draftService: FormDraftService) {}
  // FormDraftService created with this component
  // FormDraftService destroyed with this component
  // draft auto-saved to localStorage on destruction
  }
```

When to use component-scoped services:

```
  Use root service (providedIn: 'root') when:
  State or functionality shared across the entire app
  (auth, theme, notifications, cart, user preferences)
```

```
HTTP services, data fetching, API communication

Singleton pattern one instance for everyone

Caching that should persist across navigation


Use component-scoped service (providers: [MyService]) when:

State only relevant while a specific component is mounted

(form draft, wizard step state, local selection state)

Each instance of a component needs its own independent state

(two different forms open simultaneously, each with own draft)

Service should self-cleanup when component is gone

(WebSocket for a specific chat room, polling for a specific page)

Avoiding state leaking between component instances
```

The DestroyRef lifetime architecture insight:

```
// Pattern: component-scoped service that self-manages WebSocket

@Injectable()

export class ChatRoomService {

private readonly destroyRef = inject(DestroyRef);

private socket: WebSocket | null = null;

readonly messages$ = new Subject&lt;ChatMessage&gt;();


constructor() {

this.destroyRef.onDestroy(() =&gt; {

// Automatically closes WebSocket when ChatRoomComponent is destroyed

this.socket .close();

this.messages$.complete();

console.log('Chat room disconnected component gone');

});

}


connect(roomId: string): void {

this.socket = new WebSocket(`wss://chat.example.com/rooms/${roomId}`);
```

```
this.socket.onmessage = e =&gt; this.messages$.next(JSON.parse(e.data));

}

}


@Component({

providers: [ChatRoomService], // each chat room component gets its OWN service

template: '...'

})

export class ChatRoomComponent implements OnInit {

@Input() roomId!: string;


constructor(private readonly chatService: ChatRoomService) {}


ngOnInit(): void {

this.chatService.connect(this.roomId);

// If THREE ChatRoomComponents are open THREE independent WebSockets

// When any one is destroyed ITS WebSocket closes automatically

// Root service would share one socket completely wrong for this case

}

}
```

Practical rule to take away:

> "Default to providedIn: 'root' for services shared across the app. Reach for component-scoped services when each component instance needs its own independent state, or when a service should self-cleanup in sync with a specific component's lifecycle. DestroyRef makes component-scoped services far more powerful -- the service manages its own cleanup without the component having to orchestrate it. The test: if you opened two instances of the same component, should they share service state or have independent state Independent = component-scoped."

---

12. Related Topics to Learn Next

Angular-Specific Concepts

- async pipe (Topic 22 -- the template-level alternative -- how it relates to toSignal)*
- Angular Signals (signal, computed, effect) (Topic 23 -- toSignal is the bridge -- signals are the destination)*
- OnPush Change Detection (Topic 24 -- signals + toSignal + OnPush = zero manual change detection)*
- Angular Services & Dependency Injection (Topic 27 -- component-scoped vs root-scoped DI -- directly extends this topic)*

Component Lifecycle Hooks (Topic 25 -- how ngOnDestroy relates to DestroyRef -- the full lifecycle picture)*

13. Updated Learning Tracker

```
Topics 1 19 ES6/JS + TypeScript Foundation COMPLETE

F1 F5 Angular Foundations Overview COMPLETE

Topic 20 RxJS Subscription Management COMPLETE

Topic 21 takeUntilDestroyed &amp;amp; DestroyRef COMPLETE

Topic 22 async pipe NEXT


TIER 1 REMAINING:

22. async pipe

23. Angular Signals (signal, computed, effect)

24. OnPush Change Detection

25. Component Lifecycle Hooks

26. @Input() &amp;amp; @Output() in depth

27. Services &amp;amp; Dependency Injection


TIER 2 UPCOMING:

28 36. Template Syntax HTTP Interceptors


TIER 3 WHEN NEEDED:

37 46. Directives RxJS Operators
```

Key Concept Added to Quick Reference
Topic 21 -- takeUntilDestroyed & DestroyRef

> DestroyRef is the hotel checkout system -- one switch turns everything off. inject(DestroyRef) gets the switch. takeUntilDestroyed plugs subscriptions into it. toSignal plugs itself in automatically. Always declare it as a field.

* DestroyRef = Angular's destruction notification -- injectable, passable, flexible
* inject(DestroyRef) = get the switch -- must be in injection context (field or constructor)
* takeUntilDestroyed(destroyRef) = plug subscription into switch -- auto-cancels on destroy
* takeUntilDestroyed() no arg = injection context only -- auto-detects -- NOT in ngOnInit
* destroyRef.onDestroy(cb) = register ANY cleanup -- timers, WebSockets, third-party libs
* toSignal(obs$, { initialValue }) = Observable -> Signal -- DestroyRef managed internally
* Always provide initialValue with toSignal -- prevents T | undefined type

* Component-scoped service = new instance per component + DestroyRef fires on component destroy

Topic 22 -- async Pipe in Angular

0. Difficulty & Priority Rating

* Difficulty: Beginner -- one pipe, one concept; the mental model clicks immediately; the nuances around error handling and multiple subscriptions take one careful read
* Angular Relevance: Critical -- the async pipe is Angular's cleanest way to display Observable and Promise data in templates; it eliminates subscription management entirely for display data, prevents memory leaks automatically, and works seamlessly with OnPush change detection

1. Explanation

The Personal Assistant Analogy

Imagine you're a busy executive. You need information from various departments -- sales figures, inventory reports, customer feedback. Instead of calling each department yourself, waiting on hold, taking notes, and remembering to close each call when you're done -- you hire a personal assistant.

You tell the assistant: "I need the latest sales figures on my desk."

The assistant:

* Makes the call for you
* Waits for the response
* Puts the result on your desk the moment it arrives
* Updates your desk whenever new figures come in
* Cancels all open calls automatically when you leave the office

The async pipe is that personal assistant for your Angular template.

You hand it an Observable or Promise and say: "show me whatever this produces." The pipe subscribes for you, puts the value in the template the moment it arrives, updates automatically when new values come, and cancels the subscription when the component is destroyed -- all without you writing a single line of subscription management code.

```
You (template) ask the assistant for the value

Assistant (async pipe) subscribes, waits, delivers, updates

You leave (destroy) assistant cancels all open requests automatically
```

The Live Scoreboard Analogy

Think of a sports scoreboard. It doesn't store the score -- it just displays whatever the score currently is. When the score changes, the board updates. When the game ends, the board turns off. The scoreboard itself has no opinion about the score -- it's purely a display mechanism.

The async pipe is the scoreboard for your Observable data.

It doesn't store the data -- it displays whatever the Observable currently emits. When a new value arrives, the template updates. When the component is destroyed, the pipe stops listening. Pure display, zero storage, automatic lifecycle.

In Plain English

* async pipe = a template pipe that subscribes to an Observable or Promise, displays its current value, and unsubscribes automatically when the component is destroyed

* No .subscribe() needed = the pipe handles the full subscription lifecycle

* No ngOnDestroy needed = Angular calls .unsubscribe() when the component is destroyed

* Always shows the latest value = whenever the Observable emits, the template updates automatically

* Works with both Observables and Promises = same syntax for both

Now the Technical Terms

* async pipe = Angular's built-in AsyncPipe -- implements PipeTransform and OnDestroy -- subscribes to Observable<T> or Promise<T> and returns T | null

* Returns null = before the first emission the async pipe returns null -- the template must handle this

* Automatic subscription = the pipe subscribes when the template is first evaluated

* Automatic unsubscription = the pipe unsubscribes via its own ngOnDestroy when the component is destroyed

* Change detection = the async pipe calls markForCheck() when a new value arrives -- works perfectly with OnPush

---

2. Connection to Prior Concepts

The async pipe is the template-level version of everything you learned in Topics 20 and 21:

* Topic 20 (RxJS Subscription Management) -- the async pipe IS subscription management -- it just does it inside the pipe instead of the component class. The problem of "who unsubscribes" is answered: the pipe does

* Topic 21 (takeUntilDestroyed & DestroyRef) -- takeUntilDestroyed manages subscriptions in the class; async pipe manages them in the template. Same problem, different layer. toSignal() bridges between them -- async pipe is the simpler template-only alternative

* Topic 18 (Error Handling) -- async pipe has no built-in error handling -- errors from the Observable bubble up uncaught unless you add catchError to the pipe before it reaches the template. The safety net must be in the Observable chain, not in the template

* Topic 19 (. and) -- async pipe returns null before the first emission. . and in the template are the runtime companions -- {{ (user$ | async) .name 'Loading...' }}

* Topic 17 (Decorators) -- async pipe must be imported via CommonModule or AsyncPipe in standalone components. Without importing it, Angular doesn't recognise | async in the template

Bridging note for your records:

"The async pipe moves subscription management from the class into the template. Instead of this.service.getData().subscribe(d => this.data = d) in the class plus ngOnDestroy, you write data$ | async in the template and the pipe handles everything. The Observable stays as an Observable -- you never unwrap it manually."

---

3. Code Example

Example 1 -- The simplest async pipe usage

Hand an Observable to the template -- the pipe does everything else.

```
// ---- Component class Observable stays as Observable ----

@Component({

selector: 'app-user-list',
```

```
standalone: true,

imports: [CommonModule], // AsyncPipe is in CommonModule

template: `

&lt;!-- async pipe subscribes, displays, unsubscribes automatically --&gt;

&lt;div *ngFor="let user of users$ | async"&gt;

{{ user.name }}

&lt;/div&gt;

`

})

export class UserListComponent {


// Observable NOT subscribed to in the class

// The async pipe in the template handles everything

readonly users$ = this.userService.getUsers();


constructor(private readonly userService: UserService) {}


// No ngOnInit needed Observable set up as field

// No ngOnDestroy needed async pipe unsubscribes

// No Subscription variable needed

}



// ---- Standalone component import AsyncPipe directly ----

@Component({

selector: 'app-product-list',

standalone: true,

imports: [AsyncPipe, NgFor], // import AsyncPipe directly no CommonModule needed

template: `

&lt;div *ngFor="let product of products$ | async"&gt;

{{ product.name }}

&lt;/div&gt;
```

```
`

})

export class ProductListComponent {

readonly products$ = this.productService.getProducts();

constructor(private readonly productService: ProductService) {}

}
```

Example 2 -- Handling null before first emission

The async pipe returns null initially -- templates must handle this gracefully.

```
@Component({

standalone: true,

imports: [CommonModule],

template: `

&lt;!-- PROBLEM: async pipe returns null before first emission --&gt;

&lt;!-- user$ | async is null initially accessing .name would crash --&gt;


&lt;!-- Without null handling crashes on initial render --&gt;

&lt;h2&gt;{{ (user$ | async).name }}&lt;/h2&gt;

&lt;!-- TypeError: Cannot read properties of null (reading 'name') --&gt;



&lt;!-- Option 1 *ngIf to unwrap and guard ----

Most common pattern named variable with 'as' syntax --&gt;

&lt;ng-container *ngIf="user$ | async as user"&gt;

&lt;!-- Inside here: user is guaranteed non-null --&gt;

&lt;h2&gt;{{ user.name }}&lt;/h2&gt;

&lt;p&gt;{{ user.email }}&lt;/p&gt;

&lt;p&gt;{{ user.role }}&lt;/p&gt;

&lt;/ng-container&gt;


&lt;!-- Option 2 safe navigation operator --&gt;

&lt;h2&gt;{{ (user$ | async) .name 'Loading...' }}&lt;/h2&gt;
```

```
&lt;!-- Option 3 *ngIf with else template --&gt;

&lt;ng-container *ngIf="user$ | async as user; else loading"&gt;

&lt;h2&gt;{{ user.name }}&lt;/h2&gt;

&lt;/ng-container&gt;

&lt;ng-template #loading&gt;

&lt;p&gt;Loading user...&lt;/p&gt;

&lt;/ng-template&gt;


&lt;!-- Option 4 Angular 17+ @if with as --&gt;

@if (user$ | async; as user) {

&lt;h2&gt;{{ user.name }}&lt;/h2&gt;

&lt;p&gt;{{ user.email }}&lt;/p&gt;

} @else {

&lt;p&gt;Loading...&lt;/p&gt;

}

`

})

export class UserProfileComponent {

readonly user$ = this.userService.getCurrentUser();

constructor(private readonly userService: UserService) {}

}
```

Example 3 -- Multiple async pipes -- the problem and the solution
Each | async in a template creates a separate subscription -- ngIf as shares one.*

```
@Component({

standalone: true,

imports: [CommonModule],

template: `

&lt;!-- PROBLEM: Three separate async pipes = THREE subscriptions

Three HTTP calls to the same endpoint --&gt;

&lt;h2&gt;{{ (user$ | async) .name }}&lt;/h2&gt;

&lt;p&gt;{{ (user$ | async) .email }}&lt;/p&gt;
```

```
&lt;span&gt;{{ (user$ | async) .role }}&lt;/span&gt;



&lt;!-- SOLUTION: One *ngIf wraps all uses ONE subscription --&gt;

&lt;ng-container *ngIf="user$ | async as user"&gt;

&lt;h2&gt;{{ user.name }}&lt;/h2&gt; &lt;!-- no async pipe here uses 'user' variable
--&gt;

&lt;p&gt;{{ user.email }}&lt;/p&gt;

&lt;span&gt;{{ user.role }}&lt;/span&gt;

&lt;/ng-container&gt;


&lt;!-- Angular 17+ @if version same principle --&gt;

@if (user$ | async; as user) {

&lt;h2&gt;{{ user.name }}&lt;/h2&gt;

&lt;p&gt;{{ user.email }}&lt;/p&gt;

&lt;span&gt;{{ user.role }}&lt;/span&gt;

}

`

})

export class UserCardComponent {

// shareReplay(1) replays latest value to any new subscribers

// Important when multiple async pipes were used (the bad pattern above)

readonly user$ = this.userService.getCurrentUser().pipe(

shareReplay(1) // all subscribers get the same value

);

constructor(private readonly userService: UserService) {}

}
```

Angular Example -- Complete real-world async pipe usage

A real Angular component using async pipe patterns across different scenarios.

```
@Component({

selector: 'app-dashboard',

standalone: true,
```

```
imports: [CommonModule, AsyncPipe, CurrencyPipe, DatePipe],

template: `

&lt;!-- Pattern 1: *ngIf as single subscription, null guarded --&gt;

&lt;ng-container *ngIf="currentUser$ | async as user"&gt;

&lt;header&gt;

&lt;h1&gt;Welcome back, {{ user.name }}&lt;/h1&gt;

&lt;span class="role-badge"&gt;{{ user.role }}&lt;/span&gt;

&lt;/header&gt;

&lt;/ng-container&gt;


&lt;!-- Pattern 2: Loading + error + data states --&gt;

&lt;ng-container *ngIf="ordersState$ | async as state"&gt;


@if (state.isLoading) {

&lt;div class="spinner"&gt;Loading orders...&lt;/div&gt;

}


@if (state.error) {

&lt;div class="error-banner"&gt;{{ state.error }}&lt;/div&gt;

}


@if (!state.isLoading &amp;&amp; !state.error) {

@for (order of state.orders; track order.id) {

&lt;div class="order-card"&gt;

&lt;span&gt;{{ order.id }}&lt;/span&gt;

&lt;span&gt;{{ order.total | currency:'GBP' }}&lt;/span&gt;

&lt;span&gt;{{ order.createdAt | date:'shortDate' }}&lt;/span&gt;

&lt;/div&gt;

}

@empty {

&lt;p&gt;No orders found&lt;/p&gt;

}
```

```
}

&lt;/ng-container&gt;

&lt;!-- Pattern 3: Simple Promise with async pipe --&gt;
&lt;p&gt;Server time: {{ serverTime$ | async | date:'medium' }}&lt;/p&gt;

&lt;!-- Pattern 4: Boolean Observable for conditional display --&gt;
&lt;button
[disabled]="isLoading$ | async"
(click)="onRefresh()"&gt;
{{ (isLoading$ | async) 'Loading...' : 'Refresh' }}
&lt;/button&gt;
`
})
export class DashboardComponent {

// ---- Observables as class fields async pipe subscribes in template ----

// Simple data stream
readonly currentUser$ = this.userService.getCurrentUser();

// Combined state stream single async pipe shows all states
readonly ordersState$ = this.orderService.getOrders().pipe(
map(orders =&gt; ({
orders,
isLoading: false,
error: null as string | null
})),
startWith({
orders: [] as Order[],
isLoading: true,
error: null
```

```
}),
catchError(err =&gt; of({
orders: [] as Order[],
isLoading: false,
error: err .message 'Failed to load orders'
}))
);

// Promise also works with async pipe
readonly serverTime$ = fetch('/api/server-time')
.then(r =&gt; r.json())
.then((d: { time: string }) =&gt; new Date(d.time));

// Boolean stream disables button while loading
readonly isLoading$ = this.ordersState$.pipe(
map(state =&gt; state.isLoading)
);

constructor(
private readonly userService: UserService,
private readonly orderService: OrderService
) {}

onRefresh(): void {
// Trigger a new emission component logic stays clean
// No subscription management needed here either
}

// No ngOnInit, no ngOnDestroy, no Subscription variables
// Async pipe handles everything
}
```

4. Before & After Refactor

The Problem in Plain English:

Without the async pipe, every piece of Observable data needs a three-step process in the class -- subscribe in ngOnInit, store in a property, unsubscribe in ngOnDestroy. For a component that shows three pieces of data, that's nine steps. The component class fills up with subscription management code that has nothing to do with business logic. The async pipe collapses all nine steps into three template expressions -- the component class becomes pure logic, zero plumbing.

```
// BEFORE manual subscription management for every Observable


@Component({

selector: 'app-user-profile',

standalone: true,

imports: [CommonModule],

template: `

&lt;div *ngIf="!isLoading"&gt;

&lt;h2&gt;{{ user .name }}&lt;/h2&gt;

&lt;p&gt;{{ user .email }}&lt;/p&gt;

&lt;div *ngFor="let order of recentOrders"&gt;

{{ order.id }}

&lt;/div&gt;

&lt;p&gt;Notifications: {{ notificationCount }}&lt;/p&gt;

&lt;/div&gt;

&lt;div *ngIf="isLoading"&gt;Loading...&lt;/div&gt;

`

})

export class UserProfileComponent implements OnInit, OnDestroy {


// Three class properties just to store Observable data

user: User | null = null;

recentOrders: Order[] = [];

notificationCount: number = 0;

isLoading = true;


// Three subscription variables to manage
```

```
private userSub: Subscription;

private orderSub: Subscription;

private notifSub: Subscription;


constructor(

private readonly userService: UserService,

private readonly orderService: OrderService,

private readonly notifService: NotificationService

) {}


// ngOnInit just for subscription setup

ngOnInit(): void {

this.userSub = this.userService.getCurrentUser().subscribe(user =&gt; {

this.user = user;

this.isLoading = false;

});


this.orderSub = this.orderService.getRecentOrders().subscribe(orders =&gt; {

this.recentOrders = orders;

});


this.notifSub = this.notifService.getCount().subscribe(count =&gt; {

this.notificationCount = count;

});

}


// ngOnDestroy just for cleanup

ngOnDestroy(): void {

this.userSub .unsubscribe();

this.orderSub .unsubscribe();

this.notifSub .unsubscribe();

}
```

```
// 40 lines of boilerplate zero business logic

}


// AFTER async pipe zero subscription boilerplate


@Component({

selector: 'app-user-profile',

standalone: true,

imports: [CommonModule],

template: `

&lt;!-- Single *ngIf as one subscription for user --&gt;

@if (user$ | async; as user) {

&lt;h2&gt;{{ user.name }}&lt;/h2&gt;

&lt;p&gt;{{ user.email }}&lt;/p&gt;

} @else {

&lt;div&gt;Loading...&lt;/div&gt;

}


&lt;!-- Direct iteration async pipe on array Observable --&gt;

@for (order of recentOrders$ | async []; track order.id) {

&lt;div&gt;{{ order.id }}&lt;/div&gt;

}


&lt;!-- Simple value --&gt;

&lt;p&gt;Notifications: {{ notificationCount$ | async 0 }}&lt;/p&gt;

`

})

export class UserProfileComponent {


// Three Observables no subscriptions, no storage, no cleanup

readonly user$ = this.userService.getCurrentUser();

readonly recentOrders$ = this.orderService.getRecentOrders();
```

```
readonly notificationCount$ = this.notifService.getCount();


constructor(

private readonly userService: UserService,

private readonly orderService: OrderService,

private readonly notifService: NotificationService

) {}


// No ngOnInit, no ngOnDestroy, no Subscription variables

// 15 lines of pure intent zero boilerplate

}
```

Why it matters: The before version has 40 lines -- most of it plumbing. The after version has 15 lines -- all of it intent. Every piece of boilerplate removed is one fewer thing that can be forgotten, one fewer potential memory leak, one fewer reason to scroll through the file hunting for subscription cleanup. The component class now only says what it does -- not how it manages subscriptions.

5. Common Mistakes & Misconceptions

"I can use | async multiple times on the same Observable -- it's the same stream"

Every | async in a template creates an independent subscription. Using user$ | async three times in a template creates three subscriptions, makes three HTTP calls (if user$ is an HTTP Observable), and might deliver three different values if the Observable is not shared. This is the most common async pipe mistake.

Think of it like asking three different assistants to get the same report. Three phone calls to the same department, three copies of the same document delivered -- wasteful and potentially inconsistent.

```
&lt;!-- THREE subscriptions THREE HTTP calls --&gt;

&lt;h2&gt;{{ (user$ | async) .name }}&lt;/h2&gt;

&lt;p&gt;{{ (user$ | async) .email }}&lt;/p&gt;

&lt;span&gt;{{ (user$ | async) .role }}&lt;/span&gt;


&lt;!-- ONE subscription *ngIf as creates one variable --&gt;

&lt;ng-container *ngIf="user$ | async as user"&gt;

&lt;h2&gt;{{ user.name }}&lt;/h2&gt;

&lt;p&gt;{{ user.email }}&lt;/p&gt;

&lt;span&gt;{{ user.role }}&lt;/span&gt;

&lt;/ng-container&gt;
```

```
// If multiple async pipes are unavoidable use shareReplay(1)

// All subscribers share ONE execution and get the same cached value

readonly user$ = this.userService.getCurrentUser().pipe(

shareReplay(1) // multiple async pipes now share one HTTP call

);
```

"The async pipe handles errors -- I don't need catchError"

The async pipe has absolutely no error handling. If the Observable errors, the async pipe propagates the error -- the template breaks, the component shows nothing, and the Observable stream is permanently dead (Topic 20's dead stream problem). The safety net must be in the Observable chain -- not in the template.

Think of the personal assistant analogy -- if the department they called is unreachable, the assistant just shrugs and says nothing. They don't offer alternatives. You need to tell them upfront: "if you can't reach the department, bring me the backup report."

```
// No error handling Observable errors crash the template

readonly users$ = this.userService.getUsers();

// If HTTP fails: template shows nothing, stream dies, user sees blank page


// catchError in the chain async pipe receives a safe fallback

readonly users$ = this.userService.getUsers().pipe(

catchError(err =&gt; {

console.error('Failed to load users:', err);

return of([] as User[]); // async pipe gets [] instead of an error

})

);


// With full state management loading, error, data

readonly usersState$ = this.userService.getUsers().pipe(

map(users =&gt; ({ users, isLoading: false, error: null })),

startWith({ users: [] as User[], isLoading: true, error: null }),

catchError(err =&gt; of({

users: [] as User[],

isLoading: false,

error: err .message 'Failed to load'
```

```
}))

);
```

"async pipe is always better than toSignal -- keep everything in the template"

async pipe is perfect for display-only data. But it has real limitations -- you can't use its value in computed() signals, you can't easily derive from it in the class, and it requires CommonModule or AsyncPipe import. toSignal is better when you need to derive computed values from the data or use it in class logic.

Think of it like two tools -- the async pipe is a display screen (show the value), toSignal is a variable (use the value anywhere). Display screens are great for displaying -- but you can't do arithmetic on a display screen.

```
// async pipe display only can't derive from it in computed()

@Component({

template: `

&lt;!-- Can show the value --&gt;

&lt;div *ngFor="let user of users$ | async"&gt;{{ user.name }}&lt;/div&gt;

&lt;!-- Cannot use users$ | async value in a computed() --&gt;

`

})

export class DisplayOnlyComponent {

readonly users$ = this.service.getUsers();

// Cannot do: readonly adminCount = computed(() =&gt; )

// The async pipe value is only in the template not accessible in the class

}


// toSignal value available in class AND template

@Component({

template: `

&lt;div *ngFor="let user of users()"&gt;{{ user.name }}&lt;/div&gt;

&lt;p&gt;Admins: {{ adminCount() }}&lt;/p&gt;

`

})

export class FlexibleComponent {

readonly users = toSignal(this.service.getUsers(), { initialValue: [] });
```

```
// Can derive from the signal value in computed()

readonly adminCount = computed(() =&gt;

this.users().filter(u =&gt; u.role === 'admin').length

);

}
```

6. Do's and Don'ts

| [Y] Do | [N] Don't |

|---|---|

| Use *ngIf="obs$ \| async as value" to unwrap once and use multiple times | Use \| async on the same Observable multiple times in a template -- multiple subscriptions |

| Always add catchError to Observables used with async pipe -- it has no error handling | Rely on async pipe to handle errors -- it will crash the template silently |

| Use [] or 0 after \| async for array and number Observables -- pipe returns null initially | Access properties on \| async result without null guard -- null.name crashes |

| Import AsyncPipe directly in standalone components -- avoids importing all of CommonModule | Forget to import AsyncPipe or CommonModule -- \| async won't be recognised |

| Use shareReplay(1) when the same Observable is genuinely subscribed to multiple times | Share cold Observables (HTTP calls) without shareReplay -- multiple requests fired |

| Use toSignal when you need the Observable value in computed() or class logic | Use async pipe when you need the value for derived computations -- wrong tool |

7. Debugging Tips

Bug: Template shows nothing / blank -- async pipe seems not working

```
Template renders empty *ngFor shows nothing
```

* Plain English cause: Either the Observable errored (async pipe has no error handling -- silently shows nothing), or AsyncPipe / CommonModule not imported in standalone component

* Fix: Check browser console for errors. Add catchError to the Observable. Verify AsyncPipe is in imports

Bug: NullInjectorError or InvalidPipeArgument for async

```
InvalidPipeArgument: '[object Object]' for pipe 'AsyncPipe'
```

* Plain English cause: You passed the actual value (already unwrapped) to async pipe instead of the Observable -- or passed a plain object instead of an Observable/Promise

* Fix: Ensure the field is typed as Observable<T> not T -- async pipe needs the Observable itself

```
// Already subscribed passing the value, not the Observable

users: User[] = []; // subscribed manually async pipe can't handle User[]
```

```
// Keep as Observable let async pipe subscribe

readonly users$ = this.service.getUsers(); // Observable&lt;User[]&gt;
```

Bug: HTTP request fires multiple times

```
Network tab shows same endpoint called 2 or 3 times
```

* Plain English cause: Multiple | async on the same HTTP Observable -- each creates a new subscription and a new HTTP call

Fix: Use ngIf as pattern to subscribe once, OR add shareReplay(1) to the Observable

Bug: async pipe shows null in template instead of data

```
Template shows "null" as literal text
```

* Plain English cause: async pipe returns null before first emission -- no null guard in template

Fix: Add 'fallback' after the pipe, or wrap in ngIf as

```
&lt;!-- Shows literal "null" --&gt;

&lt;p&gt;{{ count$ | async }}&lt;/p&gt;


&lt;!-- Shows 0 before data arrives handles null --&gt;

&lt;p&gt;{{ count$ | async 0 }}&lt;/p&gt;
```

Bug: Old data shown after Observable updates

```
Template not updating when Observable emits new value
```

* Plain English cause: Using ChangeDetectionStrategy.OnPush but the async pipe isn't triggering change detection -- this is actually rare since async pipe calls markForCheck(). More likely cause: the Observable isn't actually emitting a new value (reference didn't change -- Topic 7)

* Fix: Verify the Observable actually emits. Check if mutation (not new reference) is the issue. async pipe handles markForCheck() automatically with OnPush

8. Real-World Applications

Application 1 -- Paginated Data Table with Loading and Error States
Plain English first:

A data table needs to show three different states -- loading while data is fetching, an error if it fails, and the actual data when it arrives. Without async pipe you'd need three class properties (isLoading, error, data) each manually set in subscription callbacks. With async pipe and a state object Observable, the entire state machine lives in the template with one pipe -- one subscription, three visual states, automatic cleanup.

```
@Component({

selector: 'app-user-table',

standalone: true,

imports: [CommonModule, CurrencyPipe, DatePipe],
```

```
template: `
&lt;ng-container *ngIf="tableState$ | async as state"&gt;

&lt;!-- Loading state --&gt;
@if (state.isLoading) {
&lt;div class="table-skeleton"&gt;
&lt;div *ngFor="let i of [1,2,3,4,5]" class="skeleton-row"&gt;&lt;/div&gt;
&lt;/div&gt;
}

&lt;!-- Error state --&gt;
@if (state.error) {
&lt;div class="error-panel"&gt;
&lt;p&gt;{{ state.error }}&lt;/p&gt;
&lt;button (click)="onRetry()"&gt;Try again&lt;/button&gt;
&lt;/div&gt;
}

&lt;!-- Data state --&gt;
@if (!state.isLoading &amp;&amp; !state.error) {
&lt;table&gt;
&lt;thead&gt;
&lt;tr&gt;
&lt;th&gt;Name&lt;/th&gt;
&lt;th&gt;Email&lt;/th&gt;
&lt;th&gt;Joined&lt;/th&gt;
&lt;th&gt;Orders&lt;/th&gt;
&lt;/tr&gt;
&lt;/thead&gt;
&lt;tbody&gt;
@for (user of state.users; track user.id) {
```

```
&lt;tr&gt;

&lt;td&gt;{{ user.name }}&lt;/td&gt;

&lt;td&gt;{{ user.email }}&lt;/td&gt;

&lt;td&gt;{{ user.joinedAt | date:'shortDate' }}&lt;/td&gt;

&lt;td&gt;{{ user.orderCount }}&lt;/td&gt;

&lt;/tr&gt;

} @empty {

&lt;tr&gt;&lt;td colspan="4"&gt;No users found&lt;/td&gt;&lt;/tr&gt;

}

&lt;/tbody&gt;

&lt;/table&gt;


&lt;div class="pagination"&gt;

&lt;span&gt;{{ state.users.length }} of {{ state.total }} users&lt;/span&gt;

&lt;/div&gt;

}


&lt;/ng-container&gt;

`

})

export class UserTableComponent {


// One Observable, one async pipe, full state machine

readonly tableState$ = this.userService.getUsers().pipe(

map(result =&gt; ({

users: result.items,

total: result.total,

isLoading: false,

error: null as string | null

})),

startWith({
```

```
users: [] as User[],

total: 0,

isLoading: true,

error: null

}),

catchError(err =&gt; of({

users: [] as User[],

total: 0,

isLoading: false,

error: err .message 'Failed to load users'

}))

);


constructor(private readonly userService: UserService) {}


onRetry(): void {

// Trigger a reload could be done with a Subject

window.location.reload();

}

// No class properties for loading/error/data

// No ngOnInit or ngOnDestroy

}
```

Application 2 -- Real-time Notification Badge
Plain English first:

A navigation bar notification badge shows an unread count that updates in real time. Without async pipe, this needs a subscription in ngOnInit, a class property for the count, and cleanup in ngOnDestroy -- in a component that might be a simple layout component with one job: show a number. With async pipe, the entire feature is two lines -- the Observable definition and the template expression. The badge component becomes so simple it barely needs to exist as a class.

```
@Component({

selector: 'app-nav-badge',

standalone: true,

imports: [CommonModule],
```

```
template: `
&lt;!-- Badge only shown when count &gt; 0 --&gt;
@if (unreadCount$ | async; as count) {
@if (count &gt; 0) {
&lt;span class="badge"&gt;
{{ count &gt; 99 '99+' : count }}
&lt;/span&gt;
}
}
`
})
export class NavBadgeComponent {

// WebSocket or polling stream async pipe manages it completely
readonly unreadCount$ = this.notifService.getUnreadCount().pipe(
catchError(() =&gt; of(0)), // network error = show 0
distinctUntilChanged() // only re-render when count actually changes
);

constructor(
private readonly notifService: NotificationService
) {}

// Entire component = 10 lines
// Zero subscription management code
// Automatically cleans up when nav is destroyed
}
```

9. Practice Exercises

Exercise 1 -- Beginner

Build a WeatherComponent that displays current weather from a WeatherService.getCurrentWeather() Observable. Use the async pipe with *ngIf as to show three states: loading (while data is null), data (temperature, description, city name), and an error message if

the Observable errors. Write the error handling in the Observable chain -- not in the template. Verify the component has zero subscription management code in the class -- no ngOnInit, no ngOnDestroy, no Subscription variables.

Exercise 2 -- Intermediate

Build a SearchResultsComponent that takes a searchQuery$: Observable<string> as an @Input(). Using the async pipe for display, implement a search that: debounces 300ms, only searches when query is 2+ characters, shows a loading state during each search, shows results, shows "no results" when empty, shows an error message if the search fails, and correctly handles rapid queries (latest result always shown). The entire subscription must be managed by the async pipe -- no manual subscribe in the class.

Exercise 3 -- Advanced

Build a LiveDashboardComponent that displays three separate data streams simultaneously -- statsStream$, activityFeed$, and alertsStream$ -- each updating at different intervals (5s, 10s, 30s). Each stream should: show a loading state on initial load, show an error state with a retry option if it fails, show a "last updated" timestamp that updates with each emission. Use ONE async pipe per stream -- no multiple pipes on the same Observable. Handle the case where streams emit at different times gracefully. Add shareReplay(1) appropriately. The component class should contain zero subscription management -- only Observable definitions.

---

10. Key Takeaways

Core Concepts in Plain English

* async pipe = the personal assistant -- subscribes for you, delivers values, cancels when you leave

Returns null initially = before the first emission -- always guard with fallback or ngIf as

* One pipe = one subscription = using | async twice on the same Observable = two subscriptions, two HTTP calls

ngIf as = the pattern to subscribe once and use the value multiple times in the template

* No error handling = errors must be caught with catchError in the Observable chain -- async pipe doesn't catch them

* Auto-unsubscribes = pipe calls .unsubscribe() in its own ngOnDestroy -- no component cleanup needed

* Works with OnPush = pipe calls markForCheck() on every emission -- perfect change detection integration

The Three Null-Handling Patterns

```
&lt;!-- Pattern 1 *ngIf as (most common subscribes once) --&gt;

&lt;ng-container *ngIf="data$ | async as data"&gt;

{{ data.name }}

&lt;/ng-container&gt;


&lt;!-- Pattern 2 @if as (Angular 17+) --&gt;

@if (data$ | async; as data) {
```

```
  {{ data.name }}

  }


  &lt;!-- Pattern 3 nullish coalescing (for simple values) --&gt;

  {{ count$ | async 0 }}

  {{ name$ | async 'Loading...' }}

  {{ items$ | async [] }}
```

async pipe vs toSignal vs manual subscribe

| | async pipe | toSignal | Manual subscribe |

|---|---|---|---|

| Where | Template only | Class + template | Class only |

| Null before emit | Yes -- returns null | No -- initialValue | No -- you control it |

| Use in computed() | [N] No | [Y] Yes | [N] No |

| Error handling | None -- needs catchError | None -- needs catchError | error callback |

| Boilerplate | Minimal | Minimal | Medium |

| Best for | Display-only data | Data + derived logic | Side effects |

Angular-Specific Rules

* Import AsyncPipe directly in standalone components -- don't import all of CommonModule just for async
* Always add catchError to Observables used with async pipe -- errors crash the template silently
* Use shareReplay(1) when the same Observable might be subscribed to multiple times
* Never mix async pipe AND manual .subscribe() on the same Observable -- double subscription

Use ngIf as or @if as to unwrap once when showing multiple properties from the same Observable

* For complex derived state: use toSignal() -- for pure display: async pipe is cleaner

One-Line Memory Aid

> async pipe = personal assistant in the template -- subscribes, delivers, cancels when you leave. *ngIf as = one subscription shared by the whole block. No error handling built in -- always add catchError to the chain first.

---

11. Thought-Provoking Question + Answer

> Angular now has three ways to consume an Observable in a template -- async pipe, toSignal(), and manual subscribe + class property. Angular's own team seems to be pushing toward toSignal() as the default in Angular 17+. If toSignal() can do everything async pipe does and more -- why does async pipe still exist and when would you genuinely choose it over toSignal()

Plain English explanation first:

Think of three ways to get coffee at a hotel:

* Room service (manual subscribe) -- you call, wait, receive delivery, call again when you want more. Full control but lots of effort

* Coffee machine in the lobby (async pipe) -- walk to the lobby, press a button, get coffee immediately. Simple, reliable, requires no equipment in your room

* Coffee machine in your room (toSignal) -- press a button from your desk, get coffee at your desk. More convenient, available everywhere you are, works with everything else in your room

Both lobby machine and room machine make coffee. The room machine is newer, more convenient, integrates better with your room. But the lobby machine:

* Doesn't require installation in every room

* Works fine for a quick cup without committing to a room machine

* Is simpler for visitors who don't know the room setup

```
// toSignal full-featured, integrates with Signal graph

@Component({ ... })

export class FullFeaturedComponent {

readonly users = toSignal(this.service.getUsers(), { initialValue: [] });

readonly adminCount = computed(() =&gt; this.users().filter(u =&gt; u.role ===
'admin').length);

readonly hasAdmins = computed(() =&gt; this.adminCount() &gt; 0);

// Full integration derived values, used in computed, reactive everywhere

}


// async pipe simpler, display-only, lower cognitive overhead

@Component({

template: `

@if (user$ | async; as user) {

&lt;app-user-card [user]="user" /&gt;

}

`


})

export class SimpleDisplayComponent {

readonly user$ = this.service.getCurrentUser();

// Three lines no inject(DestroyRef), no toSignal, no initialValue

// When you JUST need to display something async pipe is genuinely simpler

}
```

When async pipe genuinely wins over toSignal:

```
1. Truly display-only data no derivation needed
```

```
async pipe is fewer lines and less ceremony

toSignal requires inject context and initialValue


2. Lazy loading async pipe subscribes when template renders

toSignal subscribes immediately at class creation

For expensive streams you only want active when visible: async pipe wins


3. Pre-Angular 17 / no Signals available

async pipe works in all Angular versions

toSignal requires @angular/core/rxjs-interop


4. Templates written by designers/non-engineers

async pipe is in the template visible to template authors

toSignal value is in the class requires understanding signals


5. Quick prototyping

async pipe: one pipe in the template

toSignal: inject DestroyRef, call toSignal, provide initialValue


When toSignal genuinely wins:

Need derived computed() values from the data

Need the value accessible in class methods

Building with Signals throughout consistency

Need initialValue to avoid null type

Multiple derivations from the same source data
```

The Angular team's actual guidance:

The Angular team has indicated that for new Angular 17+ applications built with Signals, toSignal() is preferred -- it integrates with the Signal graph and reduces the need for async pipe. However, they explicitly state that async pipe remains valid, supported, and idiomatic Angular -- it isn't deprecated and won't be.

The practical reality for most teams:

```
Pure Signal-based app (Angular 17+):

toSignal() for most cases

async pipe for lazy/conditional display scenarios
```

```
Both are valid Angular


Mixed Observable/Signal app (most real-world Angular):

async pipe for straightforward display

toSignal() when derivation or class access needed

takeUntilDestroyed for side effects


Legacy Observable app (Angular &amp;lt; 16):

async pipe everywhere it's the right tool

Manual subscribe + takeUntil(destroy$) when needed
```

Practical rule to take away:

> "async pipe and toSignal solve the same core problem differently -- both are correct, both are idiomatic Angular. Default to toSignal in new Signal-based Angular apps -- it integrates better with the reactivity model and eliminates null guards. Reach for async pipe when the component is purely display-focused with no derived logic, when you want simpler templates, or when working in pre-17 codebases. The test: do you need to derive or compute from this data in the class Yes -> toSignal. No -> either works, async pipe is simpler."

---

12. Related Topics to Learn Next

Angular-Specific Concepts

- Angular Signals (signal, computed, effect) (Topic 23 -- toSignal is the bridge -- understanding signals makes the async pipe vs toSignal decision obvious)*
- OnPush Change Detection (Topic 24 -- async pipe calls markForCheck() making it OnPush-compatible -- understanding why)*
- Component Lifecycle Hooks (Topic 25 -- async pipe has its own lifecycle -- understanding component lifecycle clarifies when it subscribes and unsubscribes)*
- Angular Reactive Forms (Topic 33 -- valueChanges Observable + async pipe is a common form pattern)*
- RxJS Operators in depth (Topic 46 -- startWith, shareReplay, catchError -- the operators that make async pipe production-ready)*

---

13. Updated Learning Tracker

```
Topics 1 19 ES6/JS + TypeScript Foundation COMPLETE

F1 F5 Angular Foundations Overview COMPLETE

Topic 20 RxJS Subscription Management COMPLETE

Topic 21 takeUntilDestroyed &amp;amp; DestroyRef COMPLETE

Topic 22 async pipe COMPLETE

Topic 23 Angular Signals NEXT
```

```
TIER 1 REMAINING:

23. Angular Signals (signal, computed, effect)

24. OnPush Change Detection

25. Component Lifecycle Hooks

26. @Input() &amp;amp; @Output() in depth

27. Services &amp;amp; Dependency Injection


TIER 2 UPCOMING:

28 36. Template Syntax HTTP Interceptors


TIER 3 WHEN NEEDED:

37 46. Directives RxJS Operators
```

Key Concept Added to Quick Reference
Topic 22 -- async pipe

> Personal assistant in the template -- subscribes, delivers, cancels when you leave. *ngIf as = one subscription for the whole block. No error handling -- always catchError first.

* async pipe = subscribes to Observable/Promise, displays current value, auto-unsubscribes Returns null before first emission -- always guard with fallback or ngIf as
* One | async = one subscription -- multiple pipes on same Observable = multiple HTTP calls ngIf as / @if as = subscribe once, use value everywhere in the block
* No error handling -- must add catchError to Observable chain before it reaches the pipe
* Import AsyncPipe directly in standalone components -- not CommonModule
* shareReplay(1) = share one execution across multiple subscribers
* async pipe vs toSignal: pure display -> async pipe; derived logic -> toSignal

Topic 23 -- Angular Signals (signal, computed, effect) in Angular

0. Difficulty & Priority Rating

* Difficulty: Intermediate -- signal() and computed() click immediately; effect() has important rules that take practice; the mental shift from Observable-thinking to Signal-thinking is the real challenge
* Angular Relevance: Critical -- Signals are Angular's modern reactivity model introduced in Angular 16 and made stable in Angular 17; they replace BehaviorSubject for most state management, eliminate zone.js dependency for change detection, and are the foundation of Angular's future architecture

1. Explanation

The Spreadsheet Analogy
Imagine a spreadsheet with three cells:

* Cell A1 = 150 (your salary)
* Cell B1 = 0.2 (your tax rate)

Cell C1 = =A1 B1 (your tax amount -- a formula)

When you change A1 from 150 to 200, Cell C1 automatically updates to 40. You didn't tell C1 to recalculate -- the spreadsheet knows C1 depends on A1, so it updates C1 automatically the moment A1 changes.

Angular Signals work exactly like this spreadsheet.

* signal() = Cell A1 -- a value you can read and change
* computed() = Cell C1 -- a formula that automatically recalculates when its dependencies change
* effect() = a macro that runs automatically whenever cells it reads change -- like a formula that sends an email when the tax amount changes

```
signal(150) Cell A1 raw value, you control it

computed(()=&gt;a*b) Cell C1 formula, auto-updates

effect(()=&gt;log(c)) macro runs when C changes
```

The Smart Home Analogy
Think of a smart home system:

* signal() = the thermostat dial -- you set the temperature value
* computed() = the display screen -- automatically shows whether to heat or cool based on current temperature vs target
* effect() = the boiler -- when the display says "heat", the boiler automatically turns on

Each part knows what it depends on and reacts automatically. You don't tell the boiler to turn on -- it watches the display, which watches the thermostat. Change the dial, everything else updates in a chain.

In Plain English

* signal(value) = a reactive variable -- when its value changes, everything that reads it is notified automatically
* computed(fn) = a derived value -- reads from signals, automatically recalculates when any read signal changes, result is cached until dependencies change

* effect(fn) = a side effect runner -- runs whenever any signal it reads changes, used for logging, syncing with external systems, DOM manipulation

Reactivity = the key idea -- signals track who reads them*, so when they change, they know exactly what needs to update

* No subscription needed = unlike Observables, signals don't need .subscribe() -- Angular handles the reactivity graph automatically

Now the Technical Terms

* signal<T>(initialValue) = creates a WritableSignal<T> -- a reactive container holding a value

* signal.set(value) = replaces the current value -- triggers all dependents to update

* signal.update(fn) = computes new value from current -- count.update(c => c + 1) -- immutable update pattern

* signal.mutate(fn) = mutates the current value in place (deprecated in newer Angular -- use update with spread instead)

* computed(fn) = creates a read-only Signal<T> -- Angular memoises the result -- only recomputes when dependencies change

* effect(fn, options) = registers a side effect -- runs immediately and re-runs when any signal read inside it changes

* toSignal(observable$) = converts Observable to Signal -- bridges RxJS and Signals (Topic 21)

* toObservable(signal) = converts Signal to Observable -- bridge going the other direction

---

2. Connection to Prior Concepts

Signals are Angular's answer to the complexity accumulated across Topics 20-22:

* Topic 20 (RxJS Subscription Management) -- Signals eliminate subscription management entirely for state. No .subscribe(), no takeUntilDestroyed, no memory leaks -- the reactivity graph is managed by Angular automatically

* Topic 21 (takeUntilDestroyed) -- toSignal() from Topic 21 is the bridge between Observables and Signals. Once you convert to a Signal, subscription management disappears -- Angular handles it

* Topic 22 (async pipe) -- Signals in templates are accessed with () -- no | async pipe needed. {{ count() }} vs {{ count$ | async }}. Signals are simpler because they're synchronous

* Topic 7 (Pass by Reference) -- signal.update(c => c + 1) is the immutable update pattern from Topic 7. Signals work best with immutable updates -- update with spread instead of direct mutation

* Topic 14 (readonly) -- computed() returns a read-only Signal -- you can read it but never set it directly. Same philosophy as readonly properties -- derived values shouldn't be manually overwritten

Bridging note for your records:

"Signals are the spreadsheet model applied to Angular. signal() is the raw cell, computed() is the formula cell, effect() is the macro. Unlike Observables (which are push-based streams), Signals are synchronous reactive values -- you read them with (), Angular tracks the dependency graph automatically, everything that depends on a changed signal updates immediately."

---

3. Code Example

Example 1 -- signal() -- the reactive variable

Create, read, set, and update a signal -- the three operations you'll use constantly.

```
import { signal } from '@angular/core';
```

```
// ---- CREATE a signal ----

const count = signal(0); // WritableSignal&lt;number&gt;

const name = signal('Alice'); // WritableSignal&lt;string&gt;

const isLoading = signal(false); // WritableSignal&lt;boolean&gt;

const users = signal&lt;User[]&gt;([]); // WritableSignal&lt;User[]&gt;

const user = signal&lt;User | null&gt;(null); // WritableSignal&lt;User | null&gt;


// ---- READ a signal call it like a function ----

console.log(count()); // 0

console.log(name()); // 'Alice'

console.log(users()); // []


// ---- SET replace the value entirely ----

count.set(5);

console.log(count()); // 5


name.set('Bob');

console.log(name()); // 'Bob'


// ---- UPDATE compute new value from current ----

// The right way for immutable updates Topic 7 in action

count.update(current =&gt; current + 1);

console.log(count()); // 6


// Immutable object update spread to create new reference

user.update(current =&gt; current

{ ...current, name: 'Alice Smith' }

: current

);


// Immutable array update new array, new reference

users.update(current =&gt; [...current, newUser]); // add
```

```
users.update(current =&gt; current.filter(u =&gt; u.id !== id)); // remove

users.update(current =&gt;

current.map(u =&gt; u.id === id { ...u, ...changes } : u) // update one

);
```

Example 2 -- computed() -- the formula cell

Derived values that automatically recalculate when their dependencies change.

```
import { signal, computed } from '@angular/core';


const price = signal(100);

const quantity = signal(3);

const taxRate = signal(0.2);


// ---- computed formula that reads signals ----

const subtotal = computed(() =&gt; price() * quantity());

// Reads price() and quantity() Angular tracks both as dependencies


const tax = computed(() =&gt; subtotal() * taxRate());

// Reads subtotal() and taxRate()

// subtotal reads price and quantity dependency chain is tracked


const total = computed(() =&gt; subtotal() + tax());


// ---- Reading computed values ----

console.log(subtotal()); // 300

console.log(tax()); // 60

console.log(total()); // 360


// ---- When a dependency changes all dependents update ----

price.set(150);

// Angular knows: price changed subtotal depends on price recompute subtotal

// tax depends on subtotal recompute tax

// total depends on tax recompute total

// All in one synchronous update no async, no subscription
```

```
console.log(subtotal()); // 450

console.log(tax()); // 90

console.log(total()); // 540


// ---- computed is MEMOISED only recomputes when dependencies change ----

console.log(total()); // 540 cached result, no recomputation

console.log(total()); // 540 still cached

price.set(200); // price changed now recomputes on next read

console.log(total()); // 720 recomputed once, then cached again



// ---- computed with objects and arrays ----

const users = signal&lt;User[]&gt;([

{ id: 1, name: 'Alice', role: 'admin', isActive: true },

{ id: 2, name: 'Bob', role: 'user', isActive: false },

{ id: 3, name: 'Carol', role: 'user', isActive: true }

]);


const searchTerm = signal('');

const filterRole = signal&lt;string&gt;('all');


// Derived list filters reapply automatically when users, searchTerm, or filterRole
change

const filteredUsers = computed(() =&gt; {

const term = searchTerm().toLowerCase();

const role = filterRole();


return users().filter(user =&gt; {

const matchesTerm = !term || user.name.toLowerCase().includes(term);

const matchesRole = role === 'all' || user.role === role;

return matchesTerm &amp;&amp; matchesRole &amp;&amp; user.isActive;

});

});
```

```
const filteredCount = computed(() =&gt; filteredUsers().length);

const hasResults = computed(() =&gt; filteredCount() &gt; 0);


// Changing searchTerm automatically updates filteredUsers, filteredCount, hasResults

searchTerm.set('ali');

console.log(filteredUsers()); // [{ name: 'Alice', ... }]

console.log(filteredCount()); // 1

console.log(hasResults()); // true
```

Example 3 -- effect() -- the side effect runner

Runs when signals it reads change -- for logging, external sync, DOM work.

```
import { signal, computed, effect } from '@angular/core';


const theme = signal&lt;'light' | 'dark'&gt;('light');

const userId = signal&lt;number | null&gt;(null);

const isLoggedIn = computed(() =&gt; userId() !== null);


// ---- effect runs immediately, re-runs when dependencies change ----


// Side effect 1: sync theme to DOM

effect(() =&gt; {

// Reads theme() so it re-runs whenever theme changes

document.body.setAttribute('data-theme', theme());

console.log('Theme changed to:', theme());

});

// Runs immediately: sets data-theme="light"


theme.set('dark');

// Runs again: sets data-theme="dark"



// Side effect 2: analytics tracking

effect(() =&gt; {

if (isLoggedIn()) {
```

```
// Reads isLoggedIn() tracks userId dependency too via computed

analytics.track('user_active', { userId: userId() });

}

});

// Side effect 3: localStorage sync

effect(() =&gt; {

// Sync signal state to localStorage automatically

localStorage.setItem('theme', theme());

});


// ---- IMPORTANT: effects run ASYNCHRONOUSLY ----

// Not immediately synchronous scheduled by Angular's change detection

theme.set('dark');

// effect doesn't fire right here synchronously

// it fires in Angular's next effect flush cycle

// ---- Writing to signals INSIDE effect needs allowSignalWrites ----

const lastTheme = signal&lt;string&gt;('');

// Writing to signal inside effect without option warning in dev

effect(() =&gt; {

lastTheme.set(theme()); // writing to signal inside effect

});

// With allowSignalWrites option

effect(() =&gt; {

lastTheme.set(theme()); // allowed with the option

}, { allowSignalWrites: true });

// Better pattern: use computed instead when possible

const lastTheme2 = computed(() =&gt; theme()); // no effect needed
```

Angular Example -- Signals in a real Angular component

A complete component using signal, computed, and effect together.

```
import {

Component, signal, computed, effect,

inject, OnInit

} from '@angular/core';

import { toSignal } from '@angular/core/rxjs-interop';

import { CommonModule } from '@angular/common';

import { FormsModule } from '@angular/forms';


@Component({

selector: 'app-product-catalogue',

standalone: true,

imports: [CommonModule, FormsModule],

template: `

&lt;!-- Signals accessed with () no async pipe needed --&gt;


&lt;!-- Search input two-way bound to signal via ngModel --&gt;

&lt;input

[ngModel]="searchTerm()"

(ngModelChange)="searchTerm.set($event)"

placeholder="Search products..."&gt;


&lt;!-- Filter buttons --&gt;

&lt;div class="filters"&gt;

@for (cat of categories(); track cat) {

&lt;button

[class.active]="selectedCategory() === cat"

(click)="selectedCategory.set(cat)"&gt;

{{ cat }}

&lt;/button&gt;

}
```

```
&lt;/div&gt;

&lt;!-- Results summary computed updates automatically --&gt;
&lt;p&gt;
Showing {{ filteredProducts().length }}
of {{ products().length }} products
&lt;/p&gt;

&lt;!-- Loading state from HTTP Observable via toSignal --&gt;
@if (isLoading()) {
&lt;div&gt;Loading products...&lt;/div&gt;
}

&lt;!-- Product list filteredProducts is computed --&gt;
@for (product of filteredProducts(); track product.id) {
&lt;div class="product-card"
[class.selected]="selectedProduct() .id === product.id"
(click)="selectProduct(product)"&gt;
&lt;h3&gt;{{ product.name }}&lt;/h3&gt;
&lt;p&gt; {{ product.price.toFixed(2) }}&lt;/p&gt;
&lt;span [class]="'badge badge--' + product.category"&gt;
{{ product.category }}
&lt;/span&gt;
&lt;/div&gt;
}

@if (!isLoading() &amp;&amp; filteredProducts().length === 0) {
&lt;p&gt;No products match your search&lt;/p&gt;
}

&lt;!-- Selected product detail --&gt;
@if (selectedProduct(); as product) {
```

```
&lt;div class="detail-panel"&gt;

&lt;h2&gt;{{ product.name }}&lt;/h2&gt;

&lt;p&gt;{{ product.description }}&lt;/p&gt;

&lt;p&gt;Price: {{ product.price.toFixed(2) }}&lt;/p&gt;

&lt;p&gt;In stock: {{ product.stock }}&lt;/p&gt;

&lt;/div&gt;

}

`

})

export class ProductCatalogueComponent {


private readonly productService = inject(ProductService);

private readonly analytics = inject(AnalyticsService);


// ---- toSignal HTTP Observable Signal (Topic 21) ----

// Subscription managed internally by Angular

private readonly productsData = toSignal(

this.productService.getProducts(),

{ initialValue: { products: [], isLoading: true } }

);


// ---- Derived signals from HTTP data ----

readonly products = computed(() =&gt; this.productsData().products);

readonly isLoading = computed(() =&gt; this.productsData().isLoading);


// ---- Writable signals user-controlled state ----

readonly searchTerm = signal('');

readonly selectedCategory = signal('all');

readonly selectedProduct = signal&lt;Product | null&gt;(null);


// ---- Derived categories from products list ----

readonly categories = computed(() =&gt; [
```

```
'all',
...new Set(this.products().map(p =&gt; p.category))
]);

// ---- Filtered list recomputes when products, searchTerm, or category changes ----
readonly filteredProducts = computed(() =&gt; {
const term = this.searchTerm().toLowerCase();
const category = this.selectedCategory();

return this.products()
.filter(p =&gt; {
const matchesTerm = !term || p.name.toLowerCase().includes(term);
const matchesCategory = category === 'all' || p.category === category;
return matchesTerm &amp;&amp; matchesCategory;
});
});

// ---- effect analytics side effect ----
// Tracks every product selection automatically
constructor() {
effect(() =&gt; {
const product = this.selectedProduct();
if (product) {
// Runs whenever selectedProduct signal changes
this.analytics.track('product_viewed', { productId: product.id });
}
});

// Effect: sync selected category to URL query params
effect(() =&gt; {
const category = this.selectedCategory();
// Update URL without navigation side effect on DOM/browser
```

```
const url = new URL(window.location.href);

url.searchParams.set('category', category);

window.history.replaceState({}, '', url.toString());

});

}


// ---- Methods update signals UI reacts automatically ----

selectProduct(product: Product): void {

this.selectedProduct.set(product);

// effect fires automatically analytics tracked

// template updates automatically detail panel shown

}


clearSearch(): void {

this.searchTerm.set('');

this.selectedCategory.set('all');

// filteredProducts recomputes automatically

}

}
```

4. Before & After Refactor

The Problem in Plain English:

Before Signals, component state in Angular needed BehaviorSubject for reactivity -- complex, verbose, easy to get wrong. A shopping cart component with items, totals, and loading state needed a BehaviorSubject, manual pipe chains for derived values, subscriptions to manage, and ngOnDestroy to clean up. Signals collapse all of that into plain reactive variables -- no Subjects, no pipes, no subscriptions, no cleanup.

```
// BEFORE BehaviorSubject-based state

// Complex, verbose, subscription-heavy


@Component({ selector: 'app-cart', template: '...' })

export class CartComponent implements OnInit, OnDestroy {


// Three BehaviorSubjects state containers
```

```
private readonly itemsSubject = new BehaviorSubject&lt;CartItem[]&gt;([]);

private readonly isLoadingSubject = new BehaviorSubject&lt;boolean&gt;(false);

private readonly errorSubject = new BehaviorSubject&lt;string | null&gt;(null);


// Public Observables from subjects for template

readonly items$ = this.itemsSubject.asObservable();

readonly isLoading$ = this.isLoadingSubject.asObservable();

readonly error$ = this.errorSubject.asObservable();


// Derived Observables complex pipe chains

readonly itemCount$ = this.items$.pipe(

map(items =&gt; items.length)

);

readonly total$ = this.items$.pipe(

map(items =&gt; items.reduce((sum, i) =&gt; sum + i.price * i.qty, 0))

);

readonly isEmpty$ = this.items$.pipe(

map(items =&gt; items.length === 0)

);

readonly hasItems$ = this.items$.pipe(

map(items =&gt; items.length &gt; 0)

);


private readonly destroy$ = new Subject&lt;void&gt;();


constructor(private readonly cartService: CartService) {}


ngOnInit(): void {

// Subscription to load initial cart

this.isLoadingSubject.next(true);

this.cartService.getCart().pipe(

takeUntil(this.destroy$)
```

```
).subscribe({

next: items =&gt; {

this.itemsSubject.next(items);

this.isLoadingSubject.next(false);

},

error: err =&gt; {

this.errorSubject.next(err.message);

this.isLoadingSubject.next(false);

}

});

}


addItem(item: CartItem): void {

const current = this.itemsSubject.getValue();

this.itemsSubject.next([...current, item]);

}


removeItem(id: number): void {

const current = this.itemsSubject.getValue();

this.itemsSubject.next(current.filter(i =&gt; i.id !== id));

}


ngOnDestroy(): void {

this.destroy$.next();

this.destroy$.complete();

}

// 60+ lines 3 subjects, 7 derived observables, ngOnDestroy

}



// AFTER Signal-based state

// Clean, direct, no subscriptions, no cleanup
```

```
@Component({ selector: 'app-cart', template: '...' })

export class CartComponent {


private readonly cartService = inject(CartService);


// ---- Writable signals the source of truth ----

readonly items = signal&lt;CartItem[]&gt;([]);

readonly isLoading = signal(false);

readonly error = signal&lt;string | null&gt;(null);


// ---- Computed signals derived automatically ----

readonly itemCount = computed(() =&gt; this.items().length);

readonly total = computed(() =&gt;

this.items().reduce((sum, i) =&gt; sum + i.price * i.qty, 0)

);

readonly isEmpty = computed(() =&gt; this.items().length === 0);

readonly hasItems = computed(() =&gt; this.items().length &gt; 0);


constructor() {

// Load initial cart toSignal handles subscription

const cartData = toSignal(

this.cartService.getCart().pipe(

catchError(err =&gt; {

this.error.set(err.message);

return of([]);

})

),

{ initialValue: [] as CartItem[] }

);


// Effect to sync toSignal data into items signal

effect(() =&gt; {
```

```
this.items.set(cartData());

}, { allowSignalWrites: true });

}


addItem(item: CartItem): void {

this.items.update(current =&gt; [...current, item]); // immutable

}


removeItem(id: number): void {

this.items.update(current =&gt;

current.filter(i =&gt; i.id !== id) // immutable

);

}


// No ngOnInit, no ngOnDestroy, no destroy$

// 30 lines half the code, twice the clarity

}
```

Why it matters: The Signal version is half the lines of the BehaviorSubject version. More importantly -- it's readable. signal([]) says "a reactive list". computed(() => items().length) says "a derived count". The BehaviorSubject version requires understanding RxJS operators, subscription management, and the Subject/Observable pattern. The Signal version requires understanding signal() and computed() -- far simpler mental model.

---

5. Common Mistakes & Misconceptions

"Signals are just a simpler BehaviorSubject -- they're the same thing with less syntax"

Signals and BehaviorSubject solve similar problems but have fundamentally different models. BehaviorSubject is a push-based Observable stream -- it pushes values to subscribers asynchronously. Signals are synchronous reactive values -- they update synchronously, Angular tracks dependencies automatically, and they integrate with Angular's change detection at a fundamental level.

Think of it like: BehaviorSubject is a newsagent that delivers newspapers when new editions arrive. Signal is a live billboard -- you look at it whenever you want and it always shows the current state. One pushes, one you pull.

```
// BehaviorSubject push model

const subject = new BehaviorSubject(0);

subject.subscribe(value =&gt; console.log(value)); // must subscribe
```

```
subject.next(1); // push new value to subscribers

// Async, subscription-based, must unsubscribe


// Signal pull model (with automatic tracking)

const count = signal(0);

console.log(count()); // read directly no subscribe needed

count.set(1); // set new value dependents update synchronously

// Synchronous, no subscription, no cleanup


// Key differences:

// Signals: synchronous, no subscription, tracked by Angular's renderer

// BehaviorSubject: async, requires subscribe, must unsubscribe

// Both reactive different paradigms
```

"I should use effect() to update other signals when a signal changes"

This is the most common Signal misuse. Using effect() to update signals creates hard-to-debug reactive chains where it's unclear what causes what. The Angular team explicitly recommends using computed() for derived values -- effect() is only for side effects that interact with the outside world (DOM, localStorage, analytics, HTTP).

Think of it like this -- in the spreadsheet analogy, Cell C1 doesn't need a macro to update when A1 changes. The formula =A1B1 handles it. Only use a macro when you need to do* something outside the spreadsheet -- like sending an email.

```
// Using effect to update another signal wrong pattern

const count = signal(0);

const double = signal(0);


effect(() =&gt; {

double.set(count() * 2); // using effect to derive a value

}, { allowSignalWrites: true });

// Hard to follow, creates reactive loops, Angular warns about this


// Use computed for derived values always

const count = signal(0);

const double = computed(() =&gt; count() * 2); // clean, obvious, efficient


// Use effect ONLY for side effects outside the Signal graph
```

```
effect(() =&gt; {

// DOM, localStorage, analytics, console things OUTSIDE Angular's world

localStorage.setItem('count', String(count()));

console.log('Count changed:', count());

document.title = `Count: ${count()}`;

});
```

"Signals replace RxJS completely -- I don't need Observables anymore"

Signals are excellent for state management -- values that change over time and need to be reflected in the UI. Observables are excellent for async operations -- HTTP calls, WebSockets, complex operator chains. They solve different problems and work best together.

Think of Signals as the noticeboard in an office (always shows the current state) and Observables as the postal system (delivers packages -- some arrive once, some in a stream, some need processing before delivery). The noticeboard and the postal system are different tools -- the postal system delivers new notices TO the noticeboard.

```
@Component({ ... })

export class OrderComponent {


private readonly orderService = inject(OrderService);


// ---- Signals for STATE what the UI currently shows ----

readonly selectedStatus = signal&lt;OrderStatus&gt;('all'); // user filter

readonly searchQuery = signal(''); // user search input

readonly currentPage = signal(1); // pagination


// ---- toSignal bridges Observable Signal ----

// HTTP call (Observable) reactive state (Signal)

private readonly allOrders = toSignal(

this.orderService.getOrders(), // Observable HTTP, async

{ initialValue: [] as Order[] }

);


// ---- Computed for DERIVED STATE ----

readonly filteredOrders = computed(() =&gt; {

const status = this.selectedStatus();
```

```
const query = this.searchQuery().toLowerCase();

return this.allOrders()

.filter(o =&gt; status === 'all' || o.status === status)

.filter(o =&gt; o.id.toString().includes(query));

});


// ---- Methods use Observables for ACTIONS ----

saveOrder(order: Order): void {

// HTTP POST Observable makes sense here one-shot async operation

this.orderService.save(order).pipe( // Observable

takeUntilDestroyed(this.destroyRef) // still needed for actions

).subscribe(saved =&gt; {

// Update Signal state after successful HTTP action

this.allOrders.update(orders =&gt; [...orders, saved]);

});

}

}

// Signals for state Observables for async operations both together
```

6. Do's and Don'ts

| [Y] Do | [N] Don't |

|---|---|

| Use computed() for ALL derived values -- it's memoised and auto-updates | Use effect() to update other signals -- use computed() instead |

| Use signal.update(fn) with spread for immutable object/array updates | Use signal.set() with mutation -- items.set(items().push(x)) |

| Use effect() only for side effects outside the Signal graph -- DOM, localStorage, analytics | Use effect() for business logic or derived state -- that's computed()'s job |

| Declare effect() in the constructor -- injection context required | Declare effect() in ngOnInit -- not an injection context |

| Use toSignal() to bridge HTTP Observables into the Signal graph | Keep everything as Observables when Signals are a better fit for state |

| Use signal<Type | null>(null) with explicit type -- makes null state clear | Use untyped signals -- TypeScript inference from initial value is often too narrow |

7. Debugging Tips

Bug: effect() throws NG0203: inject() must be called from an injection context

```
Error: NG0203 effect() must be called in injection context
```

* Plain English cause: effect() called in ngOnInit or a method -- not in constructor or field initialiser

* Fix: Move effect() calls to the constructor -- the injection context

```
// effect in ngOnInit not injection context

ngOnInit(): void {

effect(() =&gt; { ... }); // Error

}


// effect in constructor injection context

constructor() {

effect(() =&gt; { ... }); //

}
```

Bug: Infinite loop -- effect keeps firing

```
Maximum call stack exceeded / browser freezes
```

* Plain English cause: Effect reads a signal AND writes to a signal it depends on -- creates a circular dependency. Signal changes -> effect runs -> signal changes -> effect runs -> infinite loop

* Fix: Use computed() instead of writing signals in effects. If you must write in an effect, use untracked() to read without creating a dependency

```
// Circular effect reads count, writes double which reads count

const count = signal(0);

const double = signal(0);

effect(() =&gt; {

double.set(count() * 2); // circular if double feeds back to count

}, { allowSignalWrites: true });


// computed no circular dependency possible

const double = computed(() =&gt; count() * 2);
```

Bug: computed() not updating when expected

```
Computed signal shows stale value
```

Plain English cause: The computed function doesn't read the signal inside it -- it read a non-reactive value. Computed only tracks what it calls with ()* inside the function

* Fix: Ensure you're reading the signal with () inside the computed -- not a snapshot taken outside

```
// Reading signal OUTSIDE computed not tracked

const currentItems = items(); // snapshot not reactive

const count = computed(() =&gt; currentItems.length); // never updates


// Reading signal INSIDE computed tracked

const count = computed(() =&gt; items().length); // updates when items changes
```

Bug: Template showing [object Object] or undefined instead of signal value

```
Template shows [object Object]
```

* Plain English cause: Forgot the () when reading the signal in the template -- accessing the signal object itself, not its value

* Fix: Always call signals with () in templates -- {{ count() }} not {{ count }}

```
&lt;!-- Missing () shows [object Signal] --&gt;

&lt;p&gt;{{ count }}&lt;/p&gt;


&lt;!-- With () shows the value --&gt;

&lt;p&gt;{{ count() }}&lt;/p&gt;
```

---

8. Real-World Applications

Application 1 -- Shopping Cart with Signal-Based State
Plain English first:

A shopping cart is the classic Signal use case -- multiple pieces of derived state that all depend on the same underlying list. The item count, subtotal, discount amount, final total, and "is empty" flag all derive from the items list. With Signals, you write the items list once as a signal() and everything else as computed() -- change the list and every derived value updates automatically. No manual event coordination, no subscription chains, no intermediate subjects.

```
@Injectable({ providedIn: 'root' })

export class CartService {


// ---- Source of truth ----

private readonly _items = signal&lt;CartItem[]&gt;([]);

private readonly _voucher = signal&lt;string | null&gt;(null);


// ---- Public read-only access ----

readonly items = this._items.asReadonly(); // read-only signal

readonly voucher = this._voucher.asReadonly();
```

```
// ---- All derived values update automatically ----

readonly itemCount = computed(() =&gt; this._items().length);


readonly subtotal = computed(() =&gt;

this._items().reduce((sum, item) =&gt; sum + item.price * item.qty, 0)

);


readonly discount = computed(() =&gt; {

const code = this._voucher();

if (!code) return 0;

// Check voucher code and apply discount

return code === 'SAVE10' this.subtotal() * 0.1 : 0;

});


readonly total = computed(() =&gt; this.subtotal() - this.discount());


readonly isEmpty = computed(() =&gt; this._items().length === 0);

readonly hasVoucher = computed(() =&gt; this._voucher() !== null);


readonly itemsSummary = computed(() =&gt;

this._items().map(i =&gt; `${i.name} x${i.qty}`).join(', ')

);


// ---- Mutations all use immutable update pattern ----

addItem(product: Product): void {

this._items.update(items =&gt; {

const existing = items.find(i =&gt; i.id === product.id);

if (existing) {

// Update quantity immutable

return items.map(i =&gt;

i.id === product.id { ...i, qty: i.qty + 1 } : i

);

}
```

```
return [...items, { ...product, qty: 1 }]; // add new

});

}


removeItem(id: number): void {

this._items.update(items =&gt; items.filter(i =&gt; i.id !== id));

}


updateQty(id: number, qty: number): void {

if (qty &lt;= 0) { this.removeItem(id); return; }

this._items.update(items =&gt;

items.map(i =&gt; i.id === id { ...i, qty } : i)

);

}


applyVoucher(code: string): void { this._voucher.set(code); }

clearVoucher(): void { this._voucher.set(null); }

clearCart(): void { this._items.set([]); }

}
```

Application 2 -- Signal-Based Search and Filter

Plain English first:

A product catalogue with live search and filtering is where computed signals shine. The user types a search term (one signal), selects a category (another signal), and the filtered list (a computed) updates instantly. No debouncing needed for in-memory filtering -- signals are synchronous. No subscription setup, no manual filter logic triggered by events -- just signals and one computed that describes the relationship between inputs and output.

```
@Component({

selector: 'app-catalogue',

standalone: true,

imports: [CommonModule, FormsModule],

template: `

&lt;div class="controls"&gt;

&lt;input
```

```
[ngModel]="search()"

(ngModelChange)="search.set($event)"

placeholder="Search..."&amp;gt;


&amp;lt;select

[ngModel]="category()"

(ngModelChange)="category.set($event)"&amp;gt;

@for (cat of availableCategories(); track cat) {

&amp;lt;option [value]="cat"&amp;gt;{{ cat }}&amp;lt;/option&amp;gt;

}

&amp;lt;/select&amp;gt;


&amp;lt;select

[ngModel]="sortBy()"

(ngModelChange)="sortBy.set($event)"&amp;gt;

&amp;lt;option value="name"&amp;gt;Name&amp;lt;/option&amp;gt;

&amp;lt;option value="price-asc"&amp;gt;Price &amp;lt;/option&amp;gt;

&amp;lt;option value="price-desc"&amp;gt;Price &amp;lt;/option&amp;gt;

&amp;lt;/select&amp;gt;


&amp;lt;button (click)="clearFilters()"&amp;gt;Clear&amp;lt;/button&amp;gt;

&amp;lt;/div&amp;gt;


&amp;lt;p class="results-count"&amp;gt;

{{ results().length }} results

@if (isFiltered()) { &amp;lt;span&amp;gt;(filtered from {{ allProducts().length
}})&amp;lt;/span&amp;gt; }

&amp;lt;/p&amp;gt;


&amp;lt;div class="grid"&amp;gt;

@for (product of results(); track product.id) {

&amp;lt;app-product-card [product]="product" /&amp;gt;

} @empty {
```

```
&lt;p&gt;No products match your filters&lt;/p&gt;

}

&lt;/div&gt;

`

})

export class CatalogueComponent {


private readonly productService = inject(ProductService);


// ---- Source data from HTTP via toSignal ----

readonly allProducts = toSignal(

this.productService.getAll(),

{ initialValue: [] as Product[] }

);


// ---- User input signals ----

readonly search = signal('');

readonly category = signal('all');

readonly sortBy = signal&lt;'name' | 'price-asc' | 'price-desc'&gt;('name');


// ---- All derived all automatic ----

readonly availableCategories = computed(() =&gt; [

'all',

...new Set(this.allProducts().map(p =&gt; p.category))

]);


readonly results = computed(() =&gt; {

const term = this.search().toLowerCase();

const cat = this.category();

const sort = this.sortBy();


return [...this.allProducts()] // spread don't mutate original

.filter(p =&gt;
```

```
(!term || p.name.toLowerCase().includes(term)) &amp;&amp;

(cat === 'all' || p.category === cat)

)

.sort((a, b) =&gt; {

if (sort === 'name') return a.name.localeCompare(b.name);

if (sort === 'price-asc') return a.price - b.price;

if (sort === 'price-desc') return b.price - a.price;

return 0;

});

});


readonly isFiltered = computed(() =&gt;

this.search() !== '' || this.category() !== 'all'

);


clearFilters(): void {

this.search.set('');

this.category.set('all');

// results() recomputes automatically

}

}
```

9. Practice Exercises

Exercise 1 -- Beginner

Build a CounterComponent that has: a count signal starting at 0, increment() and decrement() methods, a minimum of 0 and maximum of 10 (use update with clamping), and three computed values: isAtMin, isAtMax, and displayCount (shows "MAX" when at 10, "MIN" when at 0, otherwise the number). Display all in the template. Disable the increment button when isAtMax() is true and the decrement button when isAtMin() is true. Add an effect() that logs to the console every time the count changes.

Exercise 2 -- Intermediate

Build a TodoListComponent with full signal-based state. The todos signal holds an array of { id, title, completed, priority }. Implement: addTodo(title, priority), toggleTodo(id), deleteTodo(id), and clearCompleted(). Add computed signals for: activeTodos, completedTodos, totalCount, activeCount, completedCount, and completionPercentage. Add a filter signal ('all' | 'active' | 'completed') and a

filteredTodos computed that respects it. The component class should have zero manual subscription management -- only signals and computeds.

Exercise 3 -- Advanced

Build a DataGridComponent that receives data via an HTTP service using toSignal. Implement signal-based: multi-column sorting (sort column + direction as signals), multi-field search (separate signal per searchable field), pagination (page signal + pageSize signal), row selection (selected IDs as a Set in a signal), and bulk actions (delete selected, export selected). All derived values -- sortedData, filteredData, paginatedData, selectedCount, allSelected -- must be computed() signals. Add one effect() that persists the current page and sort preference to localStorage. Zero manual subscriptions anywhere in the component.

---

10. Key Takeaways

Core Concepts in Plain English

* signal(value) = reactive variable -- read with (), change with .set() or .update()
* computed(fn) = formula cell -- reads signals, auto-recalculates, memoised until deps change
* effect(fn) = side effect runner -- runs when signals it reads change, for DOM/localStorage/analytics ONLY
* Synchronous = signals update synchronously -- unlike Observables which are async
* No subscription needed = Angular tracks the dependency graph -- no .subscribe(), no cleanup
* update over set = use .update(fn) with spread for immutable object/array changes
* computed over effect = if you're deriving a value -- always use computed, never effect

The Signal Mental Model

```
signal(initialValue) raw cell you control it

computed(() =&gt; a() + b()) formula cell auto-updates

effect(() =&gt; sync(a())) macro runs on change, outside world only


Reading: value() synchronous, no subscribe

Setting: value.set(x) triggers all dependents immediately

Updating: value.update(fn) immutable pattern fn(current) new value
```

Quick Reference Table

| | signal | computed | effect |

|---|---|---|---|

| Writable | [Y] | [N] read-only | [N] |

| Auto-updates | N/A | [Y] when deps change | [Y] when deps change |

| Returns value | [Y] | [Y] | [N] void |

| Use for | Source state | Derived state | Side effects |

| Memoised | N/A | [Y] | N/A |

| Where to declare | Anywhere | Anywhere | Constructor only |

Angular-Specific Rules

* effect() must be in the constructor -- not ngOnInit
* Never write to signals inside effect() without allowSignalWrites: true -- use computed() instead
* Always use .update() with spread for object/array signals -- immutable updates (Topic 7 + 14)
* signal.asReadonly() in services -- expose read-only signals, keep writable private
* toSignal() bridges HTTP Observables into the Signal graph -- always with initialValue
* Signals replace BehaviorSubject for state -- Observables remain for HTTP and streams
* Template syntax: {{ signal() }} not {{ signal }} -- always call with ()

One-Line Memory Aid
> signal() = reactive variable (raw cell). computed() = formula that auto-updates (never use effect for this). effect() = side effect for the outside world only -- DOM, localStorage, analytics. Read with (). Update with spread. No subscriptions, no cleanup.

---

11. Thought-Provoking Question + Answer

> Angular's new input() and output() functions in Angular 17+ are signal-based alternatives to @Input() and @Output() decorators. If component inputs are now signals, the entire component becomes a reactive graph -- parent passes a signal input, child computes from it, grandchild computes from that. What does this signal-based component tree mean for OnPush change detection, and does it make OnPush obsolete

Plain English explanation first:

Imagine a traditional factory assembly line (old Angular). Each station gets a notification slip when upstream changes -- "Part changed at station 3 -- check if you need to update." Most stations check and do nothing -- wasted work. OnPush was invented to reduce these useless checks -- "Only check station 5 if someone explicitly pushes a new part to it."

Now imagine the new factory (Signals). Every station has a sensor directly connected to its upstream parts. When a part changes, only the stations whose sensors actually triggered get updated. No notification slips flying around. No stations checking for no reason. The sensors are so precise that OnPush becomes redundant -- the system already knows exactly what changed and only updates exactly what needs to change.

```
// OLD @Input() + OnPush OnPush reduces unnecessary checks

@Component({

changeDetection: ChangeDetectionStrategy.OnPush, // needed for perf

})

export class OldChild {

@Input() user: User; // plain value change detection checks this

// OnPush: only re-check when user reference changes

// Still re-checks the whole component tree on every reference change

}


// NEW signal input() granular reactivity, OnPush less critical
```

```
@Component({

// OnPush still useful but less critical with signal inputs

})

export class NewChild {

readonly user = input&lt;User&gt;(); // signal input Angular 17+


// These only recompute when their specific signal deps change

readonly userName = computed(() =&gt; this.user() .name);

readonly userEmail = computed(() =&gt; this.user() .email);


// Template only updates the SPECIFIC parts that read changed signals

// Not the whole component tree surgical updates

}
```

The technical reality -- what signal inputs change:

```
// Signal-based component tree Angular 17+

@Component({

template: `&lt;app-child [user]="currentUser" /&gt;`

})

export class ParentComponent {

currentUser = signal&lt;User&gt;({ id: 1, name: 'Alice', email: 'alice@example.com' });


updateEmail(): void {

// Immutable update new User object

this.currentUser.update(u =&gt; ({ ...u, email: 'newalice@example.com' }));

}

}


@Component({

template: `

&lt;p&gt;Name: {{ userName() }}&lt;/p&gt; &lt;!-- reads userName signal --&gt;

&lt;p&gt;Email: {{ userEmail() }}&lt;/p&gt; &lt;!-- reads userEmail signal --&gt;

`
```

```
})

export class ChildComponent {

readonly user = input&lt;User&gt;();

readonly userName = computed(() =&gt; this.user() .name);

readonly userEmail = computed(() =&gt; this.user() .email);

}


// What happens when updateEmail() is called:

// 1. currentUser signal changes new User reference

// 2. user input signal in child receives new value

// 3. userName computed: depends on user().name 'Alice' unchanged NOT recomputed

// 4. userEmail computed: depends on user().email changed recomputed

// 5. Template: only the &lt;p&gt;Email line updates not the name line

// Surgical precision impossible with traditional OnPush
```

Does signals make OnPush obsolete

```
SHORT ANSWER: Not yet but it changes the reason you'd use it


CURRENT REALITY (Angular 17):

Signal-based components automatically get fine-grained updates

OnPush still benefits non-signal parts of the tree

Migrating to signals + OnPush gives maximum performance

OnPush without signals still valuable for performance


FUTURE DIRECTION:

Angular team: "zoneless" Angular no zone.js needed

Signals provide all change notification zone.js currently provides

OnPush will become the default for signal-based components

Eventually: signal-based components may not need OnPush

because the reactivity IS the change detection


PRACTICAL ADVICE TODAY:

Use OnPush it's still the right choice
```

```
Add signals to components they work well WITH OnPush

The combination: signals tell Angular exactly what changed

OnPush prevents unnecessary tree traversal

Together: the most performant Angular app possible
```

Practical rule to take away:

> "Signal inputs make the component tree reactive without manual change detection hints. computed() derived values only update when their specific dependencies change -- not when the whole parent rerenders. This is more granular than OnPush, which checks the whole component. Use both together today: signals for fine-grained reactivity + OnPush to prevent unnecessary parent-triggered checks. The future of Angular is zoneless + signals -- OnPush won't disappear but its importance shifts from 'required for performance' to 'belt-and-suspenders extra optimisation'."

---

12. Related Topics to Learn Next

Angular-Specific Concepts

- OnPush Change Detection (Topic 24 -- signals + OnPush = Angular's performance story -- the natural next step)*
- Component Lifecycle Hooks (Topic 25 -- how signals interact with Angular's lifecycle -- when effects run relative to lifecycle hooks)*
- @Input() & @Output() in depth (Topic 26 -- input() and output() signal functions vs decorator equivalents)*
- Services & Dependency Injection (Topic 27 -- signal-based services -- using signals for shared state across components)*
- RxJS Operators in depth (Topic 46 -- toObservable(), toSignal() -- the bridges between Signals and RxJS)*

---

13. Updated Learning Tracker

```
Topics 1 19 ES6/JS + TypeScript Foundation COMPLETE

F1 F5 Angular Foundations Overview COMPLETE

Topic 20 RxJS Subscription Management COMPLETE

Topic 21 takeUntilDestroyed &amp;amp; DestroyRef COMPLETE

Topic 22 async pipe COMPLETE

Topic 23 Angular Signals COMPLETE

Topic 24 OnPush Change Detection NEXT


TIER 1 REMAINING:

24. OnPush Change Detection

25. Component Lifecycle Hooks
```

```
26. @Input() &amp;amp; @Output() in depth

27. Services &amp;amp; Dependency Injection


TIER 2 UPCOMING:

28 36. Template Syntax HTTP Interceptors


TIER 3 WHEN NEEDED:

37 46. Directives RxJS Operators
```

Key Concept Added to Quick Reference
Topic 23 -- Angular Signals

> signal() = reactive variable (raw cell). computed() = formula that auto-updates. effect() = side effect for outside world only. Read with (). Update with spread. No subscriptions, no cleanup.

* signal(v) = writable reactive container -- .set(), .update(), .asReadonly()
* computed(fn) = derived formula -- memoised, auto-updates, read-only
* effect(fn) = side effect runner -- DOM, localStorage, analytics ONLY -- constructor only
* Never use effect() to update signals -- use computed() instead
* Synchronous -- unlike Observables -- updates immediately
* signal.update(current => ({ ...current, prop: val })) -- immutable update pattern
* toSignal(obs$, { initialValue }) -- bridge Observable -> Signal
* Template: {{ signal() }} -- always call with ()
* Signals for state -- Observables for async operations -- both together
* Signal inputs (input()) + computed() = fine-grained template updates

Topic 24 -- OnPush Change Detection in Angular

0. Difficulty & Priority Rating

Difficulty: Intermediate -- the concept of "check less, check smarter" clicks quickly; understanding exactly when* OnPush triggers and how it interacts with Signals, async pipe, and immutability takes real practice

* Angular Relevance: Critical -- OnPush is the single most impactful performance optimisation in Angular; every production Angular app should use it; understanding it explains why immutability (Topic 7/14), Signals (Topic 23), and the async pipe (Topic 22) work the way they do -- they were all designed with OnPush in mind

1. Explanation

The Security Guard Analogy

Imagine a large office building. Every evening, a security guard does rounds -- checking every single room to see if anything has changed. Even rooms where nobody has been all day. Even the archive room that hasn't been opened in months. Every room, every evening, regardless.

That's Angular's default change detection -- check every component, every time anything happens.

Now imagine a smarter security system. Instead of checking every room, each room has a motion sensor. The guard only checks rooms where the sensor was triggered. A room with no activity gets no check. The guard's time is spent only where something actually happened.

OnPush is that smart security system.

Angular only checks a component when specific, known triggers fire -- a new @Input() reference, an event from the component itself, an async pipe emitting, a Signal changing. Everything else No check. No wasted work.

```
Default every room checked every evening wasteful but safe

OnPush only rooms where sensor triggered efficient and smart
```

The Lazy Librarian Analogy

Think of a librarian who updates the catalogue. In default mode, they re-read every book every day to check if anything changed. In OnPush mode, they only update the catalogue when:

* Someone hands them a new book (new @Input() reference)
* Someone uses the library (an event from this component)
* The delivery service arrives (async pipe emits)
* A digital alert fires (a Signal changes)

Without one of these four triggers, the librarian assumes everything is the same and does nothing.

In Plain English

* Change detection = Angular's process of checking if component templates need to be re-rendered
* Default = Angular checks every component in the tree on every change detection cycle -- safe but potentially slow
* OnPush = Angular only checks a component when it has a specific reason to -- fast and efficient

The four triggers = what tells Angular "check this OnPush component now"*

Immutability connection = OnPush checks if @Input() references* changed -- mutating an object's contents without creating a new reference means OnPush never notices the change

Now the Technical Terms

* Change detection cycle = the process Angular runs to compare current state with previous state and update the DOM

* ChangeDetectionStrategy.Default = checks the entire component tree on every event, timer, HTTP response, and Promise resolution

* ChangeDetectionStrategy.OnPush = marks a component's view as "dormant" -- Angular only re-checks it when specific triggers fire

* markForCheck() = manually tells Angular "check this OnPush component on the next cycle" -- used by async pipe and Signals internally

* detectChanges() = immediately runs change detection for this component and its children

* Zone.js = the library Angular traditionally used to intercept async operations and trigger change detection -- Signals aim to remove the need for it

---

2. Connection to Prior Concepts

OnPush is the reason every previous topic was taught the way it was:

Topic 7 (Pass by Reference) -- OnPush checks if @Input() references changed. Mutating an object (this.user.name = 'Bob') doesn't change the reference -- OnPush sees the same key and doesn't check. Creating a new object (this.user = { ...this.user, name: 'Bob' }) changes the reference -- OnPush notices. This is why* immutability matters in Angular

* Topic 14 (readonly & Immutability) -- readonly arrays and Readonly<T> prevent the accidental mutations that break OnPush. They're the TypeScript enforcement of the pattern OnPush requires

* Topic 22 (async pipe) -- the async pipe calls markForCheck() internally every time the Observable emits. This is how the template updates with new values despite being in an OnPush component

* Topic 23 (Signals) -- Signals notify Angular's change detection directly. A Signal change in an OnPush component automatically marks it for checking -- Signals are the modern alternative to markForCheck()

* Topic 20/21 (Subscriptions) -- manual subscriptions in OnPush components need markForCheck() or detectChanges() after updating state -- otherwise the template won't update. async pipe and Signals handle this automatically

Bridging note for your records:

"OnPush is the reason the previous six topics were structured around immutability, Signals, and the async pipe. They're not independent ideas -- they're pieces of the same performance puzzle. OnPush is the rule; immutability, Signals, and async pipe are how you follow the rule without fighting it."

---

3. Code Example

Example 1 -- Default vs OnPush -- seeing the difference
Understanding what changes when you add OnPush.

```
// ---- DEFAULT change detection checks on everything ----

@Component({

selector: 'app-default',
```

```
// No changeDetection specified defaults to ChangeDetectionStrategy.Default

template: `&lt;p&gt;{{ user.name }}&lt;/p&gt;`

})

export class DefaultComponent {

@Input() user: User;

// Checked on EVERY change detection cycle:

// - Any click anywhere in the app

// - Any setTimeout or setInterval firing

// - Any HTTP response completing

// - Any Promise resolving

// Even if 'user' hasn't changed at all

}


// ---- ONPUSH only checks when triggered ----

@Component({

selector: 'app-onpush',

changeDetection: ChangeDetectionStrategy.OnPush, // one line added

template: `&lt;p&gt;{{ user.name }}&lt;/p&gt;`

})

export class OnPushComponent {

@Input() user: User;

// Only checked when:

// 1. user @Input reference changes

// 2. An event fires from THIS component (click, input etc.)

// 3. An async pipe in this template emits

// 4. A Signal read in this template changes

// 5. markForCheck() or detectChanges() is called manually

}
```

Example 2 -- The four OnPush triggers
Each trigger explained with code -- what causes Angular to check an OnPush component.

```
@Component({

selector: 'app-triggers-demo',

changeDetection: ChangeDetectionStrategy.OnPush,

standalone: true,

imports: [CommonModule],

template: `

&lt;!-- Trigger 2: async pipe markForCheck() called internally --&gt;

&lt;p&gt;{{ notifications$ | async | json }}&lt;/p&gt;


&lt;!-- Trigger 4: Signal Angular notified directly --&gt;

&lt;p&gt;{{ count() }}&lt;/p&gt;


&lt;!-- Trigger 3: event from this component --&gt;

&lt;button (click)="onButtonClick()"&gt;Click me&lt;/button&gt;

`

})

export class TriggersDemoComponent {


@Input() user: User; // Trigger 1 new reference = check


// Trigger 2 async pipe calls markForCheck() internally

readonly notifications$ = this.notifService.getNotifications();


// Trigger 4 Signal changes notify Angular directly

readonly count = signal(0);


constructor(

private readonly notifService: NotificationService,

private readonly cdr: ChangeDetectorRef // for manual triggers

) {}


// Trigger 3 event from THIS component

onButtonClick(): void {
```

```
this.count.update(c =&gt; c + 1); // Signal change also Trigger 4

// Angular will check this component because the click event originated here

}


// ---- What does NOT trigger OnPush ----

brokenMutation(): void {

// Mutating @Input same reference OnPush doesn't notice

this.user.name = 'Bob'; // reference unchanged no check

// Template still shows old name user.name change is invisible to OnPush

}


manualTrigger(): void {

// Trigger 5 manual markForCheck

this.cdr.markForCheck(); // tells Angular: check this component next cycle

}


manualImmediate(): void {

// Trigger 6 immediate detectChanges

this.cdr.detectChanges(); // checks right now, synchronously

}

}
```

Example 3 -- The immutability requirement in action

The most important OnPush concept -- showing exactly when it works and when it breaks.

```
// ---- PARENT passes data to OnPush child ----

@Component({

selector: 'app-parent',

standalone: true,

imports: [UserCardComponent],

template: `&lt;app-user-card [user]="currentUser" /&gt;`

})

export class ParentComponent {
```

```
currentUser: User = { id: 1, name: 'Alice', role: 'user' };

// MUTATION OnPush child WON'T update

updateNameWrong(): void {

this.currentUser.name = 'Bob'; // mutates existing object

// currentUser reference is the same object same key

// OnPush child: "I've seen this key before skip check"

// Template still shows 'Alice'

}

// NEW REFERENCE OnPush child WILL update

updateNameCorrect(): void {

this.currentUser = { ...this.currentUser, name: 'Bob' }; // new object

// currentUser is now a different reference new key

// OnPush child: "new key! check this component"

// Template updates to show 'Bob'

}

}


// ---- ONPUSH CHILD ----

@Component({

selector: 'app-user-card',

changeDetection: ChangeDetectionStrategy.OnPush,

standalone: true,

template: `

&lt;div class="card"&gt;

&lt;h3&gt;{{ user.name }}&lt;/h3&gt;

&lt;span&gt;{{ user.role }}&lt;/span&gt;

&lt;/div&gt;

`

})
```

```
export class UserCardComponent {
@Input() user!: User;
// Only re-renders when user reference changes
// Mutation of user.name without new reference = silent, invisible update
}

// ---- ARRAYS same rule applies ----
@Component({
selector: 'app-list-parent',
standalone: true,
imports: [UserListComponent],
template: `&lt;app-user-list [users]="users" /&gt;`
})
export class ListParentComponent {

users: User[] = [{ id: 1, name: 'Alice' }];

// push mutates array same reference OnPush misses it
addUserWrong(user: User): void {
this.users.push(user); // same array reference
}

// spread new array new reference OnPush sees it
addUserCorrect(user: User): void {
this.users = [...this.users, user]; // new array
}

// splice mutates in place same reference
removeUserWrong(id: number): void {
const idx = this.users.findIndex(u =&gt; u.id === id);
this.users.splice(idx, 1); // mutates
}
```

```
// filter new array new reference

removeUserCorrect(id: number): void {

this.users = this.users.filter(u =&gt; u.id !== id); // new array

}

}
```

Angular Example -- A complete OnPush component done right

Every pattern working together -- Signals, async pipe, immutable updates, and OnPush.

```
// ---- SMART component OnPush + Signals + toSignal ----

@Component({

selector: 'app-order-dashboard',

changeDetection: ChangeDetectionStrategy.OnPush,

standalone: true,

imports: [CommonModule, CurrencyPipe, DatePipe],

template: `

&lt;!-- Signal reads directly, OnPush notified automatically --&gt;

&lt;header&gt;

&lt;h1&gt;Orders {{ pendingCount() }} pending&lt;/h1&gt;

&lt;span [class.alert]="hasUrgentOrders()"&gt;

{{ hasUrgentOrders() 'URGENT ORDERS' : 'All clear' }}

&lt;/span&gt;

&lt;/header&gt;


&lt;!-- async pipe markForCheck() called on emission --&gt;

@if (currentUser$ | async; as user) {

&lt;p&gt;Logged in as: {{ user.name }}&lt;/p&gt;

}


&lt;!-- Computed Signal only rerenders when filteredOrders changes --&gt;

@for (order of filteredOrders(); track order.id) {

&lt;div class="order-row"

[class.urgent]="order.isUrgent"
```

```
(click)="selectOrder(order)"&gt;

&lt;span&gt;{{ order.id }}&lt;/span&gt;

&lt;span&gt;{{ order.total | currency:'GBP' }}&lt;/span&gt;

&lt;span&gt;{{ order.createdAt | date:'shortDate' }}&lt;/span&gt;

&lt;span class="status"&gt;{{ order.status }}&lt;/span&gt;

&lt;/div&gt;

} @empty {

&lt;p&gt;No orders match the current filter&lt;/p&gt;

}


&lt;!-- Signal-based filter buttons --&gt;

&lt;div class="filters"&gt;

@for (status of statuses; track status) {

&lt;button

[class.active]="activeFilter() === status"

(click)="activeFilter.set(status)"&gt;

{{ status }}

&lt;/button&gt;

}

&lt;/div&gt;

`

})

export class OrderDashboardComponent {


private readonly orderService = inject(OrderService);

private readonly userService = inject(UserService);


// ---- toSignal HTTP Signal OnPush notified automatically ----

private readonly allOrders = toSignal(

this.orderService.getOrders(),

{ initialValue: [] as Order[] }
```

```
);

// ---- async pipe Observable markForCheck() internal ----

readonly currentUser$ = this.userService.getCurrentUser();

// ---- Writable signals user state ----

readonly activeFilter = signal&lt;OrderStatus | 'all'&gt;('all');

readonly selectedOrder = signal&lt;Order | null&gt;(null);

// ---- Computed derived, memoised, auto-updates ----

readonly filteredOrders = computed(() =&gt; {

const filter = this.activeFilter();

return this.allOrders().filter(o =&gt;

filter === 'all' || o.status === filter

);

});

readonly pendingCount = computed(() =&gt;

this.allOrders().filter(o =&gt; o.status === 'pending').length

);

readonly hasUrgentOrders = computed(() =&gt;

this.allOrders().some(o =&gt; o.isUrgent &amp;&amp; o.status === 'pending')

);

readonly statuses: Array&lt;OrderStatus | 'all'&gt; = [

'all', 'pending', 'processing', 'shipped', 'delivered'

];

// ---- Event from THIS component Trigger 3 ----

selectOrder(order: Order): void {

this.selectedOrder.set(order); // Signal change Trigger 4 also

}
```

```
}
```

4. Before & After Refactor

The Problem in Plain English:

A component without OnPush is like a light that flickers every time anyone in the building does anything -- regardless of whether anything changed in that room. Adding OnPush is like putting the light on a motion sensor -- it only activates when something in this room actually happened. The light still works exactly the same when someone is in the room -- it's just not wasting electricity when the room is empty.

```
// BEFORE no OnPush, mutation everywhere, performance drain

@Component({

selector: 'app-product-list',

standalone: true,

imports: [CommonModule, CurrencyPipe],

// No changeDetection uses Default

// Checked on EVERY mouse move, keypress, setTimeout, HTTP response

// in the ENTIRE application even unrelated ones

template: `

&lt;input (input)="onSearch($event)" placeholder="Search..."&gt;

&lt;div *ngFor="let product of displayProducts"&gt;

{{ product.name }} {{ product.price | currency:'GBP' }}

&lt;/div&gt;

&lt;p&gt;{{ displayProducts.length }} products&lt;/p&gt;

`

})

export class ProductListComponent implements OnInit, OnDestroy {

products: Product[] = [];

displayProducts: Product[] = [];

private sub: Subscription;

constructor(private readonly productService: ProductService) {}
```

```
ngOnInit(): void {

this.sub = this.productService.getProducts().subscribe(products =&gt; {

this.products = products;

this.displayProducts = products; // copy of reference

});

}


onSearch(event: Event): void {

const term = (event.target as HTMLInputElement).value.toLowerCase();

// Direct mutation of displayProducts

this.displayProducts = this.products.filter(

p =&gt; p.name.toLowerCase().includes(term)

);

// Works but only because Default always checks

// If this were OnPush would break because we're reassigning

// a NEW array each time (actually fine) but the mutation pattern

// comes alongside other mutations that DON'T create new references

}


ngOnDestroy(): void { this.sub.unsubscribe(); }

}



// AFTER OnPush + Signals + immutability production ready


@Component({

selector: 'app-product-list',

changeDetection: ChangeDetectionStrategy.OnPush, // smart motion sensor

standalone: true,

imports: [CommonModule, CurrencyPipe],

template: `

&lt;input
```

```
[value]="searchTerm()"

(input)="onSearch($event)"

placeholder="Search..."&amp;gt;


@for (product of displayProducts(); track product.id) {

{{ product.name }} {{ product.price | currency:'GBP' }}

}


&amp;lt;p&amp;gt;{{ displayProducts().length }} products&amp;lt;/p&amp;gt;

`

})

export class ProductListComponent {


private readonly productService = inject(ProductService);


// ---- Signals handle everything ----

private readonly allProducts = toSignal(

this.productService.getProducts(),

{ initialValue: [] as Product[] }

);


readonly searchTerm = signal('');


// computed automatically updates when allProducts or searchTerm changes

// OnPush notified via Signal dependency tracking

readonly displayProducts = computed(() =&amp;gt; {

const term = this.searchTerm().toLowerCase();

return !term

this.allProducts()

: this.allProducts().filter(

p =&amp;gt; p.name.toLowerCase().includes(term)

);

});
```

```
onSearch(event: Event): void {
this.searchTerm.set(
(event.target as HTMLInputElement).value
);
// Signal change computed recomputes OnPush notified
}

// No ngOnInit, no ngOnDestroy, no Subscription, no manual markForCheck
// Component only checked when search changes or products load
}
```

Why it matters: The before version re-renders on every browser event across the entire app -- a user typing in a completely unrelated search box elsewhere on the page triggers a check of this component. With 50 components on a page doing this simultaneously, you have 50 components checking on every keypress. With OnPush + Signals, each component is checked precisely when its own data changes. The difference is invisible to the user in small apps and dramatic in large ones.

5. Common Mistakes & Misconceptions

"OnPush means the component never updates -- it's broken"

When developers first add OnPush and find their component not updating, the instinct is to remove it and conclude it "doesn't work." The component is working exactly as designed -- it's not updating because the data isn't being changed in a way that creates new references. The fix is never to remove OnPush -- it's to fix the data update pattern.

Think of the motion sensor analogy -- if the light doesn't turn on, the problem isn't the motion sensor. It's that you walked into the room without moving (mutating data without new references). The sensor is working perfectly.

```
// Developer adds OnPush template stops updating removes OnPush
@Component({
// changeDetection: ChangeDetectionStrategy.OnPush, removed "because it broke"
})
export class ItemListComponent {

@Input() items: Item[] = [];

addItem(item: Item): void {
this.items.push(item); // THIS is why it "broke"
```

```
// Push mutates same reference OnPush correctly ignored it

// Fix: this.items = [...this.items, item];

// Don't remove OnPush fix the mutation

}

}


// The correct diagnosis:

// OnPush not updating ask "am I creating new references "

// If no fix the data mutation, keep OnPush

// If yes ask "is markForCheck / Signal / async pipe involved "

// If no add the missing notification mechanism

// NEVER remove OnPush because it's "causing problems"
```

"OnPush only matters for big apps -- don't bother for small ones"

OnPush is a habit and an architecture decision -- not a scaling feature you add later. A component that works correctly with OnPush works because its data flow is correct -- immutable updates, Signals, proper event handling. If you skip OnPush during development, you can accidentally build mutation-heavy patterns that become very difficult to refactor later.

Think of it like wearing a seatbelt -- you don't start wearing one when crashes become more likely. You wear it from the start because the habit is the point, and the protection is always worth having.

```
// OnPush as default habit even for simple components

@Component({

selector: 'app-badge',

changeDetection: ChangeDetectionStrategy.OnPush, // always, from the start

template: `&lt;span class="badge"&gt;{{ count }}&lt;/span&gt;`

})

export class BadgeComponent {

@Input() count = 0;

// Even this tiny component benefits:

// Without OnPush: checked 50 times per second during user interaction

// With OnPush: checked only when count @Input reference changes

// At scale dozens of BadgeComponents the difference is real

}
```

"Manual subscribe in an OnPush component with this.data = result works fine"

Manual subscribe that updates a class property in an OnPush component will often appear to work during development (because events from the same component trigger checks) but will silently fail in specific scenarios. Without markForCheck(), Signals, or async pipe, a subscription callback updating a class property has no way to tell OnPush to check the component.

```
// Manual subscribe in OnPush template might not update

@Component({

changeDetection: ChangeDetectionStrategy.OnPush,

template: `&lt;div *ngFor="let user of users"&gt;{{ user.name }}&lt;/div&gt;`

})

export class UserListComponent implements OnInit {


users: User[] = [];

private readonly destroyRef = inject(DestroyRef);


ngOnInit(): void {

// This subscription updates users but doesn't tell OnPush to check

this.userService.getUsers().pipe(

takeUntilDestroyed(this.destroyRef)

).subscribe(users =&gt; {

this.users = users; // OnPush: no notification may not re-render

});

}

}


// Fix 1 markForCheck after update

constructor(private readonly cdr: ChangeDetectorRef) {}


ngOnInit(): void {

this.userService.getUsers().pipe(

takeUntilDestroyed(this.destroyRef)

).subscribe(users =&gt; {

this.users = users;
```

```
this.cdr.markForCheck(); // tell OnPush to check next cycle

});

}


// Fix 2 use async pipe (markForCheck built in) preferred

readonly users$ = this.userService.getUsers();

// Template: *ngFor="let user of users$ | async"


// Fix 3 use toSignal (Signal notifies OnPush automatically) preferred

readonly users = toSignal(this.userService.getUsers(), { initialValue: [] });

// Template: *ngFor="let user of users()"
```

6. Do's and Don'ts

| [Y] Do | [N] Don't |

|---|---|

| Add ChangeDetectionStrategy.OnPush to every component -- make it the default habit | Leave components on Default and plan to add OnPush later -- mutation patterns solidify |

| Use immutable update patterns -- spread, map, filter -- never push, splice in-place | Mutate @Input() objects or arrays -- OnPush will silently ignore the change |

| Use Signals for component state -- Angular notifies OnPush automatically | Use manual subscribe + class property in OnPush without markForCheck() |

| Use async pipe for Observable data -- markForCheck() built in | Use manual subscribe in OnPush templates expecting auto-update |

| Call markForCheck() when you genuinely need manual subscription + OnPush | Call detectChanges() habitually -- it's synchronous, heavier, use sparingly |

| Ensure parent components are also OnPush -- a non-OnPush parent causes all children to check anyway | Add OnPush only to leaf components -- the whole tree must use it for maximum benefit |

7. Debugging Tips

Bug: Template not updating after data changes in OnPush component

```
Component shows stale data update method called but template unchanged
```

* Plain English cause: Data mutated without creating new reference -- OnPush sees the same key and skips checking -- the motion sensor didn't trigger

* Diagnosis: Log the data before and after -- check before === after -- if true, you mutated

* Fix: Use spread to create new references -- { ...obj } for objects, [...arr] for arrays

Bug: async pipe data showing but Signal data not updating in OnPush

```
async pipe works fine but signal-based values are stale
```

* Plain English cause: Signal read in the template wasn't actually declared as a Signal -- it's a plain class property. Only real Signals notify OnPush

* Fix: Ensure the property is declared with signal() -- plain properties don't notify anything

Bug: OnPush component updates inconsistently -- sometimes yes, sometimes no

```
Template updates after a click but not after an HTTP call completes
```

* Plain English cause: Click events trigger change detection for the whole subtree (Trigger 3). HTTP callbacks don't -- they need a Signal, async pipe, or markForCheck() to notify OnPush

* Fix: Use async pipe or toSignal() for HTTP data -- or add markForCheck() in the subscribe callback

Bug: Adding markForCheck() works but feels wrong

```
Adding cdr.markForCheck() in subscribe fixes it but something feels off
```

* Plain English cause: markForCheck() is the correct manual approach but it signals a missed opportunity. It means you have manual subscribe in an OnPush component -- which is harder to maintain than async pipe or Signals

* Guidance: Use markForCheck() as a temporary fix. Refactor to async pipe or toSignal() for a cleaner solution

8. Real-World Applications

Application 1 -- Large Data Table with Frequent Updates
Plain English first:

A data table showing 1,000 rows of order data that updates every 30 seconds via polling. Without OnPush, every 30-second update re-evaluates every template expression in every row -- 1,000 rows, potentially dozens of bindings each. With OnPush, only the rows whose data actually changed get re-evaluated. Combined with trackBy, Angular reuses the existing DOM nodes -- only updating the specific cells that changed. The difference between "the table freezes for a second" and "instant smooth update."

```
@Component({

selector: 'app-orders-table',

changeDetection: ChangeDetectionStrategy.OnPush, // essential for large lists

standalone: true,

imports: [CommonModule, CurrencyPipe, DatePipe],

template: `

&lt;!-- trackBy tells Angular which rows to reuse vs recreate --&gt;

@for (order of orders(); track order.id) {

&lt;app-order-row

[order]="order"

(statusChanged)="onStatusChange($event)"&gt;
```

```
&lt;/app-order-row&gt;
}
&lt;p&gt;Total: {{ totalValue() | currency:'GBP' }}&lt;/p&gt;
`
})
export class OrdersTableComponent {

private readonly orderService = inject(OrderService);

// Polling toSignal handles subscription + OnPush notification
private readonly ordersData = toSignal(
timer(0, 30_000).pipe(
switchMap(() =&gt; this.orderService.getOrders()),
catchError(() =&gt; of([] as Order[]))
),
{ initialValue: [] as Order[] }
);

readonly orders = computed(() =&gt; this.ordersData());
readonly totalValue = computed(() =&gt;
this.orders().reduce((sum, o) =&gt; sum + o.total, 0)
);

onStatusChange(event: { id: number; status: OrderStatus }): void {
// Immutable update new array reference OnPush notified
this.ordersData.update .(orders =&gt;
orders.map(o =&gt;
o.id === event.id { ...o, status: event.status } : o
)
);
}
}
```

```
// ---- Row component also OnPush only checks when ITS order changes ----

@Component({

selector: 'app-order-row',

changeDetection: ChangeDetectionStrategy.OnPush,

standalone: true,

imports: [CurrencyPipe, DatePipe],

template: `

&lt;td&gt;{{ order.id }}&lt;/td&gt;

&lt;td&gt;{{ order.total | currency:'GBP' }}&lt;/td&gt;

&lt;td&gt;{{ order.createdAt | date:'shortDate' }}&lt;/td&gt;

&lt;td&gt;{{ order.status }}&lt;/td&gt;

&lt;td&gt;

&lt;button (click)="onComplete()"&gt;Complete&lt;/button&gt;

&lt;/td&gt;

`

})

export class OrderRowComponent {

@Input({ required: true }) order!: Order;

@Output() statusChanged = new EventEmitter&lt;{ id: number; status: OrderStatus
}&gt;();


onComplete(): void {

this.statusChanged.emit({ id: this.order.id, status: 'completed' });

}

}

// 1,000 rows only rows whose 'order' reference changed get re-rendered
```

Application 2 -- Notification Badge Tree

Plain English first:

A navigation bar with nested components -- the app shell, the nav bar, the notification section, the badge count. Without OnPush, a notification arriving causes every component in the tree to re-render -- the entire app shell, even though only the badge number changed. With OnPush on every component, Angular's change detection is surgical -- only the badge component is checked and

updated, the rest of the tree is untouched.

```
// ---- App Shell OnPush, top of tree ----

@Component({

selector: 'app-shell',

changeDetection: ChangeDetectionStrategy.OnPush,

standalone: true,

imports: [NavBarComponent, RouterOutlet],

template: `

&lt;app-nav-bar [userId]="currentUserId()" /&gt;

&lt;router-outlet /&gt;

`

})

export class AppShellComponent {

readonly currentUserId = toSignal(

this.authService.userId$,

{ initialValue: null as number | null }

);

constructor(private readonly authService: AuthService) {}

}


// ---- Nav Bar OnPush, middle of tree ----

@Component({

selector: 'app-nav-bar',

changeDetection: ChangeDetectionStrategy.OnPush,

standalone: true,

imports: [NotificationBadgeComponent],

template: `

&lt;nav&gt;

&lt;app-logo /&gt;

&lt;app-notification-badge [userId]="userId" /&gt;

&lt;/nav&gt;
```

```
  `
})
export class NavBarComponent {
@Input() userId: number | null = null;
}

// ---- Badge OnPush, leaf of tree ----
@Component({
selector: 'app-notification-badge',
changeDetection: ChangeDetectionStrategy.OnPush,
standalone: true,
imports: [CommonModule],
template: `
@if (count$ | async; as count) {
@if (count &gt; 0) {
&lt;span class="badge"&gt;{{ count &gt; 99 '99+' : count }}&lt;/span&gt;
}
}
`
})
export class NotificationBadgeComponent {
@Input() userId: number | null = null;

// async pipe handles markForCheck()
readonly count$ = this.notifService.getUnreadCount().pipe(
catchError(() =&gt; of(0))
);

constructor(private readonly notifService: NotificationService) {}
}

// When a new notification arrives:
```

```
// AppShell: currentUserId unchanged NOT checked

// NavBar: userId @Input unchanged NOT checked

// Badge: count$ async pipe emits markForCheck() CHECKED

// Only 1 component re-rendered out of the whole tree
```

9. Practice Exercises

Exercise 1 -- Beginner

Take a component with ChangeDetectionStrategy.Default that has: a users: User[] property loaded in ngOnInit via subscribe, an addUser() method using push, and a removeUser() method using splice. Add ChangeDetectionStrategy.OnPush to it. Observe what breaks. Fix every broken part using: toSignal() for the HTTP call, signal() for the array, and update() with spread/filter for mutations. Document each fix -- what was broken, why, and how the fix solves it.

Exercise 2 -- Intermediate

Build a ProductGridComponent with OnPush that: receives a category @Input() signal-style (use input() from Angular 17+ or @Input() with ngOnChanges), loads products for that category via HTTP, allows inline editing of product names (clicking a name makes it editable), filters products by a local search signal, and shows a count of filtered results. Every state update must use immutable patterns. Verify OnPush works correctly by adding console.log in ngDoCheck() -- it should log minimally -- only when actual triggers fire.

Exercise 3 -- Advanced

Build a LiveDataDashboard component tree with four levels -- DashboardComponent -> MetricsPanelComponent -> MetricCardComponent -> MetricValueComponent. Apply OnPush to all four levels. The dashboard polls an API every 10 seconds for metrics data. Use Signals throughout -- toSignal for the HTTP data, computed for derived values at each level. Add ngDoCheck() with a counter to every component. Run the app for one minute and verify: MetricValueComponent logs change detection only when its specific metric value changes, not on every poll cycle for unchanged metrics. Prove that adding trackBy to @for loops reduces DOM recreation.

10. Key Takeaways

Core Concepts in Plain English

* Change detection = Angular checking if templates need to update -- runs on every async event by default
* Default = checks every component on every event anywhere in the app -- safe but potentially slow
* OnPush = checks a component only when specific triggers fire -- efficient and smart

The core rule = OnPush checks @Input() references* -- mutation without new reference = invisible to OnPush

* Immutability is required = spread for objects, filter/map for arrays -- always new references
* Signals automatically notify = no markForCheck() needed -- Signals are OnPush-native
* async pipe automatically notifies = markForCheck() built in -- use it without worry

The Four OnPush Triggers

```
1. @Input() reference changes parent passes new object/array reference

2. Event from this component (click), (input), (submit) etc.

3. async pipe emits calls markForCheck() internally

4. Signal changes Angular's reactive graph notifies directly


+ Manual:

5. markForCheck() schedule check for next cycle

6. detectChanges() check immediately, synchronously
```

Quick Reference -- What Works With OnPush

Pattern	Works with OnPush	Why
@Input() new reference	[Y]	Trigger 1 -- reference change
@Input() mutation	[N]	Same reference -- no trigger
Signal in template	[Y]	Trigger 4 -- Signal notifies
| async in template	[Y]	Trigger 3 -- markForCheck built in
Manual subscribe + class prop	[N] without markForCheck	No notification mechanism
Manual subscribe + markForCheck	[Y]	Manual trigger 5
array.push() on @Input	[N]	Same reference -- mutated
[...array, item] on @Input	[Y]	New reference -- Trigger 1

Angular-Specific Rules

* Add ChangeDetectionStrategy.OnPush to every component -- make it a non-negotiable default
* Use Signals for component state -- no markForCheck() needed, works perfectly with OnPush
* Use async pipe for Observable data -- markForCheck() built in
* Never mutate @Input() objects or arrays -- always create new references
* Add OnPush to the whole component tree -- a Default parent forces all children to check anyway
* Use trackBy / track in @for loops with OnPush -- prevents unnecessary DOM recreation
* markForCheck() over detectChanges() -- schedules next cycle vs runs synchronously now

One-Line Memory Aid

> OnPush is the motion sensor -- only lights up when specific triggers fire. New @Input() reference, component event, async pipe emission, or Signal change. Mutation without new reference = walking invisibly past the sensor. Signals and async pipe are motion-sensor-native -- they trigger it automatically.

---

11. Thought-Provoking Question + Answer

> Angular's team has announced "zoneless Angular" -- running Angular without zone.js entirely. Zone.js is what currently triggers change detection after every async event. If zone.js is removed, what replaces it, and how does understanding OnPush help you write code that works in both zoned and zoneless Angular today

Plain English explanation first:

Think of zone.js as a building-wide PA system. Whenever anything happens anywhere in the building -- someone clicks a mouse, a timer fires, an HTTP response arrives -- zone.js announces it over the PA: "Attention all rooms -- something happened -- please check if you need to update." Every room (component) in Default mode dutifully checks. OnPush rooms only check if they're on the announcement list.

The problem: the PA system is loud, constant, and makes everyone check even when the announcement has nothing to do with them. It works -- but it's inefficient, and it makes Angular harder to integrate with other frameworks that don't expect constant interruptions.

Zoneless Angular removes the PA system entirely. Instead, each room is responsible for raising its own hand when it needs attention. Signals are the hand-raising mechanism.

```
// WITH zone.js (current default):

// setInterval fires zone.js intercepts announces to all rooms

// All Default components check most find nothing changed wasted work


setInterval(() =&gt; {

this.count++; // zone.js catches this triggers change detection

}, 1000); // works, but noisy



// WITHOUT zone.js (zoneless):

// setInterval fires nobody notices unless you raise your hand


setInterval(() =&gt; {

this.count++; // plain property nothing notified template never updates

}, 1000);


// With Signals raises hand automatically, no zone.js needed

setInterval(() =&gt; {

this.count.update(c =&gt; c + 1); // Signal change Angular notified directly

}, 1000); // works in both zoned AND zoneless Angular
```

The three patterns that work in both zoned and zoneless:

```
// PATTERN 1 Signals (best zoneless native)

@Component({

changeDetection: ChangeDetectionStrategy.OnPush,

template: `{{ count() }}`
```

```
})
export class ZonelessReadyComponent {
readonly count = signal(0);

increment(): void {
this.count.update(c =&gt; c + 1); // works with AND without zone.js
}
}


// PATTERN 2 async pipe (works in both)
@Component({
changeDetection: ChangeDetectionStrategy.OnPush,
template: `{{ data$ | async }}`
})
export class ZonelessReadyComponent {
readonly data$ = this.service.getData(); // async pipe handles markForCheck
// async pipe uses markForCheck() works with zone.js
// In zoneless: async pipe also works it calls markForCheck() directly
}


// PATTERN 3 toSignal (best for HTTP data)
@Component({
changeDetection: ChangeDetectionStrategy.OnPush,
template: `{{ data() }}`
})
export class ZonelessReadyComponent {
readonly data = toSignal(this.service.getData(), { initialValue: null });
// toSignal Signal zoneless native
}
```

```
// PATTERN TO AVOID plain property + Default

@Component({

// No OnPush Default

template: `{{ count }}`

})

export class ZonedOnlyComponent {

count = 0;


increment(): void {

this.count++; // relies on zone.js to notice this

// Works today will silently break in zoneless Angular

}

}
```

What zoneless Angular looks like in practice (Angular 18+):

```
// main.ts zoneless app

bootstrapApplication(AppComponent, {

providers: [

provideExperimentalZonelessChangeDetection(), // Angular 18+

provideRouter(routes),

provideHttpClient()

]

});


// With this zone.js is not loaded at all

// Smaller bundle no zone.js (~36KB saved)

// Faster startup no monkey-patching of browser APIs

// Works if ALL your components use:

// OnPush + Signals

// OnPush + async pipe

// OnPush + markForCheck() for manual updates

// Breaks if any component uses:
```

```
// Default change detection with plain properties

// setTimeout/setInterval updating plain properties

// HTTP subscribe updating plain properties without markForCheck
```

Practical rule to take away:

> "Write OnPush + Signals + async pipe today and your code is already zoneless-ready. The migration from zoned to zoneless Angular is: 1. Add OnPush to every component. 2. Replace plain properties with Signals. 3. Replace manual subscribe with toSignal or async pipe. If you've followed Topics 20-24, you've already done steps 2 and 3. Step 1 is what this topic teaches. Code that follows these patterns works identically in both zoned and zoneless Angular -- you're writing future-proof Angular today."

---

12. Related Topics to Learn Next

Angular-Specific Concepts

- Component Lifecycle Hooks (Topic 25 -- ngDoCheck is where you can observe change detection cycles -- important for debugging OnPush)*
- @Input() & @Output() in depth (Topic 26 -- input() signal function + OnPush = the perfect combination -- required inputs, signal inputs)*
- Services & Dependency Injection (Topic 27 -- service-level signals shared across OnPush components -- the state management pattern)*
- Angular Template Syntax (Topic 28 -- @for with track replaces ngFor with trackBy -- essential for OnPush list performance)
- Angular Reactive Forms (Topic 33 -- forms + OnPush requires careful handling -- markForCheck() interactions)*

---

13. Updated Learning Tracker

```
Topics 1 19 ES6/JS + TypeScript Foundation COMPLETE

F1 F5 Angular Foundations Overview COMPLETE

Topic 20 RxJS Subscription Management COMPLETE

Topic 21 takeUntilDestroyed &amp;amp; DestroyRef COMPLETE

Topic 22 async pipe COMPLETE

Topic 23 Angular Signals COMPLETE

Topic 24 OnPush Change Detection COMPLETE

Topic 25 Component Lifecycle Hooks NEXT


TIER 1 REMAINING:

25. Component Lifecycle Hooks

26. @Input() &amp;amp; @Output() in depth
```

```
27. Services &amp;amp; Dependency Injection


TIER 2 UPCOMING:

28 36. Template Syntax HTTP Interceptors


TIER 3 WHEN NEEDED:

37 46. Directives RxJS Operators
```

Key Concept Added to Quick Reference
Topic 24 -- OnPush Change Detection

> OnPush is the motion sensor -- only lights up when specific triggers fire. New @Input() reference, component event, async pipe emission, or Signal change. Mutation without new reference = walking past the sensor invisibly.

* Default = every room checked every event -- safe but slow
* OnPush = only checked when specific triggers fire -- efficient
* Four triggers: new @Input() reference, component event, async pipe emits, Signal changes
* Mutation without new reference = invisible to OnPush -- always spread
* Signals = OnPush-native -- no markForCheck() needed
* async pipe = markForCheck() built in -- always works with OnPush
* Manual subscribe + class property in OnPush = needs markForCheck() or switch to Signal/async pipe
* Apply to EVERY component -- Default parent forces all children to check anyway
* Zoneless Angular = Signals replace zone.js -- OnPush + Signals = future-proof code today

Topic 25 -- Component Lifecycle Hooks in Angular

0. Difficulty & Priority Rating

Difficulty: Beginner-Intermediate -- the individual hooks are simple; knowing which hook to use for which* task, and how they interact with Signals, OnPush, and @Input() takes real practice

* Angular Relevance: Critical -- every Angular component goes through a predictable lifecycle; knowing when each hook fires is what separates components that work correctly from ones that have subtle timing bugs -- loading data too early, accessing the DOM before it exists, or leaking resources

1. Explanation

The Employee Onboarding Analogy

Imagine a new employee joining a company. Their working life follows a predictable sequence:

* Hired -> constructor -- the paperwork is signed, the employee exists, but hasn't started work yet. Don't ask them to do anything complex yet -- their desk isn't set up

* Briefed on their role -> ngOnChanges -- they receive their job description and responsibilities (the @Input() values). Happens before their first day and every time their role changes

* First day starts -> ngOnInit -- they're at their desk, settled in, ready to do real work. Load data, set up subscriptions, start processes

* Doing their job -> change detection cycles -- Angular checks their work regularly

* Given access to the office layout -> ngAfterViewInit -- the building is fully constructed, they can now see and use all the rooms (child components, DOM elements)

* Leaving the company -> ngOnDestroy -- final cleanup, hand over work, close accounts

```
constructor hired exists but not ready

ngOnChanges briefed inputs received

ngOnInit first day start real work

(change cycles) doing the job

ngAfterViewInit office access DOM/children ready

ngOnDestroy leaving clean up everything
```

The Theatre Show Analogy

Think of a component like a theatre performance:

* constructor = the theatre is built -- the stage exists but curtain is closed

* ngOnChanges = the script arrives -- actors know their lines (inputs)

* ngOnInit = rehearsals begin -- real preparation work happens

* ngAfterViewInit = the set is constructed -- physical stage is ready

* ngOnDestroy = the show closes -- tear down the set, release the venue

In Plain English

* Lifecycle hooks = methods Angular calls at specific moments in a component's life -- from creation to destruction

* constructor = called when Angular creates the class instance -- before inputs arrive -- for dependency injection only

* ngOnChanges = called when @Input() values change -- before ngOnInit on first load -- and on every subsequent change

* ngOnInit = called once after the first ngOnChanges -- the right place for initialisation work

* ngAfterViewInit = called after the component's template and all child components are fully rendered -- for DOM access

* ngOnDestroy = called just before Angular destroys the component -- the right place for all cleanup

* Order matters = the hooks fire in a guaranteed order -- knowing that order prevents timing bugs

Now the Technical Terms

* OnChanges = interface requiring ngOnChanges(changes: SimpleChanges) -- fires when bound @Input() properties change

* OnInit = interface requiring ngOnInit() -- fires once after first ngOnChanges

* DoCheck = interface requiring ngDoCheck() -- fires on every change detection cycle -- use sparingly

* AfterContentInit = fires after ng-content projected content is initialised

* AfterContentChecked = fires after projected content is checked

* AfterViewInit = interface requiring ngAfterViewInit() -- fires after the component's view and child views are initialised

* AfterViewChecked = fires after the view is checked -- use sparingly

* OnDestroy = interface requiring ngOnDestroy() -- fires just before the component is destroyed

* SimpleChanges = an object mapping each changed @Input() property name to a SimpleChange object containing previousValue, currentValue, and firstChange

---

2. Connection to Prior Concepts

Lifecycle hooks are where every previous Angular topic connects to timing:

Topic 20/21 (Subscriptions & takeUntilDestroyed) -- subscriptions are set up in ngOnInit and cleaned up in ngOnDestroy. takeUntilDestroyed replaces manual ngOnDestroy cleanup. Now you understand why* -- ngOnDestroy is the destruction notification that DestroyRef abstracts

* Topic 22 (async pipe) -- the async pipe subscribes when the template is first evaluated (after ngOnInit runs) and unsubscribes via its own internal ngOnDestroy. The lifecycle hooks make this timing clear

* Topic 23 (Signals) -- effect() must be in the constructor because it needs injection context. Signal updates in ngOnInit work fine. ngAfterViewInit is where you'd access DOM elements to read into a Signal

* Topic 24 (OnPush) -- ngDoCheck is Angular's hook that fires on every change detection cycle -- useful for debugging OnPush. ngAfterViewChecked fires after every view check -- relevant for understanding when OnPush actually runs

* Topic 17 (Decorators) -- @Component sets up the lifecycle system. implements OnInit, OnDestroy are TypeScript interfaces that ensure you spell the hook methods correctly -- without them, a typo like ngOnint() silently never fires

Bridging note for your records:

"Lifecycle hooks are Angular's event system for a component's own existence. ngOnInit is when you start work, ngOnDestroy is when you stop. Everything between -- subscriptions, HTTP calls, DOM

access, Signal effects -- has a correct hook to live in. Using the wrong hook causes timing bugs that are subtle and hard to diagnose."

3. Code Example

Example 1 -- The complete lifecycle -- all hooks in order
See every hook fire in sequence -- understanding the order prevents every timing bug.

```
import {

Component, OnChanges, OnInit, DoCheck,

AfterContentInit, AfterContentChecked,

AfterViewInit, AfterViewChecked, OnDestroy,

Input, SimpleChanges

} from '@angular/core';


@Component({

selector: 'app-lifecycle-demo',

standalone: true,

template: `&lt;p&gt;{{ title }}&lt;/p&gt;`

})

export class LifecycleDemoComponent implements

OnChanges, OnInit, DoCheck,

AfterContentInit, AfterContentChecked,

AfterViewInit, AfterViewChecked, OnDestroy {


@Input() title = '';

@Input() userId: number;


// CONSTRUCTOR runs first DI only no inputs yet

constructor() {

console.log(' constructor class created, DI ready');

// this.title = '' @Input not received yet

// DON'T: make HTTP calls, access DOM, use @Input values

// DO: inject services via constructor parameters

}
```

```
// NGONCHANGES runs before ngOnInit AND on every @Input change

ngOnChanges(changes: SimpleChanges): void {

console.log(' ngOnChanges @Input values changed:', changes);


// changes object: { title: SimpleChange, userId: SimpleChange }

if (changes['title']) {

const { previousValue, currentValue, firstChange } = changes['title'];

console.log(`title: ${previousValue} ${currentValue}`);

console.log('Is this the first change ', firstChange);

}


// React to specific @Input changes

if (changes['userId'] &amp;&amp; !changes['userId'].firstChange) {

// userId changed AFTER the first load reload data

this.loadUserData(changes['userId'].currentValue);

}

}


// NGONINIT runs once the right place for real initialisation

ngOnInit(): void {

console.log(' ngOnInit component ready, @Inputs available');

// @Input values ARE available here

// DO: HTTP calls, set up subscriptions, signal effects

// DON'T: access DOM, access @ViewChild not rendered yet

}


// NGDOCHECK runs on EVERY change detection cycle use sparingly

ngDoCheck(): void {

// console.log(' ngDoCheck'); // commented fires very frequently

// Use for custom change detection of objects/arrays Angular can't track

// Very rarely needed with Signals

}
```

```
// NGAFTERCONTENTINIT runs once after ng-content is projected

ngAfterContentInit(): void {

console.log(' ngAfterContentInit projected content ready');

// Only relevant if this component uses ng-content

}


// NGAFTERCONTENTCHECKED runs after projected content is checked

ngAfterContentChecked(): void {

// Very rarely needed fires frequently

}


// NGAFTERVIEWINIT runs once after template + children rendered

ngAfterViewInit(): void {

console.log(' ngAfterViewInit DOM and child components ready');

// @ViewChild IS available here

// DO: access DOM elements, initialise third-party libraries

// DON'T: update component state here without using setTimeout/Signal

// causes ExpressionChangedAfterItHasBeenChecked error

}


// NGAFTERVIEWCHECKED runs after every view check

ngAfterViewChecked(): void {

// Very rarely needed fires frequently

}


// NGONDESTROY runs just before component is removed

ngOnDestroy(): void {

console.log(' ngOnDestroy component being destroyed clean up');

// DO: unsubscribe, clear timers, close connections

// With takeUntilDestroyed/DestroyRef this is often empty

}
```

```
  private loadUserData(id: number): void {

  console.log('Loading data for user:', id);

  }

  }


  // ---- LIFECYCLE ORDER (first load): ----

  // constructor

  // ngOnChanges (firstChange = true)

  // ngOnInit

  // ngDoCheck

  // ngAfterContentInit

  // ngAfterContentChecked

  // ngAfterViewInit

  // ngAfterViewChecked


  // ---- ON EVERY CHANGE DETECTION CYCLE (after first load): ----

  // ngOnChanges (only if @Input changed)

  // ngDoCheck

  // ngAfterContentChecked

  // ngAfterViewChecked


  // ---- ON DESTRUCTION: ----

  // ngOnDestroy
```

Example 2 -- ngOnChanges -- reacting to @Input() changes
The most misunderstood hook -- fires before ngOnInit AND on every subsequent input change.

```
  @Component({

  selector: 'app-user-detail',

  changeDetection: ChangeDetectionStrategy.OnPush,

  standalone: true,

  imports: [CommonModule],

  template: `

  @if (user) {
```

```
&lt;h2&gt;{{ user.name }}&lt;/h2&gt;

&lt;p&gt;{{ user.email }}&lt;/p&gt;

} @else {

&lt;p&gt;Loading...&lt;/p&gt;

}

`

})

export class UserDetailComponent implements OnChanges, OnDestroy {


@Input({ required: true }) userId!: number;


user: User | null = null;

isLoading = false;


private readonly destroyRef = inject(DestroyRef);


constructor(private readonly userService: UserService) {}


// ---- ngOnChanges fires BEFORE ngOnInit on first load ----

// AND whenever userId @Input changes (e.g. navigating to different user)

ngOnChanges(changes: SimpleChanges): void {

if (changes['userId']) {

// Safe to use both first load AND subsequent changes

this.loadUser(this.userId);

}

}


// ---- WHY ngOnChanges instead of ngOnInit here ----

// ngOnInit: userId is available BUT only fires once

// If userId @Input changes later (different route param) ngOnInit won't fire again

// ngOnChanges: fires on first load AND every subsequent change


private loadUser(id: number): void {
```

```
this.isLoading = true;

this.user = null;


this.userService.getUser(id).pipe(

takeUntilDestroyed(this.destroyRef)

).subscribe({

next: user =&gt; { this.user = user; this.isLoading = false; },

error: () =&gt; { this.isLoading = false; }

});

}

}


// ---- SimpleChanges in detail ----

ngOnChanges(changes: SimpleChanges): void {


// Check if a specific input changed

if (changes['userId']) {

const change = changes['userId'];


change.previousValue; // what it was before

change.currentValue; // what it is now

change.firstChange; // true only on the very first change


// Common pattern: skip first change if ngOnInit handles it

if (!change.firstChange) {

console.log('userId changed from', change.previousValue, 'to', change.currentValue);

}

}


// Check if ANY input changed

Object.keys(changes).forEach(key =&gt; {

console.log(`${key} changed:`, changes[key]);
```

```
});

}
```

Example 3 -- ngAfterViewInit -- DOM and child component access
The hook where @ViewChild becomes available -- never before.

```
import { Component, AfterViewInit, ViewChild, ElementRef } from '@angular/core';

@Component({

selector: 'app-chart',

standalone: true,

template: `

&lt;canvas #chartCanvas width="400" height="200"&gt;&lt;/canvas&gt;

&lt;app-chart-legend #legend [data]="chartData" /&gt;

`

})

export class ChartComponent implements AfterViewInit, OnDestroy {

@ViewChild('chartCanvas') canvasRef!: ElementRef&lt;HTMLCanvasElement&gt;;

@ViewChild('legend') legendRef!: ChartLegendComponent;

chartData = [/* ... */];

private chart: ChartJS | null = null;

ngOnInit(): void {

// Can NOT access ViewChild here template not rendered yet

// this.canvasRef.nativeElement.getContext('2d') undefined

}

ngAfterViewInit(): void {

// Template is fully rendered @ViewChild available here

const ctx = this.canvasRef.nativeElement.getContext('2d');

// Initialise third-party chart library

this.chart = new ChartJS(ctx, {
```

```
type: 'bar',

data: this.chartData

});

// Access child component instance

console.log('Legend component:', this.legendRef);

this.legendRef.highlight(0); // call child component method

}

ngOnDestroy(): void {

// Destroy third-party library DestroyRef.onDestroy() is better

// but ngOnDestroy also works for this

this.chart .destroy();

}

}

// ---- IMPORTANT: Don't update state in ngAfterViewInit ----

@Component({ template: `&lt;p&gt;{{ title }}&lt;/p&gt;` })

export class ProblemComponent implements AfterViewInit {

title = 'Initial';

ngAfterViewInit(): void {

// Causes ExpressionChangedAfterItHasBeenCheckedError in dev mode

this.title = 'Updated in AfterViewInit';

// Angular ran change detection, committed the view, now you changed state

// Angular detects the inconsistency in dev mode and throws

// Fix wrap in Promise.resolve() or use setTimeout

Promise.resolve().then(() =&gt; {

this.title = 'Updated safely'; // runs after current change detection cycle

});
```

```
// Better fix use a Signal

// this.title = signal('Initial');

// ngAfterViewInit: this.title.set('Updated'); // Signal handles timing

}

}
```

Angular Example -- Lifecycle hooks in a production component
A real Angular component showing the right hook for every task.

```
@Component({

selector: 'app-product-editor',

changeDetection: ChangeDetectionStrategy.OnPush,

standalone: true,

imports: [CommonModule, ReactiveFormsModule],

template: `

@if (isLoading()) {

&lt;div class="spinner"&gt;Loading...&lt;/div&gt;

}


@if (product()) {

&lt;form [formGroup]="form" (ngSubmit)="onSubmit()"&gt;

&lt;input formControlName="name" placeholder="Product name"&gt;

&lt;input formControlName="price" type="number" placeholder="Price"&gt;

&lt;textarea formControlName="description"&gt;&lt;/textarea&gt;

&lt;button type="submit" [disabled]="form.invalid || isSaving()"&gt;

{{ isSaving() 'Saving...' : 'Save' }}

&lt;/button&gt;

&lt;/form&gt;

}


&lt;!-- Image preview needs DOM access --&gt;

&lt;canvas #previewCanvas width="200" height="200"&gt;&lt;/canvas&gt;

`
```

```
})
export class ProductEditorComponent
implements OnChanges, OnInit, AfterViewInit, OnDestroy {

@Input({ required: true }) productId!: number;

// Signals for state
readonly product = signal&lt;Product | null&gt;(null);
readonly isLoading = signal(false);
readonly isSaving = signal(false);

// Form set up in ngOnInit
form: FormGroup;

// ViewChild available in ngAfterViewInit
@ViewChild('previewCanvas') canvasRef!: ElementRef&lt;HTMLCanvasElement&gt;;

private readonly destroyRef = inject(DestroyRef);

constructor(
private readonly productService: ProductService,
private readonly fb: FormBuilder,
private readonly analytics: AnalyticsService
) {
// CONSTRUCTOR DI only form builder needs no data yet
// effect() for side effects constructor = injection context
effect(() =&gt; {
const p = this.product();
if (p) {
// Analytics side effect fires when product Signal changes
this.analytics.track('product_viewed', { id: p.id });
}
});
```

```
}

// NGONCHANGES react to productId changes

ngOnChanges(changes: SimpleChanges): void {

if (changes['productId']) {

// Fires on first load AND when productId changes

// Use instead of ngOnInit when @Input drives the data load

this.loadProduct(this.productId);

}

}

// NGONINIT one-time setup that doesn't depend on @Input values

ngOnInit(): void {

// Form setup doesn't need product data set up once

this.form = this.fb.group({

name: ['', Validators.required],

price: [0, [Validators.required, Validators.min(0)]],

description: ['']

});

// Sync product Signal to form when product loads

effect(() =&gt; {

const p = this.product();

if (p) {

// Patch form when product Signal changes

this.form.patchValue({

name: p.name,

price: p.price,

description: p.description

});

}
```

```
}, { allowSignalWrites: true });

}

// NGAFTERVIEWINIT DOM access @ViewChild available here

ngAfterViewInit(): void {

// canvas is rendered initialise image preview

if (this.canvasRef) {

this.initImagePreview(this.canvasRef.nativeElement);

}

}

// NGONDESTROY cleanup (mostly handled by takeUntilDestroyed)

ngOnDestroy(): void {

// DestroyRef handles subscription cleanup automatically

// Add anything else that needs explicit cleanup

this.form.reset(); // optional clean up form state

}

// ---- Methods ----

private loadProduct(id: number): void {

this.isLoading.set(true);

this.product.set(null);

this.productService.getProduct(id).pipe(

takeUntilDestroyed(this.destroyRef)

).subscribe({

next: p =&gt; { this.product.set(p); this.isLoading.set(false); },

error: () =&gt; { this.isLoading.set(false); }

});

}

onSubmit(): void {
```

```
if (this.form.invalid) return;

this.isSaving.set(true);


this.productService.update(this.productId, this.form.value).pipe(

takeUntilDestroyed(this.destroyRef)

).subscribe({

next: () =&gt; { this.isSaving.set(false); },

error: () =&gt; { this.isSaving.set(false); }

});

}


private initImagePreview(canvas: HTMLCanvasElement): void {

const ctx = canvas.getContext('2d');

ctx .fillRect(0, 0, 200, 200); // example DOM work

}

}
```

4. Before & After Refactor

The Problem in Plain English:

The most common lifecycle mistake is putting everything in ngOnInit -- including things that should react to @Input() changes, things that need the DOM, and things that need cleanup. This creates a component that works on first load but breaks when inputs change, crashes when accessing @ViewChild, and leaks memory. Using the right hook for each task is the difference between a fragile component and a robust one.

```
// BEFORE everything crammed into ngOnInit common beginner pattern


@Component({

selector: 'app-user-profile',

template: `

&lt;canvas #chart&gt;&lt;/canvas&gt;

&lt;div *ngFor="let item of data"&gt;{{ item.name }}&lt;/div&gt;

`

})

export class UserProfileComponent implements OnInit, OnDestroy {
```

```
@Input() userId: number;

@ViewChild('chart') chartRef: ElementRef; // needs AfterViewInit

data: Item[] = [];
private chart: ChartJS;
private sub: Subscription;

ngOnInit(): void {

// Accessing @ViewChild not rendered yet chartRef is undefined
this.chart = new ChartJS(
this.chartRef.nativeElement, // TypeError: cannot read nativeElement of undefined
{ type: 'bar', data: [] }
);

// Data load in ngOnInit works on first load
// but if userId @Input changes later ngOnInit won't fire again stale data
this.sub = this.dataService.getData(this.userId).subscribe(data =&gt; {
this.data = data;
});

// No cleanup for chart third-party library leaked
}

ngOnDestroy(): void {
this.sub .unsubscribe(); // partial cleanup chart not destroyed
}
}


// AFTER right hook for every task

@Component({
```

```
selector: 'app-user-profile',

changeDetection: ChangeDetectionStrategy.OnPush,

standalone: true,

imports: [CommonModule],

template: `

&lt;canvas #chart&gt;&lt;/canvas&gt;

@for (item of data(); track item.id) {

&lt;div&gt;{{ item.name }}&lt;/div&gt;

}

`

})

export class UserProfileComponent

implements OnChanges, AfterViewInit, OnDestroy {


@Input({ required: true }) userId!: number;

@ViewChild('chart') chartRef!: ElementRef&lt;HTMLCanvasElement&gt;;


readonly data = signal&lt;Item[]&gt;([]);

readonly isLoading = signal(false);


private chart: ChartJS | null = null;

private readonly destroyRef = inject(DestroyRef);


constructor(private readonly dataService: DataService) {}


// ngOnChanges handles BOTH first load AND userId changes

ngOnChanges(changes: SimpleChanges): void {

if (changes['userId']) {

this.loadData(this.userId);

}

}


// ngAfterViewInit DOM is ready, @ViewChild available
```

```
ngAfterViewInit(): void {

this.chart = new ChartJS(

this.chartRef.nativeElement, // rendered now available

{ type: 'bar', data: { labels: [], datasets: [] } }

);


// Sync chart when data signal changes

effect(() =&gt; {

this.chart .data.datasets[0] .data.push(...this.data());

this.chart .update();

});

}


// ngOnDestroy complete cleanup

ngOnDestroy(): void {

this.chart .destroy(); // third-party cleanup

// takeUntilDestroyed handles subscription cleanup

}


private loadData(userId: number): void {

this.isLoading.set(true);

this.dataService.getData(userId).pipe(

takeUntilDestroyed(this.destroyRef)

).subscribe({

next: items =&gt; { this.data.set(items); this.isLoading.set(false); },

error: () =&gt; { this.isLoading.set(false); }

});

}

}
```

Why it matters: The before version has three separate bugs -- @ViewChild accessed before the DOM exists (crash), ngOnInit not responding to input changes (stale data), and a leaked chart library (memory leak). The after version uses exactly the right hook for each task -- no crashes, no stale data, no leaks.

---

5. Common Mistakes & Misconceptions

"Put everything in ngOnInit -- it runs after the component is ready"

ngOnInit runs after the first ngOnChanges -- meaning inputs are available. But the DOM is NOT ready (@ViewChild is undefined), child components are NOT rendered, and it fires only ONCE -- so it won't respond to input changes. Putting DOM access or third-party library initialisation here crashes. Putting input-driven data loading here breaks when inputs change.

Think of the employee analogy -- ngOnInit is "first day starts." You can read your job description (inputs) but you can't use the meeting rooms yet (DOM not rendered) and you only have a first day once -- if your role changes later, there's no second first day.

```
// Three common ngOnInit mistakes:


ngOnInit(): void {


// Mistake 1: DOM access not rendered yet

this.canvasRef.nativeElement.focus(); // undefined


// Mistake 2: input-driven data that won't reload when input changes

this.loadData(this.userId); // only runs once won't re-run if userId changes


// Mistake 3: third-party library that needs the DOM

this.map = new GoogleMap(this.mapContainer.nativeElement); // undefined

}


// Each in the right hook:

ngOnChanges(changes: SimpleChanges): void {

if (changes['userId']) { this.loadData(this.userId); } // re-runs on change

}


ngAfterViewInit(): void {

this.canvasRef.nativeElement.focus(); // DOM ready

this.map = new GoogleMap(this.mapContainer.nativeElement); // DOM ready

}
```

"ngOnChanges only fires when the component is created"

ngOnChanges fires on first load and every time any @Input() value changes. If a parent changes a bound input -- userId going from 1 to 2 -- ngOnChanges fires again. This makes it the right place for data loading that depends on @Input() values. This is one of the most valuable and under-used hooks.

```
// Parent changes userId input over time

@Component({

template: `&lt;app-user [userId]="currentId" /&gt;`

})

export class ParentComponent {

currentId = 1;

switchUser(): void { this.currentId = 2; } // triggers child's ngOnChanges

}


// Child ngOnChanges fires BOTH times

@Component({ selector: 'app-user', ... })

export class UserComponent implements OnChanges {

@Input() userId: number;


ngOnChanges(changes: SimpleChanges): void {

if (changes['userId']) {

// Fires when: userId = 1 (firstChange = true)

// AND when: userId = 2 (firstChange = false)

this.loadUser(this.userId); // always loads the correct user

}

}


// ngOnInit would only load userId=1 never userId=2

}
```

"Updating state in ngAfterViewInit is fine -- the view is ready"

Angular runs change detection before ngAfterViewInit fires -- the view has been evaluated and committed. If you update a template-bound property in ngAfterViewInit, Angular detects a mismatch between the committed state and the new state during the dev mode extra check. This throws ExpressionChangedAfterItHasBeenCheckedError -- one of the most confusing Angular errors.

```
// Updating template-bound property in ngAfterViewInit

@Component({

template: `&lt;p&gt;{{ title }}&lt;/p&gt;`

})
```

```
export class ProblemComponent implements AfterViewInit {

title = 'Loading...';

ngAfterViewInit(): void {
this.title = 'Ready'; // ExpressionChangedAfterItHasBeenCheckedError
// Angular committed 'Loading...' then found 'Ready' in the extra check
}
}

// Fix 1 Promise.resolve() defers to next microtask
ngAfterViewInit(): void {
Promise.resolve().then(() =&gt; {
this.title = 'Ready'; // runs after current change detection cycle
});
}

// Fix 2 Signal Signal updates are handled correctly
title = signal('Loading...');

ngAfterViewInit(): void {
this.title.set('Ready'); // Signals handle the timing correctly
}

// Fix 3 ChangeDetectorRef.detectChanges()
constructor(private cdr: ChangeDetectorRef) {}

ngAfterViewInit(): void {
this.title = 'Ready';
this.cdr.detectChanges(); // run change detection immediately after
}
```

6. Do's and Don'ts

| [Y] Do | [N] Don't |

|---|---|

| Use ngOnChanges for data loading driven by @Input() values -- handles both first load and changes | Use ngOnInit alone for @Input()-driven data loading -- won't respond to input changes |

| Use ngOnInit for setup that doesn't depend on @Input() values -- form creation, non-input subscriptions | Access @ViewChild in ngOnInit -- it's not rendered yet -- always undefined |

| Use ngAfterViewInit for DOM access, @ViewChild usage, and third-party library initialisation | Update template-bound properties directly in ngAfterViewInit -- throws ExpressionChangedAfterItHasBeenCheckedError |

| Use ngOnDestroy for cleanup that can't use takeUntilDestroyed -- third-party libraries, timers | Leave ngOnDestroy empty thinking Angular handles all cleanup -- it doesn't handle third-party libraries |

| Implement lifecycle interfaces explicitly -- implements OnInit, OnDestroy | Skip implementing interfaces -- a typo in the method name silently never fires |

| Use takeUntilDestroyed / DestroyRef.onDestroy() -- reduces what ngOnDestroy needs to do | Put all cleanup in ngOnDestroy -- DestroyRef is more composable |

---

7. Debugging Tips

Bug: @ViewChild is undefined in ngOnInit

```
TypeError: Cannot read properties of undefined (reading 'nativeElement')
```

* Plain English cause: ngOnInit runs before the template is rendered -- @ViewChild has nothing to reference yet

* Fix: Move all @ViewChild access to ngAfterViewInit

Bug: Component shows stale data when navigating between items

```
Navigating from user 1 to user 2 component shows user 1's data
```

* Plain English cause: Data loading is in ngOnInit -- only fires once -- changing userId input doesn't trigger a reload

* Fix: Move data loading to ngOnChanges and check changes['userId']

Bug: ExpressionChangedAfterItHasBeenCheckedError in dev mode

```
ExpressionChangedAfterItHasBeenCheckedError: Expression has changed after it was
checked
```

* Plain English cause: A template-bound property was changed after Angular committed the view -- usually in ngAfterViewInit or in a lifecycle hook that fires after change detection

* Fix: Wrap the state update in Promise.resolve().then(...), use a Signal, or call ChangeDetectorRef.detectChanges() immediately after the update

Bug: ngOnChanges not firing on object @Input() mutations

```
Parent updates object property ngOnChanges never fires in child
```

Plain English cause: Angular's ngOnChanges only fires when the input reference* changes -- mutation (same object, new property value) is invisible to it. Same principle as OnPush (Topic 24)

* Fix: Parent must create a new object reference -- { ...existingObj, prop: newVal }

Bug: Memory leak -- third-party library not cleaned up

```
Chart/map/widget still active after component is destroyed
```

* Plain English cause: Third-party library initialised in ngAfterViewInit but destroy() not called in ngOnDestroy

* Fix: Call the library's cleanup method in ngOnDestroy or via destroyRef.onDestroy()

8. Real-World Applications

Application 1 -- User Profile with Navigation Between Profiles
Plain English first:

A user profile component that can show any user's data -- the active user ID comes from the route as an @Input(). When the user navigates from /users/1 to /users/2, the component is reused by Angular's router -- it doesn't get recreated. ngOnInit won't fire again. Only ngOnChanges fires with the new userId. If data loading is in ngOnInit, user 2 always shows user 1's data.

```
@Component({

selector: 'app-user-profile',

changeDetection: ChangeDetectionStrategy.OnPush,

standalone: true,

imports: [CommonModule],

template: `

@if (isLoading()) {

&lt;div class="skeleton-profile"&gt;Loading...&lt;/div&gt;

}


@if (user(); as u) {

&lt;div class="profile"&gt;

&lt;img [src]="u.avatarUrl '/default-avatar.png'" [alt]="u.name"&gt;

&lt;h1&gt;{{ u.name }}&lt;/h1&gt;

&lt;p&gt;{{ u.bio 'No bio provided' }}&lt;/p&gt;

&lt;p&gt;Member since: {{ u.joinedAt | date:'longDate' }}&lt;/p&gt;

&lt;p&gt;Posts: {{ u.postCount 0 }}&lt;/p&gt;

&lt;/div&gt;

}

`


})

export class UserProfileComponent implements OnChanges, OnDestroy {
```

```
// Route input provided by Angular router in v16+
@Input({ required: true }) userId!: string;

readonly user = signal&amp;lt;User | null&amp;gt;(null);
readonly isLoading = signal(false);

private readonly destroyRef = inject(DestroyRef);

constructor(private readonly userService: UserService) {}

// ngOnChanges fires on FIRST LOAD and on EVERY userId change
// Correct: loads the right user on navigation
ngOnChanges(changes: SimpleChanges): void {
if (changes['userId']) {
this.user.set(null); // clear previous user
this.isLoading.set(true);

this.userService.getUser(Number(this.userId)).pipe(
takeUntilDestroyed(this.destroyRef)
).subscribe({
next: user =&amp;gt; { this.user.set(user); this.isLoading.set(false); },
error: () =&amp;gt; { this.isLoading.set(false); }
});
}
}

ngOnDestroy(): void {
// takeUntilDestroyed handles subscription cleanup
// Nothing else to clean up here
}
}
```

Application 2 -- Interactive Map Component
Plain English first:

A map component using a third-party library (Leaflet, Google Maps, Mapbox) needs the DOM element before it can initialise -- the library needs to attach to a real HTML element. ngOnInit fires before the element exists. ngAfterViewInit fires after it's rendered. The map must be initialised in ngAfterViewInit and destroyed in ngOnDestroy -- third-party libraries don't clean themselves up.

```
@Component({

selector: 'app-location-map',

changeDetection: ChangeDetectionStrategy.OnPush,

standalone: true,

template: `

&lt;div #mapContainer class="map-container" style="height: 400px"&gt;&lt;/div&gt;

`

})

export class LocationMapComponent implements OnChanges, AfterViewInit, OnDestroy {


@Input() latitude: number = 51.5074;

@Input() longitude: number = -0.1278;

@Input() zoom: number = 13;


@ViewChild('mapContainer') mapContainer!: ElementRef&lt;HTMLDivElement&gt;;


private map: LeafletMap | null = null;

private marker: LeafletMarker | null = null;


private readonly destroyRef = inject(DestroyRef);


// ngAfterViewInit mapContainer is rendered library can attach

ngAfterViewInit(): void {

this.map = L.map(this.mapContainer.nativeElement).setView(

[this.latitude, this.longitude],

this.zoom

);


L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png').addTo(this.map);


this.marker = L.marker([this.latitude, this.longitude]).addTo(this.map);
```

```
// Register cleanup via destroyRef cleaner than ngOnDestroy

this.destroyRef.onDestroy(() =&gt; {

this.map .remove(); // Leaflet cleanup releases DOM and listeners

this.map = null;

this.marker = null;

});

}


// ngOnChanges update map when coordinates change

ngOnChanges(changes: SimpleChanges): void {

if (!this.map) return; // map not initialised yet AfterViewInit hasn't run


if (changes['latitude'] || changes['longitude']) {

const latlng: [number, number] = [this.latitude, this.longitude];

this.map.setView(latlng, this.zoom);

this.marker .setLatLng(latlng);

}


if (changes['zoom']) {

this.map.setZoom(this.zoom);

}

}


ngOnDestroy(): void {

// destroyRef.onDestroy already registered above

// ngOnDestroy can be empty or removed entirely if using DestroyRef

}

}
```

9. Practice Exercises

Exercise 1 -- Beginner

Build a CountdownTimerComponent that takes a @Input() seconds: number -- the countdown duration. It should: start counting down from seconds when initialised, update every second using

setInterval, show the remaining time in the template, show "Time's up!" when it reaches zero, and restart correctly when the seconds input changes to a new value. Use the correct lifecycle hook for starting the interval, correctly restart when input changes, and clean up the interval in ngOnDestroy. Add console.log to every lifecycle hook to observe the firing order.

Exercise 2 -- Intermediate

Build a DataTableComponent with two @Input() values -- endpoint: string (the API URL) and pageSize: number. The component should: load data from the endpoint in the correct lifecycle hook, reload when the endpoint changes, use @ViewChild to access a <table> element and measure its width in ngAfterViewInit, store the width in a Signal, apply conditional styling when the table is too narrow (< 600px) for full column display, and clean up properly. Verify it works when switching between two different endpoints.

Exercise 3 -- Advanced

Build a VideoPlayerComponent that wraps a third-party video player library (simulate with a class that needs .init(element) and .destroy()). The component should: receive @Input() videoUrl: string and @Input() autoPlay: boolean, initialise the player in ngAfterViewInit, load the new video URL in ngOnChanges without reinitialising the player (use the library's .loadUrl(url) method), handle the case where ngOnChanges fires before ngAfterViewInit (URL arrives before player is ready -- queue it), clean up the player in ngOnDestroy, and use ChangeDetectionStrategy.OnPush throughout. Document exactly why each lifecycle hook was chosen for each task.

---

10. Key Takeaways

Core Concepts in Plain English

* constructor = class created, DI ready -- inputs NOT available -- only inject services here
* ngOnChanges = inputs received -- fires before ngOnInit AND on every input change -- for input-driven work
* ngOnInit = first day -- inputs available, DOM not ready -- one-time setup not driven by inputs
* ngAfterViewInit = DOM and children rendered -- @ViewChild available -- third-party library init
* ngOnDestroy = component being removed -- final cleanup -- takeUntilDestroyed reduces what goes here
* ngDoCheck = every change detection cycle -- rarely needed with Signals -- use for custom detection only

The Lifecycle Order

```
CREATION:

constructor DI only

ngOnChanges inputs available (firstChange = true)

ngOnInit one-time setup

ngDoCheck

ngAfterContentInit ng-content ready

ngAfterContentChecked

ngAfterViewInit DOM + @ViewChild ready key milestone
```

```
ngAfterViewChecked


ON EVERY CHANGE:

ngOnChanges only if @Input changed

ngDoCheck

ngAfterContentChecked

ngAfterViewChecked


DESTRUCTION:

ngOnDestroy clean up everything
```

The Right Hook for Every Task

```
Task Correct hook


Inject a service constructor (parameter)

Set up an effect() constructor (injection context)

Load data from @Input ngOnChanges

One-time setup (form, non-input) ngOnInit

Subscribe to non-input Observables ngOnInit

Access @ViewChild ngAfterViewInit

Init third-party DOM library ngAfterViewInit

Respond to @Input object changes ngOnChanges (need new reference)

React to every CD cycle ngDoCheck (rare)

Clean up subscriptions takeUntilDestroyed (preferred)

Clean up third-party libraries ngOnDestroy or destroyRef.onDestroy()
```

Angular-Specific Rules

* Always implement lifecycle interfaces -- implements OnInit -- typos silently fail without them
* ngOnChanges receives SimpleChanges -- check changes['propName'] before acting
* firstChange property -- use to distinguish initial load from subsequent changes
* @ViewChild only available from ngAfterViewInit -- never in ngOnInit
* Use takeUntilDestroyed for subscriptions -- ngOnDestroy then only handles non-Observable cleanup
* Updating state in ngAfterViewInit throws ExpressionChangedAfterItHasBeenChecked -- use Promise.resolve() or Signal

One-Line Memory Aid

> constructor = hired (DI only). ngOnChanges = briefed (inputs ready, fires every time). ngOnInit = first day (setup, once). ngAfterViewInit = office access (DOM ready). ngOnDestroy = leaving (clean up). Wrong hook = subtle timing bug.

---

11. Thought-Provoking Question + Answer

> Angular 17+ introduced input() signal functions as a replacement for @Input() decorators. If inputs are now Signals, ngOnChanges becomes redundant -- you can use computed() or effect() to react to input changes instead. Does this mean ngOnChanges is being deprecated, and what's the cleanest way to react to input changes in modern Angular

Plain English explanation first:

Think of ngOnChanges as a notification letter you receive whenever your job description changes. The letter lists all the changes -- what changed, what it was before, what it is now. Useful, but you have to open the letter, parse it, and figure out what to do.

Signal inputs are like a live display board showing your current responsibilities. You don't wait for a letter -- you just look at the board whenever you need to know. And anything that watches the board (a computed() or effect()) updates automatically the moment the board changes.

```
// OLD WAY @Input() + ngOnChanges notification letter system

@Component({ ... })

export class OldComponent implements OnChanges {

@Input() userId: number;


ngOnChanges(changes: SimpleChanges): void {

if (changes['userId']) {

this.loadUser(this.userId); // load on every userId change

}

}

}



// NEW WAY input() Signal + effect() live display board

@Component({ ... })

export class NewComponent {


// input() returns a Signal readable with ()

readonly userId = input.required&lt;number&gt;(); // WritableSignal equivalent


constructor(private readonly userService: UserService) {
```

```
// effect() reacts to userId signal changes like watching the board

effect(() =&gt; {

const id = this.userId(); // read Signal tracked as dependency

this.loadUser(id); // re-runs automatically when userId changes

});

}

}
```

The full migration from @Input() lifecycle hooks to signal inputs:

```
// ---- SIGNAL INPUT PATTERNS ----


// input() optional with default

readonly pageSize = input(20); // Signal&lt;number&gt;, default 20


// input.required() must be provided by parent

readonly userId = input.required&lt;number&gt;(); // Signal&lt;number&gt;, no default


// input() with transform converts incoming value

readonly isActive = input(false, {

transform: booleanAttribute // converts string 'true'/'false' to boolean

});


// input() with alias

readonly userId = input.required&lt;number&gt;({ alias: 'id' }); // parent uses [id]



// ---- REACTING TO SIGNAL INPUT CHANGES ----


// Pattern 1 computed() for derived values (no side effects)

readonly user = computed(() =&gt; {

const id = this.userId();

// But: computed can't do async this returns a Signal, not an Observable

// Use for synchronous derivation only

return this.userCache.get(id) null;
```

```
});

// Pattern 2 effect() for side effects (HTTP calls, DOM, analytics)

constructor() {

effect(() =&gt; {

const id = this.userId(); // tracked

this.loadUser(id); // side effect HTTP call

});

}

// Pattern 3 toSignal() with switchMap for async with cancellation

readonly userData = toSignal(

toObservable(this.userId).pipe( // Signal Observable

switchMap(id =&gt; this.userService.getUser(id)), // cancel previous on new id

catchError(() =&gt; of(null))

),

{ initialValue: null }

);

// Template: {{ userData() .name }}

// Auto-cancels previous HTTP call when userId changes

// Auto-manages subscription via DestroyRef

// This is the most complete pattern for async input-driven data


// ---- IS ngOnChanges DEPRECATED ----

// Not deprecated Angular team confirmed ngOnChanges remains supported

// But with signal inputs: ngOnChanges does NOT fire

// Signal inputs bypass the ngOnChanges system entirely

// Current Angular stance:

// @Input() + ngOnChanges = still supported, still idiomatic

// input() + effect/computed = modern, preferred for new code
```

```
// Both work Angular won't break @Input() and ngOnChanges


// The migration path:

// @Input() userId: number userId = input.required&lt;number&gt;()

// ngOnChanges + if(changes.userId) effect(() =&gt; { this.userId() ... })

// or toSignal(toObservable(this.userId).pipe(switchMap(...)))
```

Practical rule to take away:

> "In modern Angular (17+), prefer input() signal functions over @Input() decorators for new components. React to signal input changes with effect() for side effects and computed() for derived values. For async operations driven by an input (like loading data when userId changes), toSignal(toObservable(this.inputSignal).pipe(switchMap(...))) is the most complete pattern -- it handles cancellation, subscription management, and OnPush automatically. ngOnChanges remains valid and supported -- you'll use it when maintaining existing codebases -- but new code reads more cleanly with signal inputs and reactive derivations."

12. Related Topics to Learn Next

Angular-Specific Concepts

- @Input() & @Output() in depth (Topic 26 -- signal inputs input(), required inputs, transforms, aliases -- the direct extension of this topic)*
- Services & Dependency Injection (Topic 27 -- services injected in constructor, used in lifecycle hooks -- the full DI picture)*
- Angular Template Syntax (Topic 28 -- @if, @for, @switch -- what gets rendered during the AfterViewInit phase)*
- Angular Reactive Forms (Topic 33 -- forms initialised in ngOnInit, form values in ngOnChanges -- lifecycle hooks throughout)*
- Angular Testing Basics (Topic 44 -- testing lifecycle hooks -- TestBed.createComponent(), fixture.detectChanges() -- how lifecycle is triggered in tests)*

13. Updated Learning Tracker

```
Topics 1 19 ES6/JS + TypeScript Foundation COMPLETE

F1 F5 Angular Foundations Overview COMPLETE

Topic 20 RxJS Subscription Management COMPLETE

Topic 21 takeUntilDestroyed &amp; DestroyRef COMPLETE

Topic 22 async pipe COMPLETE

Topic 23 Angular Signals COMPLETE

Topic 24 OnPush Change Detection COMPLETE

Topic 25 Component Lifecycle Hooks COMPLETE

Topic 26 @Input() &amp; @Output() in depth NEXT
```

```
TIER 1 REMAINING:

26. @Input() &amp;amp; @Output() in depth

27. Services &amp;amp; Dependency Injection


TIER 2 UPCOMING:

28 36. Template Syntax HTTP Interceptors


TIER 3 WHEN NEEDED:

37 46. Directives RxJS Operators
```

Key Concept Added to Quick Reference
Topic 25 -- Component Lifecycle Hooks

> constructor = hired (DI only). ngOnChanges = briefed (inputs ready, fires every time). ngOnInit = first day (setup, once). ngAfterViewInit = office access (DOM ready). ngOnDestroy = leaving (clean up). Wrong hook = subtle timing bug.

* constructor = DI injection only -- inputs NOT available
* ngOnChanges = fires BEFORE ngOnInit and on EVERY @Input() change -- use for input-driven data loading
* ngOnInit = fires once -- inputs available, DOM NOT ready -- one-time setup
* ngAfterViewInit = DOM and @ViewChild ready -- third-party library init
* ngOnDestroy = final cleanup -- takeUntilDestroyed handles subscriptions
* ExpressionChangedAfterItHasBeenChecked = updating state in ngAfterViewInit -- use Promise.resolve() or Signal
* Signal inputs (input()) + effect() = modern alternative to @Input() + ngOnChanges

ngOnChanges only fires when @Input() reference* changes -- mutation invisible (same as OnPush rule)

Topic 26 -- @Input() & @Output() in Depth in Angular

0. Difficulty & Priority Rating

* Difficulty: Beginner-Intermediate -- the basic syntax is simple; the depth comes from understanding required inputs, transforms, aliases, signal inputs, and the correct patterns for passing data between components

* Angular Relevance: Critical -- @Input() and @Output() are the primary mechanism for component communication in Angular; every parent-child relationship in your app uses these; getting them wrong causes silent bugs, stale data, and broken templates

1. Explanation

The Manager and Employee Analogy

Imagine a company with a manager (parent component) and an employee (child component):

@Input() = the manager giving the employee their work brief. The manager decides what information the employee receives -- the employee can't ask for it themselves. The brief arrives before the employee starts work. If the brief changes, the employee gets an updated brief.

@Output() = the employee's reporting system. When they finish a task, complete a form, or discover something important, they send a report UP to the manager. The manager decides what to do with the report. The employee doesn't know what the manager does with it -- they just send it.

```
Manager (Parent)


gives brief (Input): [user]="selectedUser"

employee receives it: @Input() user: User


employee reports back: @Output() userSaved = new EventEmitter&lt;User&gt;()

manager handles report: (userSaved)="onUserSaved($event)"
```

The USB Port Analogy

Think of @Input() as a USB port on a device. The device doesn't reach out to get power -- power is pushed INTO it. @Output() is like a speaker jack -- sound comes OUT of the device when something internal triggers it. The device defines what shape of plug fits (the type) but it doesn't control what's plugged in or what listens to the output.

In Plain English

* @Input() = a property that receives data pushed from a parent -- the parent controls what goes in

* @Output() = an EventEmitter that sends data up to the parent -- the child controls when it emits

* Unidirectional data flow = data flows DOWN via inputs, events flow UP via outputs -- Angular's fundamental design principle

* input() function = Angular 17+ Signal-based alternative to @Input() decorator -- returns a Signal, more powerful

* output() function = Angular 17+ alternative to @Output() -- cleaner syntax, no EventEmitter needed

Now the Technical Terms

* @Input() = a decorator that marks a class property as an input binding -- parent template binds to it with [property]="value"

* @Input({ required: true }) = Angular 16+ -- TypeScript and Angular both enforce the input must be provided

* @Input({ transform }) = Angular 16+ -- automatically transforms the incoming value before it reaches the property

* @Input('alias') = the parent uses a different name than the internal property name

* @Output() = a decorator marking an EventEmitter property -- parent template listens with (eventName)="handler($event)"

* EventEmitter<T> = a class that extends RxJS Subject -- .emit(value) triggers all parent listeners

* input() / output() = Angular 17+ functional alternatives -- inputs return Signals, outputs use a simpler API

* $event = the Angular template variable that receives whatever the EventEmitter emitted

---

2. Connection to Prior Concepts

@Input() and @Output() connect directly to almost every previous topic:

* Topic 7 (Pass by Reference) -- when a parent passes an object via @Input(), the child receives a reference -- a house key. If the child mutates that object, the parent sees the mutation. Never mutate @Input() objects in the child -- always spread first

Topic 24 (OnPush Change Detection) -- OnPush checks if @Input() references* change. Same reference = no check. This is why immutable updates from Topic 14 are mandatory in OnPush component trees

* Topic 25 (Lifecycle Hooks) -- @Input() values are available from ngOnChanges (fires before ngOnInit). ngOnChanges fires on every @Input() change -- essential for reloading data when inputs update

* Topic 23 (Signals) -- input() function returns a Signal -- you can use computed() to derive values from it and effect() to react to changes. More powerful than @Input() + ngOnChanges

* Topic 13 (Interfaces) -- every @Input() should have a TypeScript type -- @Input() user!: User -- not @Input() user: any

Bridging note for your records:

"@Input() is the USB port -- data pushed in from outside. @Output() is the speaker -- signals sent out when something happens inside. The parent controls what goes in and decides what to do with what comes out. The child just defines the shape of the port and what it reports."

---

3. Code Example

Example 1 -- Basic @Input() and @Output()
The fundamental pattern -- every Angular developer writes this daily.

```
// ---- CHILD COMPONENT ----

@Component({

selector: 'app-user-card',

standalone: true,

imports: [CommonModule],
```

```
template: `
&lt;div class="card" [class.selected]="isSelected"&gt;
&lt;img [src]="user.avatar '/default.png'" [alt]="user.name"&gt;
&lt;h3&gt;{{ user.name }}&lt;/h3&gt;
&lt;p&gt;{{ user.email }}&lt;/p&gt;
&lt;span class="badge"&gt;{{ user.role }}&lt;/span&gt;
&lt;button (click)="onSelect()"&gt;Select&lt;/button&gt;
&lt;button (click)="onDelete()"&gt;Delete&lt;/button&gt;
&lt;/div&gt;
`
})
export class UserCardComponent {

// @Input() receives data FROM parent
@Input({ required: true }) user!: User; // required: true = Angular 16+
@Input() isSelected = false; // optional with default

// @Output() sends events TO parent
@Output() userSelected = new EventEmitter&lt;User&gt;();
@Output() userDeleted = new EventEmitter&lt;number&gt;(); // emits the id
@Output() dismissed = new EventEmitter&lt;void&gt;(); // no data

onSelect(): void {
this.userSelected.emit(this.user); // sends User up to parent
}
onDelete(): void {
this.userDeleted.emit(this.user.id); // sends id up to parent
}
}


// ---- PARENT COMPONENT ----
```

```
@Component({
selector: 'app-user-list',
standalone: true,
imports: [CommonModule, UserCardComponent],
template: `
@for (user of users; track user.id) {
&lt;app-user-card
[user]="user"
[isSelected]="selectedId === user.id"
(userSelected)="onUserSelected($event)"
(userDeleted)="onUserDeleted($event)"&gt;
&lt;/app-user-card&gt;
}
`
})
export class UserListComponent {

users: User[] = [];
selectedId: number | null = null;

// $event = whatever the EventEmitter emitted
onUserSelected(user: User): void {
this.selectedId = user.id;
}

onUserDeleted(id: number): void {
// Immutable update Topic 7 + 14
this.users = this.users.filter(u =&gt; u.id !== id);
// OnPush safe new array reference
}
}
```

Example 2 -- Angular 16+ @Input options -- required, transform, alias
The modern ways to use @Input() with more control.

```
@Component({ selector: 'app-product-card', template: '...' })

export class ProductCardComponent implements OnChanges {


// ---- required: true Angular AND TypeScript enforce this ----

@Input({ required: true }) product!: Product;

// Parent must provide [product]="..." TypeScript error if missing


// ---- transform automatically converts incoming value ----

// Parent passes a string: [price]="'29.99'"

// Component receives a number: 29.99

@Input({ transform: (v: string) =&gt; parseFloat(v) })

price = 0;


// Angular built-in transforms

@Input({ transform: numberAttribute }) quantity = 0; // string number

@Input({ transform: booleanAttribute }) isActive = false; // string boolean

// These are essential for HTML attribute bindings:

// &lt;app-product quantity="5" isActive&gt; (HTML attributes are always strings)

// Without transform: quantity would be the string '5', not number 5


// ---- alias parent uses different name than internal property ----

@Input('productData') product!: Product;

// Parent template: [productData]="selectedProduct"

// Internal code: this.product.name (uses internal name)

// Useful when: public API name differs from internal implementation name


// ---- @Output with alias ----

@Output('productClick') clicked = new EventEmitter&lt;Product&gt;();

// Parent: (productClick)="onProductClick($event)"

// Internal: this.clicked.emit(product)


ngOnChanges(changes: SimpleChanges): void {
```

```
// React to @Input() changes fires BEFORE ngOnInit and on every change

if (changes['product']) {

console.log('Product changed:', changes['product'].currentValue);

}

}

}
```

Example 3 -- Angular 17+ input() and output() -- signal-based
The modern alternative -- inputs are Signals, reactions via computed/effect.

```
import { input, output, computed, effect } from '@angular/core';


@Component({

selector: 'app-user-editor',

standalone: true,

template: `

&lt;!-- Signal inputs accessed with () like any other signal --&gt;

&lt;h2&gt;Editing: {{ user().name }}&lt;/h2&gt;

&lt;p&gt;Display: {{ displayName() }}&lt;/p&gt;

&lt;p&gt;Role badge: {{ roleBadge() }}&lt;/p&gt;


&lt;input

[value]="user().name"

(input)="onNameChange($event)"&gt;


&lt;button (click)="onSave()"&gt;Save&lt;/button&gt;

&lt;button (click)="onCancel()"&gt;Cancel&lt;/button&gt;

`

})

export class UserEditorComponent {


// input() returns a Signal&lt;User&gt;

readonly user = input.required&lt;User&gt;(); // required signal input

readonly mode = input&lt;'create' | 'edit'&gt;('edit'); // optional with default
```

```
readonly theme = input('light'); // inferred as Signal&lt;string&gt;

// computed() derived from signal input updates automatically

readonly displayName = computed(() =&gt;

`${this.user().name} (${this.user().role})`

);

readonly roleBadge = computed(() =&gt;

this.user().role === 'admin' ' Admin' : ' User'

);

// output() simpler than EventEmitter

readonly saved = output&lt;User&gt;();

readonly cancelled = output&lt;void&gt;();

constructor() {

// effect() reacts to user signal input changes

effect(() =&gt; {

console.log('User input changed:', this.user());

// Analytics, logging, DOM side effects here

});

}

onNameChange(event: Event): void {

// Note: input() signals are read-only you can't set them

// If you need local editable state, copy to a writable signal

console.log('Name changed to:', (event.target as HTMLInputElement).value);

}

onSave(): void {

this.saved.emit(this.user()); // emit the current signal value

}
```

```
onCancel(): void {
this.cancelled.emit(); // void no data needed
}
}

// ---- Parent using signal-input component same template syntax ----
@Component({
template: `
&lt;app-user-editor
[user]="selectedUser"
[mode]="editorMode"
(saved)="onSaved($event)"
(cancelled)="onCancelled()"&gt;
&lt;/app-user-editor&gt;
`
})
export class ParentComponent {
selectedUser: User = { id: 1, name: 'Alice', role: 'user' };
editorMode: 'create' | 'edit' = 'edit';

onSaved(user: User): void { console.log('Saved:', user); }
onCancelled(): void { console.log('Cancelled'); }
// Template syntax is identical whether child uses @Input() or input()
}
```

Angular Example -- Parent-child with immutable updates and OnPush

The complete production pattern -- everything from Topics 7, 14, 24, 25 working together.

```
// ---- Settings component receives complex @Input(), emits changes ----
@Component({
selector: 'app-settings-panel',
changeDetection: ChangeDetectionStrategy.OnPush,
standalone: true,
```

```
  imports: [CommonModule, ReactiveFormsModule],

  template: `

  @if (localSettings) {

  &lt;form [formGroup]="form" (ngSubmit)="onSave()"&gt;

  &lt;select formControlName="theme"&gt;

  &lt;option value="light"&gt;Light&lt;/option&gt;

  &lt;option value="dark"&gt;Dark&lt;/option&gt;

  &lt;/select&gt;

  &lt;input formControlName="fontSize" type="number" min="10" max="24"&gt;

  &lt;label&gt;

  &lt;input formControlName="notifications" type="checkbox"&gt;

  Notifications

  &lt;/label&gt;

  &lt;button type="submit" [disabled]="form.invalid || !form.dirty"&gt;

  Save Changes

  &lt;/button&gt;

  &lt;button type="button" (click)="onReset()"&gt;Reset&lt;/button&gt;

  &lt;/form&gt;

  }

  `

  })

  export class SettingsPanelComponent implements OnChanges, OnInit {


  @Input({ required: true }) settings!: UserSettings;

  @Input() userId!: number;


  @Output() settingsSaved = new EventEmitter&lt;UserSettings&gt;();

  @Output() settingsReset = new EventEmitter&lt;void&gt;();

  @Output() settingsChanged = new EventEmitter&lt;boolean&gt;(); // has unsaved changes


  form: FormGroup;
```

```
localSettings: UserSettings | null = null;

constructor(private readonly fb: FormBuilder) {}

ngOnInit(): void {
this.form = this.fb.group({
theme: ['light'],
fontSize: [14, [Validators.min(10), Validators.max(24)]],
notifications: [true]
});

// Track if form is dirty emit to parent
this.form.statusChanges.subscribe(() =&gt; {
this.settingsChanged.emit(this.form.dirty);
});
}

// ngOnChanges CRITICAL: handles both first load AND settings changes
ngOnChanges(changes: SimpleChanges): void {
if (changes['settings'] &amp;&amp; this.settings) {
// Spread to create local copy never mutate @Input()
this.localSettings = { ...this.settings };

// Patch form with new settings
this.form .patchValue({
theme: this.settings.theme,
fontSize: this.settings.fontSize,
notifications: this.settings.notifications
});
this.form .markAsPristine();
}
}
```

```
onSave(): void {
if (this.form.invalid) return;

// Emit new settings UP to parent parent decides what to do
// Immutable: spread localSettings + form values = new object
this.settingsSaved.emit({
...this.localSettings!,
...this.form.value
});
}

onReset(): void {
this.settingsReset.emit();
// Parent will update the @Input() settings ngOnChanges will fire
// and reset the form automatically
}
}
```

4. Before & After Refactor

The Problem in Plain English:

The two most common @Input()/@Output() mistakes are: (1) mutating the @Input() object directly in the child -- rearranging the parent's furniture -- and (2) doing all data loading in ngOnInit instead of ngOnChanges, so the component shows stale data when the input changes. Both are silent bugs that are hard to trace.

```
// BEFORE two common mistakes in one component

@Component({
selector: 'app-user-editor',
changeDetection: ChangeDetectionStrategy.OnPush,
template: `&lt;p&gt;{{ user.name }}&lt;/p&gt;`
})
export class UserEditorComponent implements OnInit {

@Input() user: User; // no required, no type safety
```

```
@Output() saved = new EventEmitter(); // untyped EventEmitter&lt;any&gt;

ngOnInit(): void {
// Mistake 1: Data loading in ngOnInit only
// If user @Input changes later (different userId), ngOnInit won't fire again
this.loadAdditionalData(this.user.id);
}

saveChanges(newName: string): void {
// Mistake 2: Mutating the @Input() directly
this.user.name = newName; // mutates parent's object!
// Parent's user.name is now changed they never asked for this
// OnPush also won't detect this same reference

this.saved.emit(this.user); // emitting mutated @Input wrong
}
}


// AFTER both mistakes fixed

@Component({
selector: 'app-user-editor',
changeDetection: ChangeDetectionStrategy.OnPush,
template: `
@if (localUser) {
&lt;p&gt;{{ localUser.name }}&lt;/p&gt;
&lt;button (click)="saveChanges('Bob')"&gt;Save&lt;/button&gt;
}
`
})
export class UserEditorComponent implements OnChanges, OnInit {
```

```
@Input({ required: true }) user!: User; // required + typed

@Output() saved = new EventEmitter&lt;User&gt;(); // typed


localUser: User | null = null; // child's own copy


ngOnInit(): void {

// ngOnInit: one-time setup NOT driven by @Input()

this.setupForm();

}


// Fix 1: ngOnChanges handles both first load AND every subsequent change

ngOnChanges(changes: SimpleChanges): void {

if (changes['user'] &amp;&amp; this.user) {

// Fix 2: Spread to create local copy never mutate @Input()

this.localUser = { ...this.user };

// Now localUser is the child's own house parent's house is untouched

}

}


saveChanges(newName: string): void {

if (!this.localUser) return;


// Immutable update of LOCAL copy parent's @Input() untouched

this.localUser = { ...this.localUser, name: newName };


// Emit new data UP to parent parent decides how to update its state

this.saved.emit({ ...this.localUser });

}


private setupForm(): void { /* form setup here */ }

}
```

Why it matters: The before version has two silent, hard-to-trace bugs. The mutation bug means the parent's data changes without the parent knowing -- causing inconsistent state across the app. The ngOnInit bug means navigating between users shows the wrong user's data. Both are fixed by one rule: treat @Input() as read-only and copy it before working with it.

5. Common Mistakes & Misconceptions

"I can mutate my @Input() object -- I'm just updating the same data"

Mutating an @Input() object is like the employee rewriting the manager's original brief. The manager still has the brief -- but it now says something different from what they wrote. Other employees who have copies of the same brief are now reading different information. The whole office is confused.

Always treat @Input() as read-only. Copy it with spread before modifying it, and emit changes back up via @Output().

```
// Mutating @Input() rearranging parent's furniture

@Input() config: AppConfig;


applyTheme(theme: string): void {

this.config.theme = theme; // mutates parent's object

// Parent sees this change without knowing it happened

// Other components sharing the same config reference are affected

}


// Copy first, emit back

@Input() config: AppConfig;

@Output() configChanged = new EventEmitter&lt;AppConfig&gt;();


localConfig: AppConfig;


ngOnChanges(changes: SimpleChanges): void {

if (changes['config']) {

this.localConfig = { ...this.config }; // own copy

}

}


applyTheme(theme: string): void {

this.localConfig = { ...this.localConfig, theme }; // local update

this.configChanged.emit({ ...this.localConfig }); // parent decides

}
```

"If @Input() data comes from an HTTP call, I can just use ngOnInit"

ngOnInit fires exactly once -- when the component is first created. If the component is reused by Angular's router with a different @Input() (e.g. navigating from /users/1 to /users/2), ngOnInit won't fire again. The component silently shows user 1's data for user 2.

Think of the employee analogy -- ngOnInit is their first briefing. If the job description changes later, they need ngOnChanges to receive the new brief -- ngOnInit only happens on day one.

```
// ngOnInit only won't reload when userId changes

@Input() userId: number;


ngOnInit(): void {

this.loadUser(this.userId); // only runs once stale data on navigation

}


// ngOnChanges runs on first load AND every userId change

ngOnChanges(changes: SimpleChanges): void {

if (changes['userId']) {

this.loadUser(this.userId); // always fresh

}

}
```

"Output events automatically update the parent's state"

@Output() just emits a value -- it does nothing else. The parent decides what to do with it. If the parent doesn't have a handler for the event, the emitted value disappears silently. The parent's state doesn't update itself -- the parent's handler method must explicitly update it.

Think of it like the employee sending a report up to the manager. Sending the report doesn't automatically change the company's records -- the manager has to receive it and take action. If the manager is busy and ignores the report, nothing happens.

```
// Child emits correctly

this.saved.emit(updatedUser);


// Parent template missing handler event emitted but nothing happens

&lt;app-user-editor

[user]="selectedUser"

(saved)=""&gt; &lt;!-- empty handler emitted data goes nowhere --&gt;


// Parent handler explicitly updates state

&lt;app-user-editor
```

```
[user]="selectedUser"

(saved)="onUserSaved($event)"&gt;


// Parent handler:

onUserSaved(updatedUser: User): void {

// Parent explicitly updates its own state immutable

this.users = this.users.map(u =&gt;

u.id === updatedUser.id updatedUser : u

);

}
```

6. Do's and Don'ts

| [Y] Do | [N] Don't |

|---|---|

| Use @Input({ required: true }) for inputs that must always be provided -- Angular 16+ | Leave required inputs without enforcement -- get confusing undefined errors at runtime |

| Use ngOnChanges for data loading driven by @Input() values -- handles first load and changes | Use ngOnInit alone for @Input()-driven data loading -- won't respond to input changes |

| Copy @Input() objects with spread before modifying -- this.local = { ...this.input } | Mutate @Input() objects directly in the child -- breaks parent state and OnPush |

| Type every @Output() EventEmitter -- new EventEmitter<User>() not new EventEmitter() | Use untyped EventEmitter -- loses all type safety on $event in parent template |

| Use input() and output() functions for new Angular 17+ components -- more powerful | Mix @Input() decorator and input() function on the same property -- pick one approach |

| Use @Input({ transform: numberAttribute }) for numeric inputs from HTML attributes | Manually parse string inputs -- the transform option is cleaner and type-safe |

7. Debugging Tips

Bug: @Input() property is undefined in ngOnInit

```
Cannot read properties of undefined user.name in ngOnInit
```

* Plain English cause: The @Input() might not have arrived yet, or the parent isn't providing it. ngOnInit fires after the first ngOnChanges so inputs should be available -- but if required: true isn't set and the parent forgets the binding, it stays undefined

* Fix: Add @Input({ required: true }) -- Angular will error at build time if parent forgets. Add . in the template as a safety net

Bug: Component shows stale data when navigating between items

```
Navigating from /users/1 to /users/2 component shows user 1's data
```

* Plain English cause: Data loading is in ngOnInit -- fires only once -- @Input() changed but the component didn't reload

* Fix: Move data loading to ngOnChanges and check changes['inputName']

Bug: Parent state unexpectedly changed -- nobody called the update method

```
Parent's user object has changed name but no explicit update was made
```

* Plain English cause: Child component mutated the @Input() object directly -- since objects are passed by reference (Topic 7), the parent sees the mutation

* Fix: In the child, always this.localUser = { ...this.user } before modifying

Bug: (outputEvent) handler not firing in parent

```
Button click in child does nothing parent handler never called
```

* Plain English cause: Either @Output() missing from the property, the event name in the template doesn't match exactly, or emit() is never called

* Fix: Verify @Output() decorator exists. Check template event name matches property name exactly. Verify emit() is called with correct data

Bug: ExpressionChangedAfterItHasBeenCheckedError with @Input() and OnPush

```
ExpressionChangedAfterItHasBeenCheckedError when @Input() changes
```

* Plain English cause: @Input() change triggered a template update after Angular committed the view. Common when the parent changes an input in response to an output event in the same change detection cycle

* Fix: Use ChangeDetectorRef.detectChanges() or markForCheck(), or restructure so the update happens before the cycle completes

8. Real-World Applications

Application 1 -- Data Table with Inline Editing

Plain English first:

A user management table where each row is a child component showing user data. Clicking "Edit" on a row makes the fields editable. When the user saves, the changes need to go back up to the parent (which owns the data and makes the HTTP call). The child receives the user data via @Input(), works on a local copy, and emits the updated user via @Output(). The parent handles saving -- the child just handles the display and editing UX.

```
@Component({

selector: 'app-user-row',

changeDetection: ChangeDetectionStrategy.OnPush,

standalone: true,

imports: [CommonModule, ReactiveFormsModule],

template: `

@if (!isEditing) {

&lt;td&gt;{{ user.name }}&lt;/td&gt;
```

```
&lt;td&gt;{{ user.email }}&lt;/td&gt;

&lt;td&gt;{{ user.role }}&lt;/td&gt;

&lt;td&gt;

&lt;button (click)="startEdit()"&gt;Edit&lt;/button&gt;

&lt;button (click)="onDelete()"&gt;Delete&lt;/button&gt;

&lt;/td&gt;

} @else {

&lt;td&gt;&lt;input [formControl]="nameControl"&gt;&lt;/td&gt;

&lt;td&gt;&lt;input [formControl]="emailControl"&gt;&lt;/td&gt;

&lt;td&gt;{{ user.role }}&lt;/td&gt;

&lt;td&gt;

&lt;button (click)="onSave()" [disabled]="isSaving"&gt;

{{ isSaving 'Saving...' : 'Save' }}

&lt;/button&gt;

&lt;button (click)="cancelEdit()"&gt;Cancel&lt;/button&gt;

&lt;/td&gt;

}

`

})

export class UserRowComponent implements OnChanges {


@Input({ required: true }) user!: User;

@Input() isSaving = false;


@Output() userUpdated = new EventEmitter&lt;User&gt;();

@Output() userDeleted = new EventEmitter&lt;number&gt;();


isEditing = false;

nameControl = new FormControl('');

emailControl = new FormControl('');


ngOnChanges(changes: SimpleChanges): void {
```

```
if (changes['user']) {

// Reset editing state when user input changes (e.g. after save)

if (this.isSaving) {

this.isEditing = false;

}

}

if (changes['isSaving'] &amp;&amp; !this.isSaving) {

// Saving completed exit edit mode

this.isEditing = false;

}

}


startEdit(): void {

// Copy @Input() values to local controls never bind form to @Input() directly

this.nameControl.setValue(this.user.name);

this.emailControl.setValue(this.user.email);

this.isEditing = true;

}


onSave(): void {

// Emit updated user UP parent makes the HTTP call

this.userUpdated.emit({

...this.user, // spread @Input() immutable

name: this.nameControl.value this.user.name,

email: this.emailControl.value this.user.email

});

}


cancelEdit(): void {

this.isEditing = false;

}
```

```
onDelete(): void {

this.userDeleted.emit(this.user.id);

}

}
```

Application 2 -- Multi-Step Form with Signal Inputs
Plain English first:

A checkout flow with multiple steps -- address, payment, confirmation -- each as a separate component. The parent orchestrates the flow, passing the current form data down and receiving completed step data up. Using Angular 17+ input() and output(), the step components react to input changes with computed() and send completed data back with output(). The parent never loses control of the data -- it always flows through the parent.

```
@Component({

selector: 'app-address-step',

standalone: true,

imports: [CommonModule, ReactiveFormsModule],

template: `

&lt;h2&gt;Step {{ stepNumber() }} of 3 {{ stepTitle() }}&lt;/h2&gt;

&lt;form [formGroup]="form" (ngSubmit)="onNext()"&gt;

&lt;input formControlName="line1" placeholder="Address line 1"&gt;

&lt;input formControlName="city" placeholder="City"&gt;

&lt;input formControlName="postcode" placeholder="Postcode"&gt;

&lt;button type="submit" [disabled]="form.invalid"&gt;Next&lt;/button&gt;

&lt;button type="button" (click)="onBack()"&gt;Back&lt;/button&gt;

&lt;/form&gt;

&lt;p *ngIf="!isValid()"&gt;Please fill in all required fields&lt;/p&gt;

`

})

export class AddressStepComponent {


// Signal inputs reacts automatically to changes

readonly stepNumber = input.required&lt;number&gt;();

readonly initialData = input&lt;Partial&lt;Address&gt;&gt;({});
```

```
// computed derives from signal inputs
readonly stepTitle = computed(() =&amp;gt;
`Delivery Address (Step ${this.stepNumber()})`
);
readonly isValid = computed(() =&amp;gt; this.form.valid);

// output() simpler than EventEmitter
readonly completed = output&amp;lt;Address&amp;gt;();
readonly back = output&amp;lt;void&amp;gt;();

readonly form = new FormGroup({
line1: new FormControl('', Validators.required),
city: new FormControl('', Validators.required),
postcode: new FormControl('', Validators.required)
});

constructor() {
// effect() sync signal input to form when initialData changes
effect(() =&amp;gt; {
const data = this.initialData();
if (data &amp;amp;&amp;amp; Object.keys(data).length &amp;gt; 0) {
this.form.patchValue(data);
}
});
}

onNext(): void {
if (this.form.invalid) return;
this.completed.emit(this.form.value as Address);
}

onBack(): void {
```

```
this.back.emit();

}

}
```

9. Practice Exercises

Exercise 1 -- Beginner

Build a RatingComponent that displays 5 stars. It should receive a @Input({ required: true }) rating: number (1-5) and emit @Output() ratingChanged = new EventEmitter<number>() when a star is clicked. The component should highlight stars up to the current rating. When a star is clicked, emit the new rating to the parent. The parent should display the current rating alongside the component. Verify that clicking a star updates the parent's display.

Exercise 2 -- Intermediate

Build a FilterPanelComponent that receives @Input() products: Product[] and @Input() categories: string[]. It should have local filter state (selected category, price range, in-stock only checkbox). When filters change, emit the filtered products via @Output() filtered = new EventEmitter<Product[]>(). Use ngOnChanges to reapply filters when the products input changes. Use OnPush change detection. Add @Input({ required: true }) where appropriate. The parent should update its displayed list whenever the filtered event fires.

Exercise 3 -- Advanced

Build a DataTableComponent<T> using Angular 17+ input() and output() functions. The component should accept: data = input.required<T[]>(), columns = input.required<TableColumn<T>[]>(), pageSize = input(10), sortable = input(true). It should emit: rowClick = output<T>(), sortChanged = output<SortEvent<T>>(), pageChanged = output<number>(). Use computed() for sorted and paginated data derived from signal inputs. Use effect() to reset to page 1 when data changes. Verify that changing the data input automatically recomputes the displayed rows.

10. Key Takeaways

Core Concepts in Plain English

* @Input() = USB port -- data pushed IN from parent -- parent controls what goes in
* @Output() = speaker jack -- signals sent OUT to parent -- child controls when it emits
* Unidirectional flow = data down via inputs, events up via outputs -- Angular's fundamental principle
* Never mutate @Input() = spread first, work on local copy, emit changes back
* ngOnChanges not ngOnInit = for @Input()-driven data loading -- fires on every change
* input() / output() = Angular 17+ signal-based alternatives -- inputs return Signals, more powerful

@Input() Options -- Quick Reference

```
@Input() // basic optional, any type

@Input({ required: true }) // required Angular 16+ enforced

@Input({ transform: numberAttribute }) // auto-transform incoming value

@Input('aliasName') // parent uses 'aliasName', code uses property name
```

```
@Input({ required: true, alias: 'data' }) // combined options


// Angular 17+ signal inputs

readonly value = input&lt;string&gt;(); // optional Signal input

readonly value = input.required&lt;string&gt;(); // required Signal input

readonly value = input('default'); // with default value

readonly value = input(0, { transform: numberAttribute }); // with transform
```

Quick Reference -- @Input() vs input() vs @Output() vs output()

Feature	@Input()	input()	@Output()	output()
Style	Decorator	Function	Decorator	Function
Returns	Property	Signal	Property	OutputEmitterRef
Computed from	ngOnChanges	computed()	N/A	N/A
React to changes	ngOnChanges	effect()	N/A	N/A
Available from	Angular 2+	Angular 17+	Angular 2+	Angular 17+
Recommended	Legacy/existing	New projects	Legacy/existing	New projects

Angular-Specific Rules

* Always @Input({ required: true }) for inputs that must always be provided
* Always copy @Input() objects with spread before modifying -- this.local = { ...this.input }
* Use ngOnChanges for @Input()-driven data loading -- not ngOnInit alone
* Type every EventEmitter -- new EventEmitter<User>() not new EventEmitter()
* $event in template = whatever the EventEmitter emitted -- same type as the generic
* Signal inputs (input()) + computed() = modern replacement for ngOnChanges pattern
* Enable strictTemplates: true -- catches wrong types passed to @Input() bindings

One-Line Memory Aid

> @Input() = USB port -- data pushed in from parent, treat as read-only. @Output() = speaker -- signals sent out when something happens inside. Copy before modifying. Use ngOnChanges not ngOnInit. Parent controls data; child controls events.

---

11. Thought-Provoking Question + Answer

> Angular 17+ introduced input() signal functions as alternatives to @Input() decorators. If input() returns a Signal, you can use computed() to derive values and effect() to react to changes -- replacing ngOnChanges entirely. Does this mean ngOnChanges is being deprecated, and what's the architectural difference between the two approaches

Plain English explanation first:

Think of ngOnChanges as a notification letter. Every time an input changes, Angular hands you a letter listing all the changes. You open the letter, read which inputs changed, and decide what to do. It works -- but it's reactive in a pull-based way: you receive the letter, then you query the new values.

Signal inputs are a live display board. You don't wait for a letter -- you describe a formula that reads from the board. When the board updates, the formula automatically recalculates. You never check for changes -- you just describe what things should be.

```
// ngOnChanges approach notification letter system

@Component({ ... })

export class OldComponent implements OnChanges {

@Input() userId: number;

@Input() filter: string;


ngOnChanges(changes: SimpleChanges): void {

// Letter arrives check what changed

if (changes['userId'] || changes['filter']) {

this.loadData(this.userId, this.filter);

}

}

}


// Signal input approach live display board

@Component({ ... })

export class NewComponent {

readonly userId = input.required&lt;number&gt;();

readonly filter = input('all');


// computed() describes the relationship auto-updates

readonly filteredData = computed(() =&gt;

this.allData().filter(item =&gt;

this.filter() === 'all' || item.category === this.filter()

)

);


// effect() for side effects (HTTP calls)

constructor() {

effect(() =&gt; {
```

```
// Runs whenever userId OR filter changes automatically

this.loadData(this.userId(), this.filter());

});

}

}
```

The architectural difference:

```
@Input() + ngOnChanges approach:

Imperative you check what changed and manually respond

ngOnChanges fires for ALL @Input() changes in one callback

You must check which inputs changed using SimpleChanges

Works in all Angular versions

ngOnChanges is NOT deprecated Angular team confirmed ongoing support


input() + computed/effect approach:

Declarative you describe what things should be

computed() only recomputes when its specific dependencies change

effect() re-runs when any Signal it reads changes

More granular each piece of derived state tracks its own dependencies

Angular 17+ only not available in older codebases


When ngOnChanges is still the right choice:

Maintaining Angular 14/15/16 codebases

You need the firstChange property (available in SimpleChanges)

You need previousValue to compare old vs new

Working with @Input() decorators (not input() functions)


When signal inputs + effect/computed are better:

New Angular 17+ projects

You have multiple derived values (computed() is cleaner than ngOnChanges)

You want fine-grained reactivity per input rather than one callback for all

You're building a fully signal-based architecture
```

Practical rule to take away:

> "ngOnChanges is not deprecated and remains fully supported -- you'll use it in every pre-17 Angular codebase you encounter. For new Angular 17+ projects, prefer input() functions -- they give you Signals that integrate with computed() and effect(), making the reactive relationship between inputs and derived state explicit and automatic. The key architectural difference: ngOnChanges is you checking a letter, input() signals are a live board that derived formulas watch automatically. Both work -- one requires less manual plumbing."

---

12. Related Topics to Learn Next

Angular-Specific Concepts

- Services & Dependency Injection (Topic 27 -- the next topic -- services are the alternative to @Input() for deeply nested data sharing)*
- Angular Template Syntax (Topic 28 -- property binding [prop], event binding (event) -- the syntax that makes @Input() and @Output() work in templates)*
- Angular Reactive Forms (Topic 33 -- forms receive data via @Input() and emit via @Output() -- the pattern repeated everywhere)*
- ViewChild & ElementRef (Topic 41 -- accessing child component instances directly -- the alternative to @Output() for reading child state)*
- Content Projection (ng-content) (Topic 40 -- a different kind of "input" -- projecting template content into a component)*

---

13. Updated Learning Tracker

```
Topics 1 19 ES6/JS + TypeScript Foundation COMPLETE

F1 F5 Angular Foundations Overview COMPLETE

Topic 20 RxJS Subscription Management COMPLETE

Topic 21 takeUntilDestroyed &amp;amp; DestroyRef COMPLETE

Topic 22 async pipe COMPLETE

Topic 23 Angular Signals COMPLETE

Topic 24 OnPush Change Detection COMPLETE

Topic 25 Component Lifecycle Hooks COMPLETE

Topic 26 @Input() &amp;amp; @Output() in depth COMPLETE

Topic 27 Services &amp;amp; Dependency Injection NEXT


TIER 1 REMAINING:

27. Services &amp;amp; Dependency Injection


TIER 2 UPCOMING:

28 36. Template Syntax HTTP Interceptors
```

```
TIER 3 WHEN NEEDED:

37 46. Directives RxJS Operators
```

Key Concept Added to Quick Reference
Topic 26 -- @Input() & @Output() in depth

> @Input() = USB port -- data pushed in from parent, treat as read-only. @Output() = speaker -- signals sent out when something happens. Copy before modifying. ngOnChanges not ngOnInit. Parent controls data; child controls events.

* @Input({ required: true }) = Angular 16+ -- enforced at build time
* @Input({ transform }) = auto-converts incoming value (numberAttribute, booleanAttribute)
* Always copy @Input() with spread before modifying -- never mutate directly
* ngOnChanges for @Input()-driven loading -- fires on EVERY change, not just first
* Type every EventEmitter -- new EventEmitter<User>() not new EventEmitter()
* input() = Angular 17+ -- returns Signal -- use with computed() and effect()
* output() = Angular 17+ -- simpler than EventEmitter, same template syntax
* Signal inputs + computed() = modern replacement for ngOnChanges pattern